



Startup with Ford SLP1

Version 9.0.1 for MICROSAR 4 Release 18

User Manual - A Step by Step Introduction

Content

1 About This Manual	11
1.1 History Information	11
1.2 Finding Information Quickly	11
1.3 Conventions	11
1.4 Certification	12
1.5 Warranty	12
1.6 Support	13
1.7 Registered Trademarks	13
1.8 Errata Sheet of Hardware Manufacturers	14
1.9 Example Code	14
1.10 What Do You Learn from This Manual	14
2 Basics	16
2.1 An Overall View	16
2.2 MICROSAR - Vector's AUTOSAR Solution	17
2.3 AUTOSAR Layer Model Ford	17
2.4 Configuration Workflow Ford SLP1	18
I STEPbySTEP	19
1 STEP1 Setup Your Project	20
1.1 Situation after Installation	20
1.2 Setup Project via DaVinci Configurator Pro	20
1.2.1 General Settings	21
1.2.2 Target	21
1.2.3 Project Folder Structure	21
1.2.4 DaVinci Developer	22
1.3 Result Project Folder - Result of the Project Setup	23
1.3.1 Appl folder	23
1.3.2 Config folder	23
1.3.3 <ProjectName>.dpa	24
1.3.4 Log Folder	24
1.4 Start Menu - Result of the Project Setup	25

1.5 DaVinci Configurator Pro Project	25
2 STEP2 Define Project Settings	26
2.1 Add Input Files	26
2.1.1 Add System Description Files	26
2.1.2 Add Diagnostic Data Files	27
2.1.3 Add Standard Configuration Files	29
2.1.4 Update Configuration	29
2.2 Define External Generation Steps and SWC Templates and Contract Phase Headers	30
2.2.1 External Generation Steps	30
2.2.2 SWC Templates and Contract Phase Headers	31
2.3 Activate Your BSW Modules	31
2.4 Change Project Settings	32
2.4.1 Postbuild Support	33
3 STEP3 Validation	34
3.1 Start Solve All Mechanism	34
3.2 Live Validation - Solving Actions	34
4 STEP4 Start BSW Configuration	36
4.1 Start Configuration with Configuration Editors	36
4.2 Base Services	36
4.2.1 Development Errors	36
4.2.2 General Purpose Timer (GPT)	36
4.2.3 Microcontroller Unit	36
4.2.4 RAM Test	37
4.3 Communication	37
4.3.1 Communication General	37
4.3.2 Bus Controller	37
4.3.3 PDUs	38
4.3.4 Signals	38
4.3.5 Socket Adapter Users	38
4.3.6 Transparent Diagnostic Gateway	38

4.4 Diagnostics	39
4.4.1 Diagnostic Event Data	39
4.4.2 Diagnostic Events	39
4.4.3 Production Error Handling	40
4.4.4 Setup Event Memory Blocks	40
4.4.5 FNOS Version Numbers	40
4.4.6 Importing OBD Freeze Frame Data in DaVinci Configurator Pro	41
4.5 Memory	41
4.5.1 Memory General	41
4.5.2 Memory Blocks	41
4.5.3 Optimize Fee	42
4.6 Mode Management Editors	42
4.6.1 BSW Management	42
4.6.2 Activate Interrupts of Peripherals Devices	44
4.6.3 ECU Management	46
4.6.4 Initialization	46
4.6.5 Watchdogs	47
4.7 Network Management	47
4.7.1 Partial Networking	48
4.8 Runtime System	48
4.8.1 Runtime System General	48
4.8.2 ECU Software Components	49
4.8.3 OS Configuration	49
4.9 Go on with Basic Editor	49
4.10 Start Solving Actions	50
4.11 Start On-demand Validation	50
4.12 BSW Configuration finished	51
5 STEP5 Design Software Components	52
5.1 Switch to DaVinci Developer	52
5.2 Design Software Components	52

6 STEP6 Mappings	53
6.1 Perform Data Mapping within DaVinci Developer or DaVinci Configurator?	53
6.2 Data Mapping within the DaVinci Developer	53
6.2.1 Data Mapping Automatically - DaVinci Developer	53
6.2.2 Data Mapping Manually - DaVinci Developer	54
6.2.3 DaVinci Developer - Save and close	55
6.3 Switch (back) to DaVinci Configurator	56
6.4 Synchronize System Description	56
6.5 Add Component Connection	56
6.6 Service Mapping	56
6.6.1 Service Mapping via Service Component	56
6.6.2 Service Mapping (overview)	58
6.7 Add Data Mapping	58
6.8 Add Memory Mapping	60
6.9 Add Task Mapping	61
6.9.1 Task Mapping Assistant	61
6.9.2 Task Mapping	62
7 STEP7 Code Generation	64
7.1 Generate SWC Templates and Contract Phase Headers	64
7.2 Start Code Generation	64
7.3 Generation Process finished!	66
8 STEP8 Add Runnable Code	67
8.1 Component Template	67
8.2 Implement Code	68
9 STEP9 Compile, Link and Test Your Project	69
9.1 Finish your project with compiling and linking	69
9.2 No error frames? Congratulations, that's it!	69
II Concept	71
1 General Overview	72
1.1 Software Component	75

1.1.1 Atomic components	75
1.1.2 Compositions	75
1.2 Runnables	75
1.3 Ports	76
1.3.1 Application Port Interfaces	76
1.3.2 Service Port Interfaces	76
1.4 Data Element Types	76
1.5 Connections	76
1.6 RTE	77
1.7 BSW – Basic Software Modules	77
1.8 Software, Tools and Files	77
1.9 Structure of the SIP Folder	79
2 Set-Up New Project	82
2.1 DaVinci Configurator	82
2.1.1 The Main Window of DaVinci Configurator Pro	82
2.1.2 Editors and Assistants	85
3 Define Project Settings	88
3.1 Input files	88
3.1.1 System Description Files	88
3.1.2 SYSEX	88
3.1.3 ECUEx	88
3.1.4 Legacy Data Base files (DBC, LDF, FIBEX, ...)	88
3.1.5 Diagnostic Data Files	89
3.1.6 CDD / ODX	89
3.1.7 State Description	89
3.1.8 Standard Configuration Files	89
3.2 External Generation Steps	89
3.3 Activate BSW	90
4 Validation	91
4.1 Validation Concept	91

5 BSW Configuration with Configuration Editors	92
5.1 DaVinci Configurator Pro Editors	92
6 Software Component (SWC) Design	93
6.1 Data Exchange between DaVinci Developer and DaVinci Configurator Pro	93
6.2 About Application Components, Ports, Connections, Runnables and More...	93
6.3 Application Components	94
6.3.1 The Library – Type and Package	94
6.3.2 New Application Components	99
6.3.3 Understand Types, Prototypes and Interfaces	103
6.4 Ports, Port Init Values and Data Elements	103
6.5 Configure Service Ports within your Application Components	108
6.6 Define your Runnables	109
6.7 Triggers for the Runnables	110
6.8 Port Access of the Runnables	112
7 Mappings	114
7.1 Data Mapping	114
7.2 Task Mapping	115
7.2.1 Information about Interaction between Runnable, Re-entrance and Task Mapping	116
7.3 Memory Mapping	118
7.4 Service Mapping	119
8 Generation	121
8.1 MICROSAR Rte Gen	121
9 Runnable Code	122
10 Compile and Link	124
10.1 Using your "real" hardware	124
III Additional Information	125
1 Update Input Files	126
1.1 System Description Files	126
1.2 Diagnostic Data Files	126
2 Update Project Settings	127

3 Support Request via DaVinci Configurator Pro	128
3.1 Result	128
4 Multiple User Concept	130
4.1 General	130
4.2 Split Files for Software Component and ECU Project Configuration	131
4.3 Configure Software Component Prototype	133
4.4 Configure ECU project	133
4.5 Split Files in DaVinci Developer	134
4.6 Split Files for BSW Configuration	134
5 Configuration Update	136
5.1 Release x-1 to Release x (necessary steps for any update)	136
5.2 Release 8 to Release 9	138
5.3 Release 7 to Release 8	138
6 Update DaVinci Tools	140
6.1 DaVinci Configurator Pro	140
6.2 DaVinci Developer	140
7 Non-Volatile Memory Block	141
7.1 Configure and use Non-Volatile Memory Block	141
7.2 Port Access of your Runnables	144
7.3 Memory Mapping in DaVinci Configurator Pro	144
7.4 Validate the RTE	146
8 Basic Software Modules	147
8.1 Generic BSW Modules	147
8.2 What is a BSW Module?	147
8.3 BSW Module Configuration	148
8.4 BSW Initialization	149
8.4.1 <Msn>_InitMemory()	149
8.4.2 <Msn>_Init()	149
8.5 BSW Module Version (xxx_GetVersionInfo)	149
8.6 Cyclic Calls	149

8.6.1 <Msn>_MainFunction()	149
8.7 Service Functions	150
8.8 Critical Sections - Exclusive Areas	150
8.8.1 Memory Section	150
8.8.2 Switch <MSN> Modules Off	150
9 Variant Handling	151
9.1 Define Criterion and Variants	151
9.2 Add and Assign Input Files to Variants	153
9.3 Define Variance for BSW Modules	154
9.4 Configure and Validate BSW	155
10 Add Module Stubs from AUTOSAR definition	157
11 SIP Update	159
12 Project Migration	160
12.1 Migration Steps from Release 17 SIP to Release 18	160
12.2 Migration Steps from Release 16 SIP to Release 17	160
IV Appendix	161
1 Frequently Asked Questions	162
1.1 Problems with using two different DaVinci Configurator Versions on the same PC	162
1.2 Annotations for any parameter	162
1.3 Find Reference Container	164
1.3.1 How to find references using the [Find] dialog	164
2 Release Notes	165
2.1 Release 18	165
2.1.1 Required AUTOSAR Tools	171
2.2 Release 17	171
2.2.1 Required AUTOSAR Tools	179
2.3 Release 16	179
2.3.1 Required AUTOSAR Tools	185
3 What's New, What's Changed	186
4 Glossary	188

5 Index 200



1 About This Manual

1.1 History Information



Cross Reference

For detailed information about what's new, what's changed in current version, refer to section What's New, What's Changed? on page 186.

1.2 Finding Information Quickly

The user manual provides the following access help:

- > You can see in the footer to which version the user manual refers,
- > At the end of the user manual you will find an index to find information quickly,
- > Also at the end of the user manual you will find a glossary of the used technical terms

1.3 Conventions

In the following two charts you will find the conventions used in the user manual regarding utilized spellings and symbols.

Style	Style Utilization
bold	Blocks, surface elements, window- and dialog names of the software. Accentuation of warnings and advices. [OK] Push buttons in brackets File Save Notation for menus and menu entries
MICROSAR	Legally protected proper names and side notes.
Source Code	File name and source code.
<u>Hyperlink</u>	Hyperlinks and references.
<CTRL> + <S>	Notation for shortcuts.



Symbol	Symbol Utilization
	Here you can obtain supplemental information.
	This symbol calls your attention to warnings.
	Here you can find additional information.
	Here is an example that has been prepared for you.
	Instructions on editing files are found at these points.
	This symbol warns you not to edit the specified file.
	This icon indicates multimedia files like e.g. video clips.
	This icon indicates an introduction into a specific topic.
	This icon indicates text areas containing basic knowledge.
	This icon indicates text areas containing expert knowledge.
	This icon indicates that something has changed.

1.4 Certification

Vector Informatik GmbH has ISO 9001:2008 certification. The ISO standard is a globally recognized standard.

1.5 Warranty

We reserve the right to modify the contents of the documentation or the software without notice. Vector disclaims all liabilities for the completeness or correctness of the contents and for damages



which may result from the use of this documentation.

1.6 Support

Germany And all other countries, not listed here.	
☎ + 49 711 80670 3900 ✉ support@de.vector.com	
BR Brazil, Argentina, Bolivia, Chile, Ecuador, Colombia, Paraguay, Peru, Uruguay, Venezuela	CN China
☎ +55 11 5180 2350 ✉ support@br.vector.com	☎ +86 21 6432 5353 ✉ support@cn.vector.com
FR France	IN India
☎ +33 1 42 31 40 10 ✉ support@fr.vector.com	☎ +91 20 6673 6673 ✉ support@in.vector.com
IT Italia	JP Japan
☎ +39 2 678171 10 ✉ support@de.vector.com	☎ TOKYO +81 3 5769 6972 ☎ NAGOYA +81 52 238 5014 ✉ support@jp.vector.com
KR South Korea	Scandinavia Sweden, Denmark, Finland, Iceland, Norway
☎ +82 2 807 0600 ✉ support@kr.vector.com	☎ +46 31 764 76 00 ✉ support@se.vector.com
UK United Kingdom & Ireland	North America USA, Canada, Mexico
☎ +44 121 7887 902 ✉ support@uk.vector.com	☎ +1 248 449 9290, Option 2 ✉ support@vectorcantech.zendesk.com

1.7 Registered Trademarks

All brand names in this documentation are either registered or non registered trademarks of their respective owners.



1.8 Errata Sheet of Hardware Manufacturers



Caution! Vector only delivers software!

Your hardware manufacturer will provide you with the necessary errata sheets concerning your used hardware. In case of errata dealing with CAN, please provide us with the relevant erratas and we will examine whether this hardware problem is already known to us or whether to get a possible workaround.



Note

Because of many NDAs with different hardware manufacturers or because we are not informed about them, we are not able to provide you with information concerning hardware errata of the hardware manufacturers.

1.9 Example Code



Caution!

All application code in any of the Vector User Manuals is for training purposes only. They are slightly tested and designed only to understand the basic idea of using a certain component or set of components. Vector gives consent that you use it as a basis for developing your own code and modify it (Vector waives its copyright to forbid you the use), however, with regard to the fact that the code is designed for training purposes only and not for the use by you, usability of the code is not warranted and Vector's liability shall be expressly excluded in cases of normal negligence, to the extent admissible by law or statute. Of course you have to test the software developed on the basis of the code with diligent care before using the software.

1.10 What Do You Learn from This Manual

Use this document to

- > Set up your project in just a few steps
- > Learn how to use the Vector tools
- > Learn how to work with the AUTOSAR means like software components, runnables, etc.
- > Work with the Vector AUTOSAR Solution on a basic level
- > Get a feeling of what you have to do when you start working for your actual project

The document is split in two parts, a **STEPbySTEP** description and an **Expert Knowledge (Concepts)** section.

> **STEPbySTEP**

This section gives you a short overview how to set up your project in just a few steps. Use the step



by step description to start up with basic information and get your project basically running.

> Expert Knowledge

This section gives you a detailed information about working with Vector AUTOSAR Solution. It contains expert knowledge based on STEPbySTEP description sections and information about tool use and general AUTOSAR concepts.

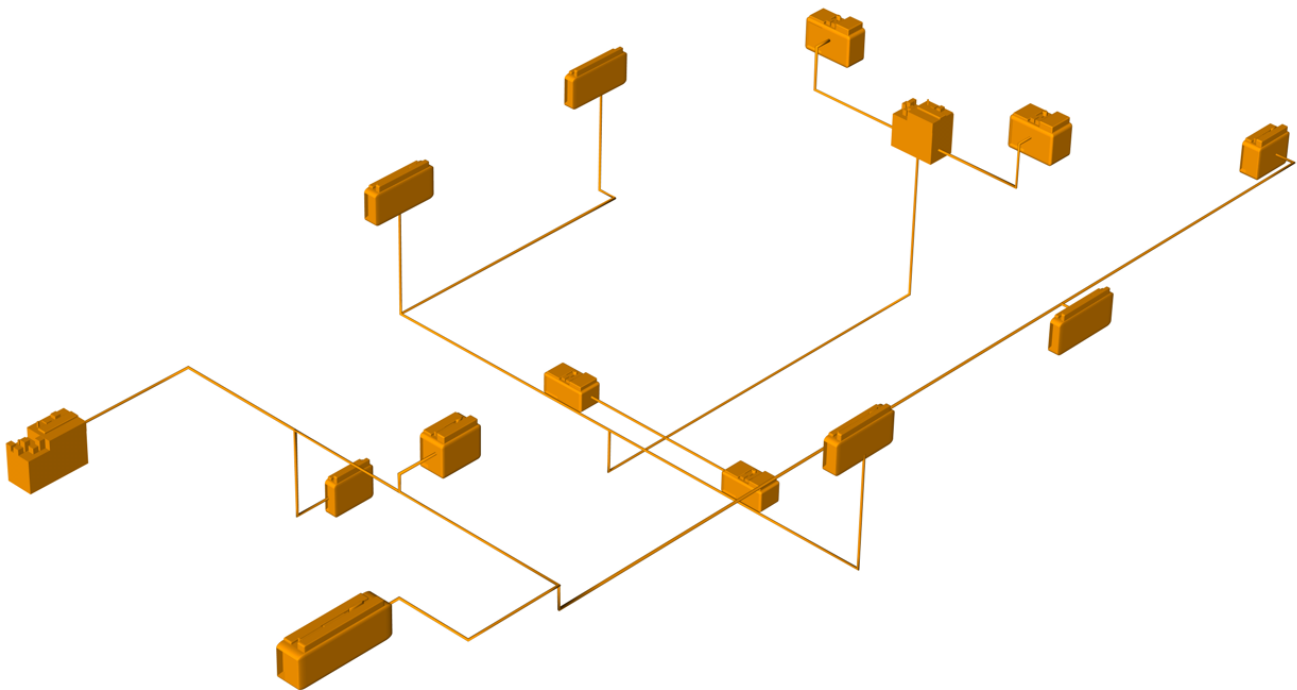


2 Basics

2.1 An Overall View

What we are talking about is an ECU, a module to be built-in a vehicle . The ECUs have to communicate with other ECUs and therefore almost every ECU participates in a bus system like e.g. CAN, FlexRay, LIN, MOST or IP. Any ECU within one bus system has to provide an identical interface to this bus system, because all ECUs have to share information via this bus system.

The illustration below shows an example of a CAN network that connects the ECUs of a vehicle.



All ECUs are built-up in a very similar way. There is a software part to perform the main job (application) of the ECU, e.g. to control the engine or a door. The other part handles the network communication and diagnostics.





2.2 MICROSAR - Vector's AUTOSAR Solution

The Vector MICROSAR Solution provides efficient and scalable basic software modules and an RTE for AUTOSAR ECUs.

The individual AUTOSAR basic software modules are grouped into MICROSAR products that of related functions. The configuration of the MICROSAR basic software with **DaVinci Configurator Pro** is simple, user-friendly, and consistent. The included command-line-based generators create optimized code for your ECU based on the configuration.

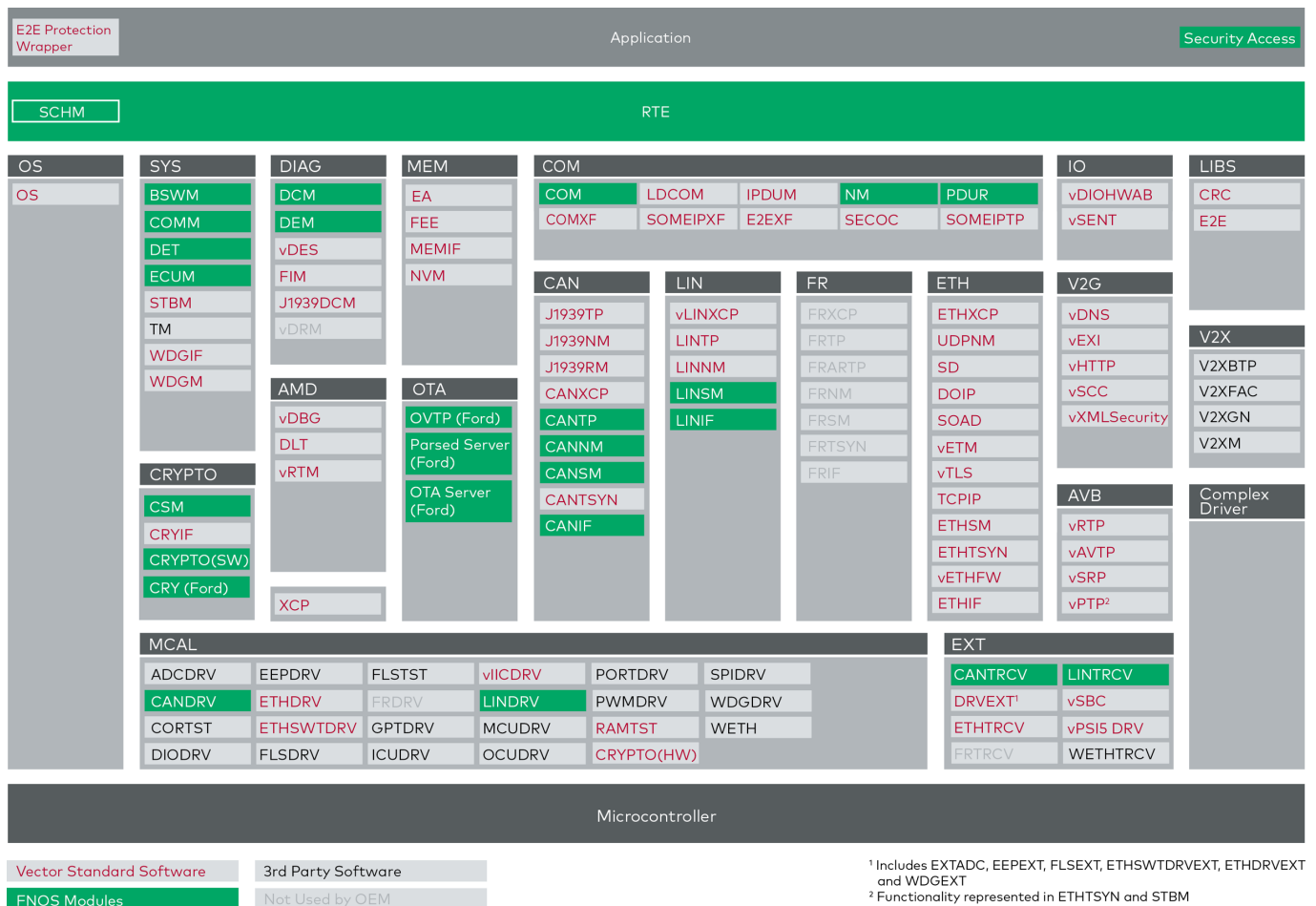


Cross Reference

To get more information about the MICROSAR concept refer to General Overview on page 72.

2.3 AUTOSAR Layer Model Ford

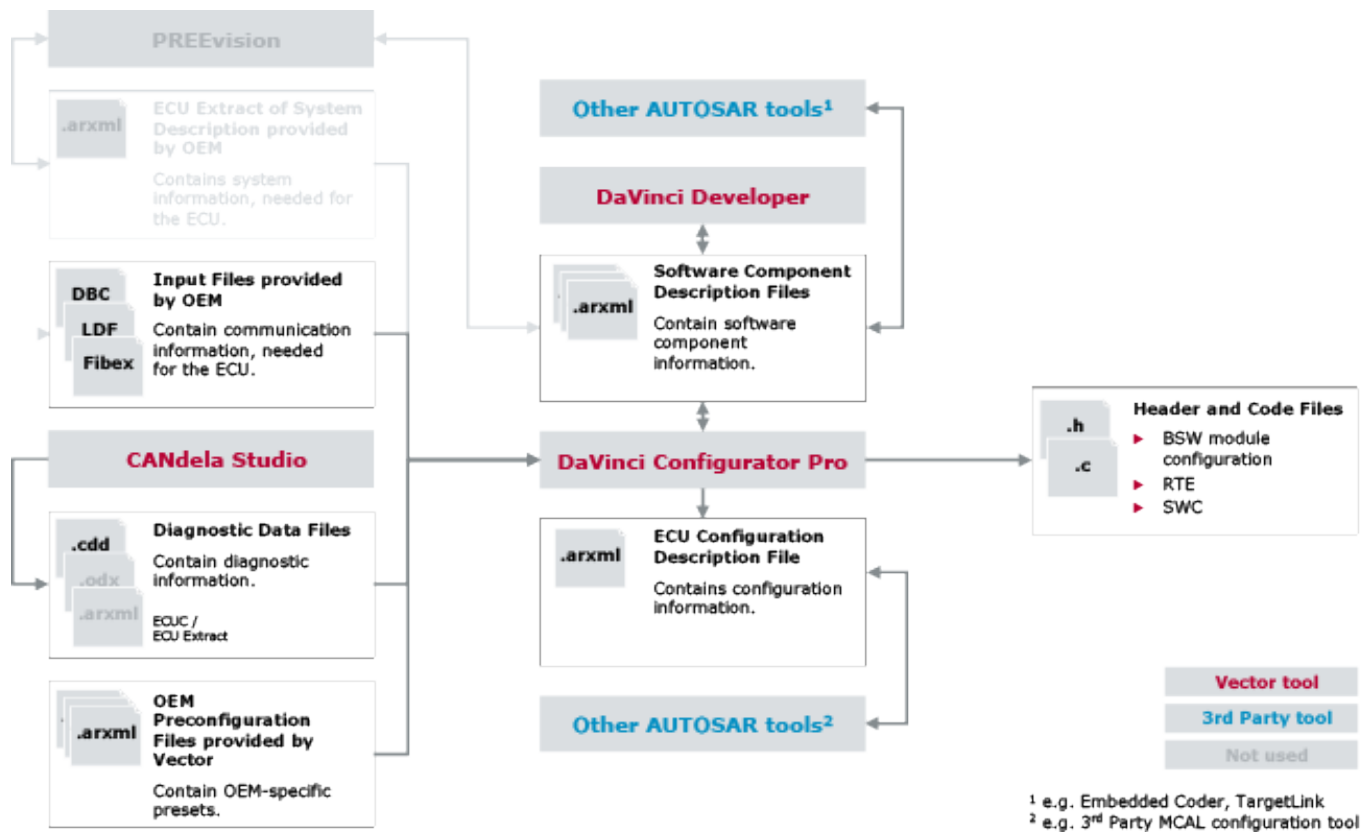
The following figure shows the AUTOSAR Layer model for Ford.





2.4 Configuration Workflow Ford SLP1

The following figure shows the configuration workflow for Ford SLP1:





I STEPbySTEP

This section shows you the few steps that are necessary to set up your project. Use it to start with basic information and get your project basically running.

- > **STEP1** Setup Your Project
- > **STEP2** Define Project Settings
- > **STEP3** Validation
- > **STEP4** Start BSW Configuration
- > **STEP5** Design Software Components
- > **STEP6** Mappings
- > **STEP7** Code Generation
- > **STEP8** Add Runnable Code
- > **STEP9** Compile, Link and Test Your Project



1 STEP1 Setup Your Project

This section includes all information for setting up a new project. It starts with the situation after installation and provides you with all information about relevant tools.



Expert Knowledge

You need to know more? See section New Project on page 82.

1.1 Situation after Installation

- > Before you start your first steps with Ford SLP1 check whether you have installed all necessary tools and software.
- > Have you read the **Installation Guide**? Have you received and installed all the necessary tools mentioned there?
- > Have you installed the MICROSAR SIP?

NO? Then please read the installation guide carefully and follow the installation guidelines given there.



Cross Reference

You will find the installation guide document at the Vector Knowledge Base:
<https://vector.com/kbp/entry/640/>

YES? You are ready to continue and to start with your first steps with the Ford SLP1 solution.

1.2 Setup Project via DaVinci Configurator Pro

Start the **DaVinci Configurator Pro** to set up your project.



Note

The DaVinci Configurator is located in your SIP Folder:
<SIPFolder> \DaVinciConfigurator\Core\DaVinciCFG.exe

1. Open **File|New**.
A set of four configuration windows will open;
General Settings,
Target,
Project Folder Structure,
Tools.
2. Enter the necessary information for each window and click **[Next>]** to get to the next window.



Note for No-Communication Projects

For projects without communication modules like e.g. DEM only, just confirm the windows **Target** and **Tools** as this information is not relevant.

Especially the **Target** window will show pull-down menus where **(undefined)** is preset and nothing else can be selected. This is not relevant for your use case and can be confirmed with **[Next]**.

1.2.1 General Settings

The following settings are required for project setup.

> SIP root path

The SIP root path will be defined automatically based on the **DaVinci Configurator Pro** location and cannot be changed by the user!

The **DaVinci Configurator Pro** is always located within the SIP.

> Project Name

The name of your project (e.g. MyECU). It will be used as name of the generated ECUC files and for the **DaVinci Developer workspace**.

> Project Folder

This is the root path of your project. Select an existing one or define a new one.

> Create entries in the start menu

The checkbox **Create entries in the start menu** is set as default. This is a comfortable feature that enables you to open your tools and project folder directly from the Windows start menu.

1.2.2 Target

Check **Derivative**, **Compiler** and **Pin layout** (automatically set, based on SIP information).

If you want to use **Postbuild-Loadable** or **Postbuild-Selectable** for your project, activate the necessary support option.

1.2.3 Project Folder Structure

Define your output paths here. You can use the given defaults or change paths to fit your project's needs.



Note

Default paths are based on the location of the defined project folder.

ECUC File Granularity

Additionally, you can specify whether the ECUC should be saved as one file (**Single file**) or is split into several files (**One file per module**).



Note

Split to several files is necessary to realize a multi user concept. Refer to [Multiple User Concept](#).

1.2.4 DaVinci Developer

> DaVinci Developer Workspace

If you have a RTE and you use the **DaVinci Developer** select the checkbox **Create DaVinci Developer Workspace** and define the path to the **DaVinci Developer** executable the path to the workspace.

> Automatic Update of DaVinci Developer Workspace

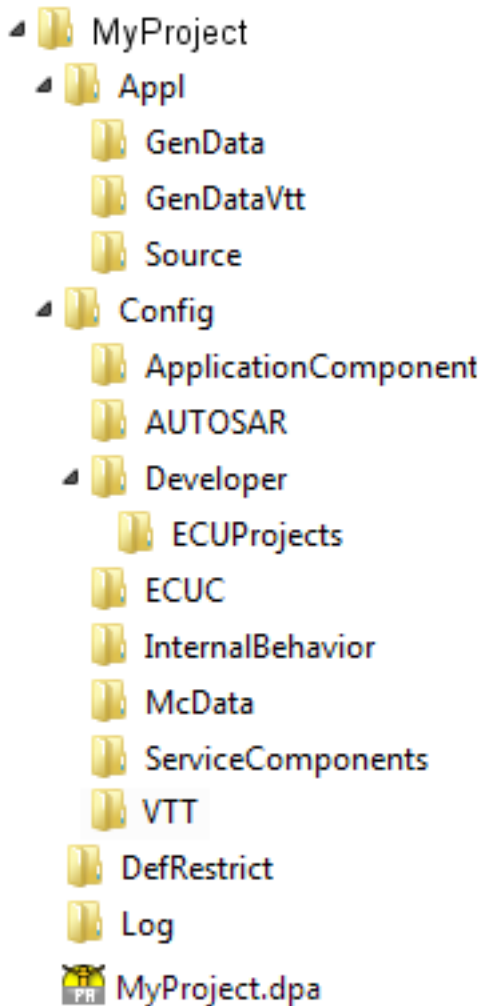
Option used for updating the **DaVinci Developer** workspace during the project update process. Details see **DaVinci Developer** help.

3. Click the button **[Finish]** to create your project.



1.3 Result Project Folder - Result of the Project Setup

The **DaVinci Configurator Pro** automatically creates and stores all relevant files to the project folder. The content of the folder looks like shown and described below.



1.3.1 Appl folder

The **Appl** folder contains the folders **GenData**, **GenDataVtt** and **Source**. In **GenData** the configuration tools generate their output files. If **vVIRTUALtarget** is used, the generated output will be stored in **GenDataVtt**. The **Source** folder is meant to carry your source code files.

1.3.2 Config folder

The **Config** folder contains the following folders:

> **ApplicationComponents**

This is a folder to put in additional SWC-Ts (Software Component Types). The **DaVinci Configurator Pro** reads in all *.arxml files of this folder.

> **Developer**

All relevant project data for **DaVinci Developer** is located in this folder.



> ECUC

This folder contains the created ECUC files. Before you add your input file(s), these files are only placeholders without content.

File Name	Description
 <ProjectName>.ecuc.arxml	ECU Configuration Description
 <ProjectName>.ecuc.Initial.arxml	Initial ECU Configuration Description for internal tool use only.

> InternalBehavior

This folder contains the bswmd files for all activated modules in the **DaVinci Configurator**.

> McData

Folder is used for storing measurement and calibration data.

> ServiceComponents

The **DaVinci Configurator** stores all generated Service Component Prototypes to this folder.



Note

After you have added your Input file(s) and updated the configuration in the **DaVinci Configurator**, the **System** folder will be added to the **Config** folder.

> System

The input file(s) and an additional created input file extract for communication use are stored to this folder.



Do not edit manually!

These files must not be modified manually!

> VTT

In case of activated Virtual Target option, the **vVIRTUALtarget** project file is located here.

1.3.3 <ProjectName>.dpa

Project File  <ProjectName>.dpa includes all relevant project data. Additionally the context menu of this file is the control center to open your tools with already loaded projects.

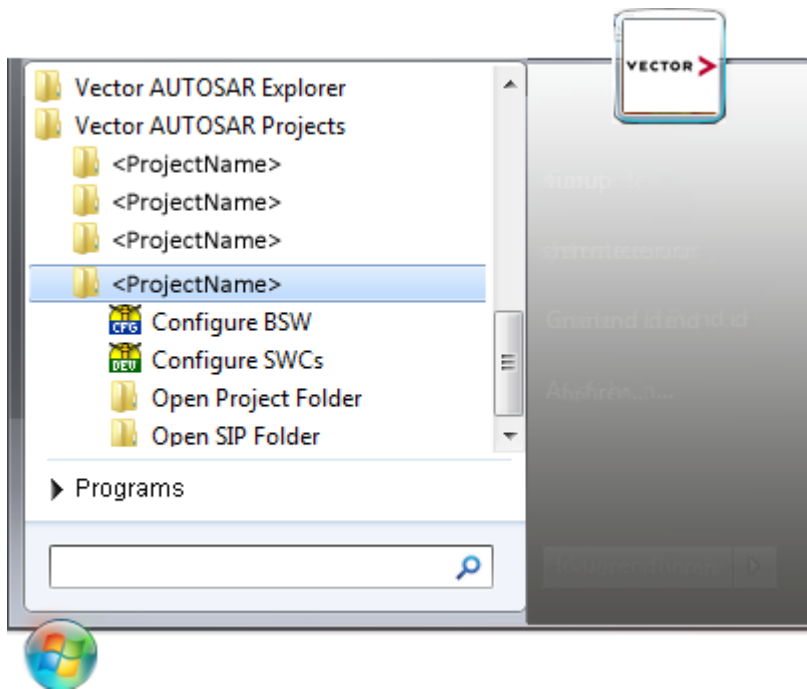
1.3.4 Log Folder

Folder for the generated log files.



1.4 Start Menu - Result of the Project Setup

The **DaVinci Configurator Pro** creates a link to start the tools with already loaded configuration. You will find this link in **Start | Programs | Vector AUTOSAR Projects | <ProjectName>** if **Create entries in the startmenu** was selected while project setup.



Info

You can also use the ***.dpa** file to start your project with already loaded configuration. Therefore right-click on your ***.dpa** file in the Windows Explorer and select **Configure BSW** to open the **DaVinci Configurator Pro** with already loaded configuration or **Configure SWCs** to open the **DaVinci Developer** for Software Design and Data Mapping.

If the Virtual Target option is enabled in the project, then **Open vVIRTUALtarget project** is additionally available.

1.5 DaVinci Configurator Pro Project

After finishing project set-up, a new empty project is loaded within the **DaVinci Configurator Pro**.



Note

Editors to configure modules will be available after input file import (see Add Input Files on page 26) or after module activation (see Activate Your BSW Modules on page 31).



2 STEP2 Define Project Settings

Before you start with the configuration it is necessary to define some project-specific data. This is the definition of project-specific data and the definition of all necessary input file(s) for communication and diagnostic purpose. Additional external generation steps and the activation of the necessary Basic Software Modules have to be done in the **DaVinci Configurator Pro** Project Settings view before.



Expert Knowledge

You need to know more? See section Project Settings on page 88.



Note

In case of OS only projects, skip the next sections **Add Input Files**, **External Generation Steps** and **SWC Templates and Contract Phase Headers** and go on with **Activate Your BSW Modules** on page 31.

2.1 Add Input Files

Open Input Files editor via **Project | Input Files** or use the **input file button**  of **DaVinci Configurator Pro** toolbar.

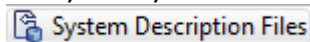


Note

Any change of the input files requires an update of the configuration. Add all relevant files before starting the update process.

2.1.1 Add System Description Files

Add your System Description File first. Therefore open the **System Description Files view**

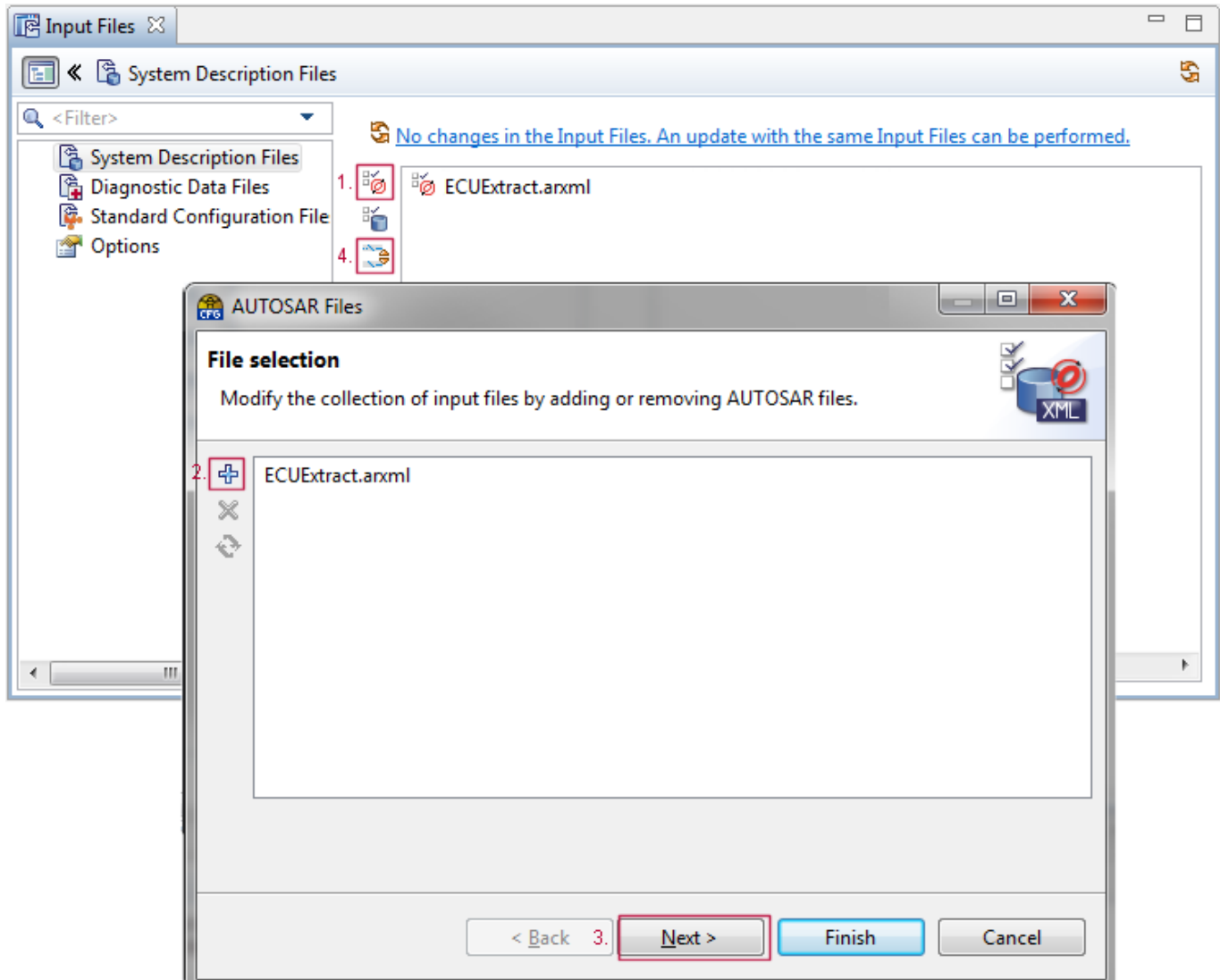


. First you have to decide if you add **AUTOSAR Files (arxml)**  or **Legacy Files (dbc, xml, ldf)** .



Note

The following section describes the import of AUTOSAR files. If you use legacy file as input, you have to perform nearly the same steps, only for 1. **Legacy Files button**  has to be selected.



1. Use the **AUTOSAR Files** button  to open the window where you can add your AUTOSAR files.
2. Add your Input Files via **[+]**
3. Go on with **[Next]** and select the **ECU** instance.
4. Make your input file path relative to your project folder via **Make Path Relative** button 

2.1.2 Add Diagnostic Data Files



Note - Diagnostic Extract as Input!

If your diagnostic input file is a **Diagnostic Extract**, then add this file as if it was a System Description file as described in the previous chapter.

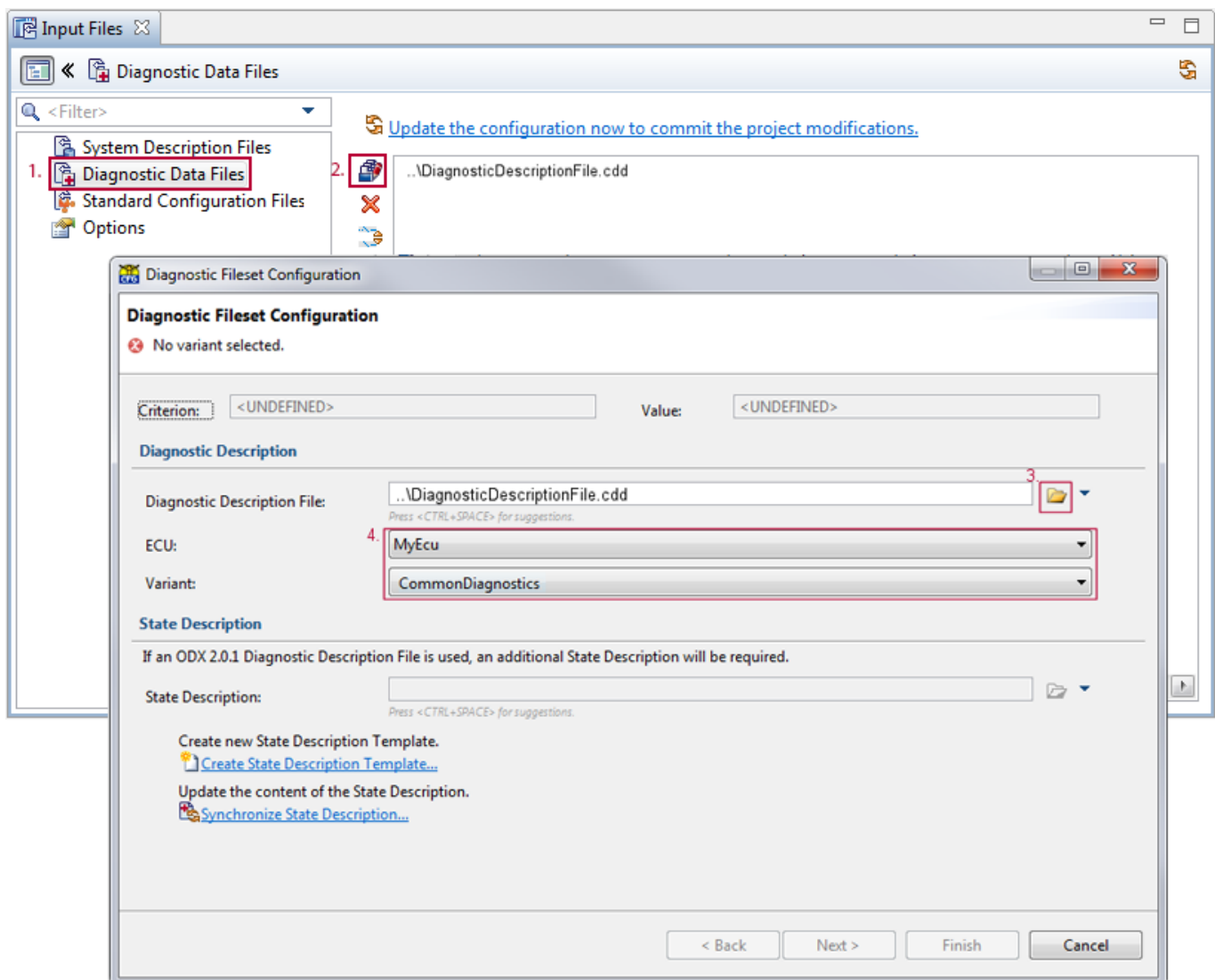
If diagnostic is used, you have to add a diagnostic data file. If not, you can skip this section. There are different use cases to import diagnostic data, decide if you import your data from a diagnostic data file or from an already existing configuration.



Use Case 1: Import Diagnostic Data from ODX, PDX or CDD File

Open your project in the **DaVinci Configurator** and go on with the following Steps to import your diagnostic data from a diagnostic data file. Open the Input Files editor via **Project | Input Files** or use the **input file button**  of **DaVinci Configurator Pro** toolbar

1. Open the **Diagnostic Data Files view**  in the Input File editor
2. Open the Diagnostic Fileset Configuration window via **Add, remove or modify diagnostic data button** 
3. Browse for the **Diagnostic Description File** (*.cdd/*.pdx/*.odx)
4. Select ECU and Variant



Use Case 2: Import Diagnostic Data From an Existing Configuration

Open your project in the **DaVinci Configurator** and go on with the following steps to import your diagnostic data from an existing Configuration.



1. Start Import Assistant via **File | Import**
2. Add **[+]** ECUC File and go on with **[Next]**
3. Select modules which should be imported and close the **Modul Configuration Import** dialog via **[Finish]**.

Make your input file path relative to your project folder via **Make Path Relative button** 

2.1.3 Add Standard Configuration Files

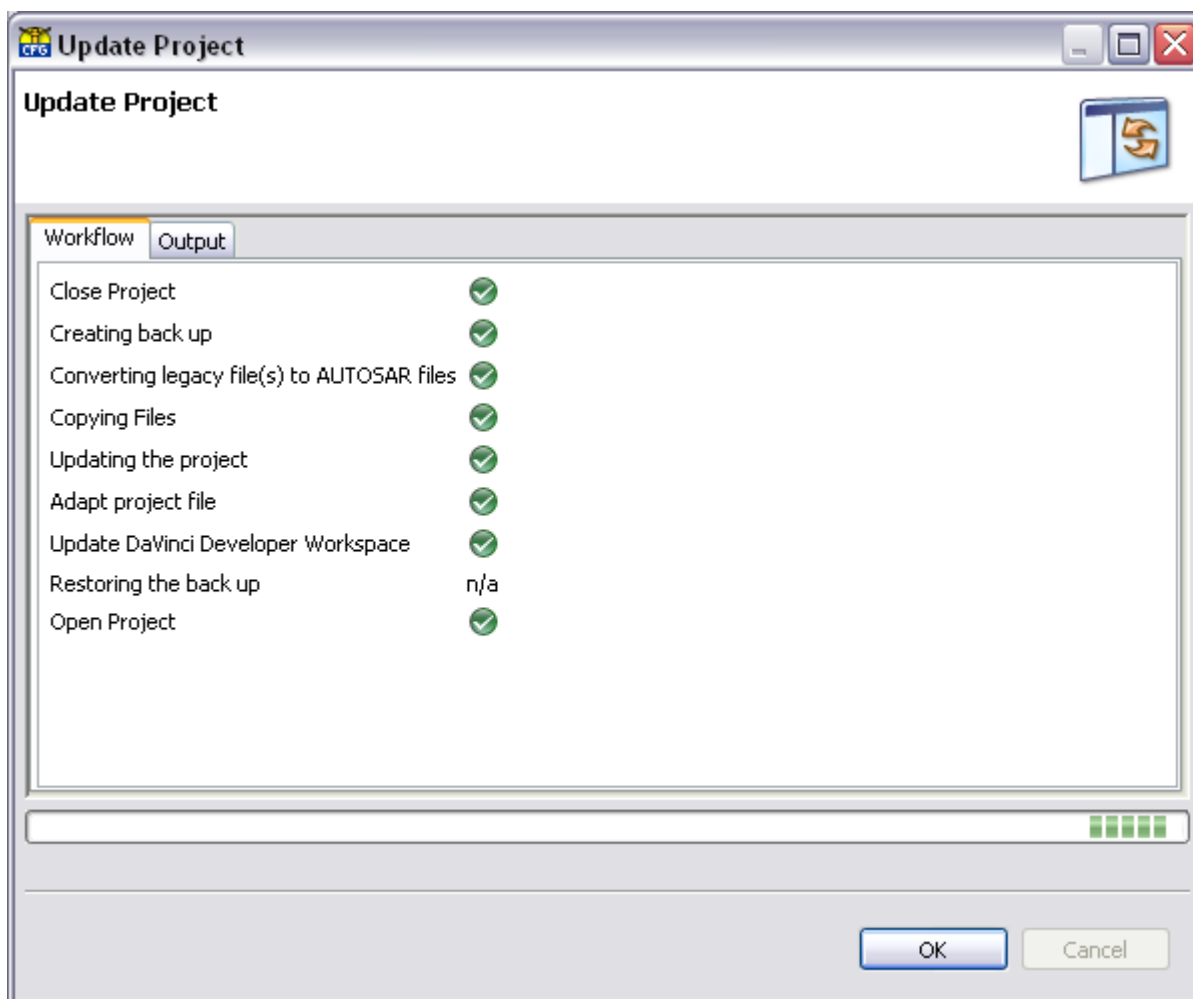
In some cases an OEM-specific preconfiguration is necessary, therefore add the Standard Configuration File provided by your OEM. Add fragments of ECUC modules which will be used as project specific mandatory configuration.

2.1.4 Update Configuration

Any change of the input files lead to Pending Update state of the project, this requires an update of the configuration.

Update the configuration via .

What happens during the configuration update process?





The project file will be updated based on loaded input/diagnostic files.



Note

After the first update/import of data bases all required modules are automatically enabled. This information is derived from the data base. If further modules are required or some shall be removed please refer to section Activate Your BSW Modules on page 31.

2.2 Define External Generation Steps and SWC Templates and Contract Phase Headers

To define external generation steps/SWC templates and contract phase headers, open **Project Settings Editor** via **Project | Project Settings** or use the **project settings icon**  of **DaVinci Configurator Pro** toolbar.

2.2.1 External Generation Steps

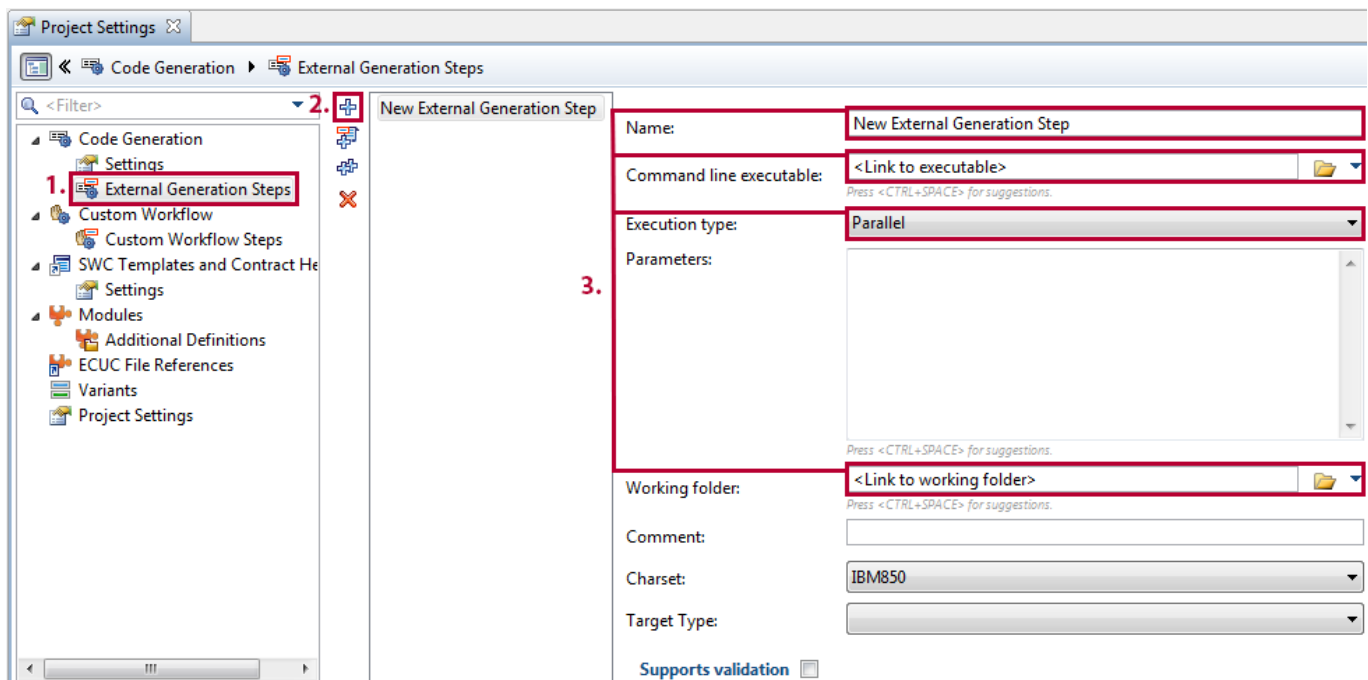
You have external software that needs to be started during the code generation of DaVinci Configurator? Or just before? Or afterwards?

Then link your software to the **External Generation Steps** of **DaVinci Configurator Pro**.

How? Follow these steps.



1. Select **Code Generation | External Generation Steps**
2. Add **[+]** step
3. Define
Name,
Command line executable,
Execution Type and
Working folder.



2.2.2 SWC Templates and Contract Phase Headers

To generate software component templates and/or contract phase headers perform the following steps:

1. Select **SWC Templates and Contract Headers | Settings** and choose the generation mode
2. Go to **SWC Template and Contract Headers** and tick the checkbox of the software components for which you want to generate SWC templates and/or contract phase headers



Cross Reference

For the generation of the SWC templates and contract phase headers please refer to STEP7 Code Generation on page 64.

2.3 Activate Your BSW Modules

Before you start configuring your Basic Software, you have to activate all modules you need for your project. Therefore open **Project Settings Editor** via **Project | Project Settings** or use **project settings icon**  at **DaVinci Configurator Pro** toolbar.

Select **Modules** to see all modules currently activated for your project.



Info

It depends on your Input File which modules are loaded initially.

You need additional modules?

1. Click **[+]** and select the source of the module definition.
2. Select the module you would like to add to your configuration and confirm with **[OK]**.



Info

Multiple selection is possible for activating or deactivating modules.

Not all modules are needed?

To delete not used modules, select the module and delete it via **[x]**.



Caution!

It is not recommended to delete initially activated modules from project!

You have a completely configured module and would use this for the current configuration?

1. Make sure, that the module you would like to import and use it, is deactivated in your current project.
2. Select **File | Import**
3. Add **[+]** ECUC file including the configured module
4. Click **[Next]**
5. Select the module configurations to be imported



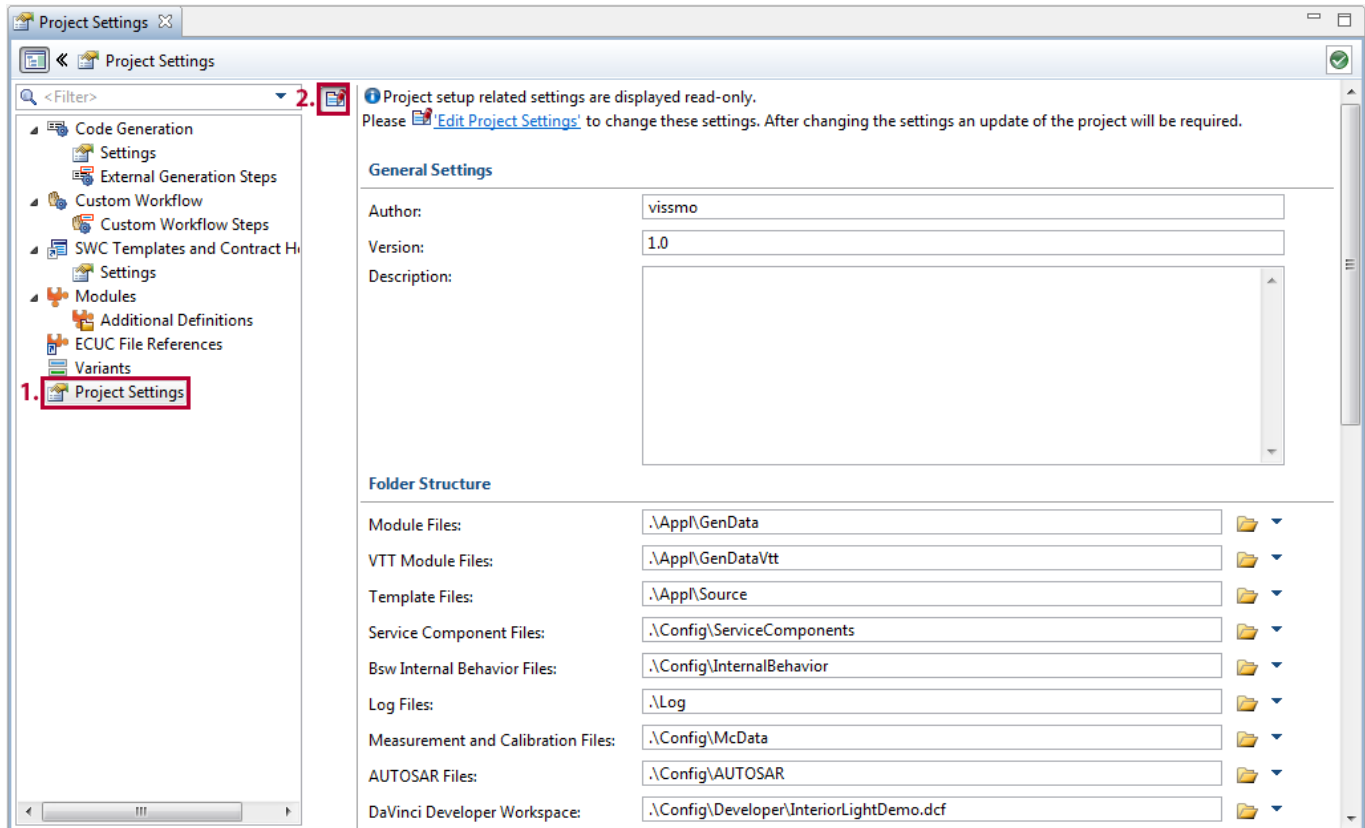
Note

It is not allowed to import BSW modules which are currently activated.

6. Click **[Finish]**

2.4 Change Project Settings

In some cases it is necessary to change project settings, which was defined while project setup. Therefore select **Project Settings** and activate editing via **Edit Project settings icon** .



2.4.1 Postbuild Support



Caution!

These parameters should be changed for compelling reasons only. Parameter changes, require a lot of manual adjustment after project update!



Note

In case of project migration, the parameters will be set automatically.



3 STEP3 Validation

Start initial validation process to solve the first warnings and errors.

The **DaVinci Configurator Pro** provides validation routines that run in the background and are called Live Validation. At the beginning of the configuration, the project is incomplete and much information in the system is still missing. Therefore the **DaVinci Configurator Pro** offers a powerful solving algorithm.



Expert Knowledge

You need to know more? See section Validation on page 91.

3.1 Start Solve All Mechanism

First of all start solving mechanism via the **Solve All button**  at the validation view of the **DaVinci Configurator**.



Note

Not all errors can be fixed with **Solve All** mechanism.

Some errors might have two or more solutions, these errors cannot be fixed automatically and need your decision.

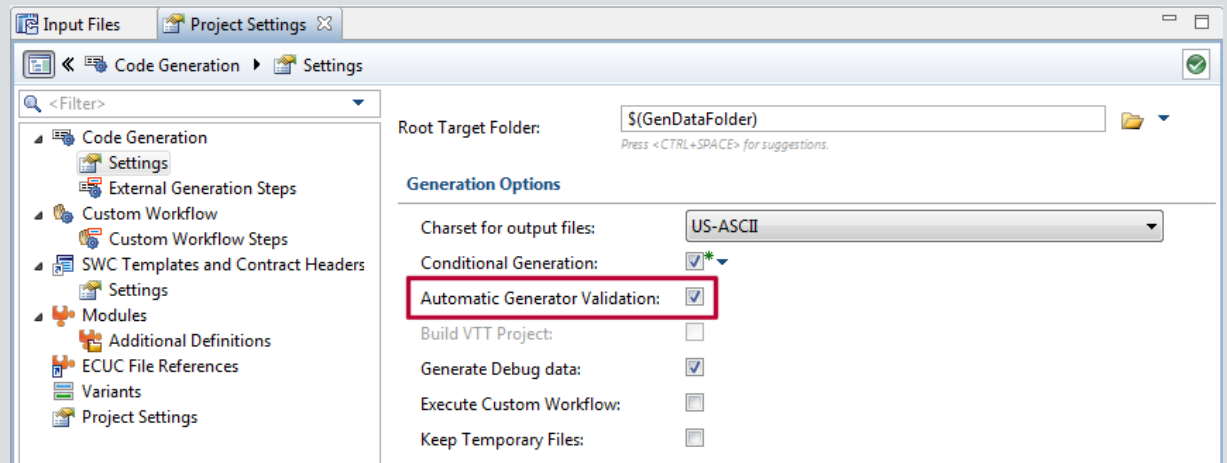
3.2 Live Validation - Solving Actions

The **DaVinci Configurator Pro** provides a live validation during project configuration. For some parameter errors the live validation process offers possible solution(s) in the validation view of the **DaVinci Configurator Pro**. These errors are marked with a bulb.



Note 3rd Party Module Live Validation

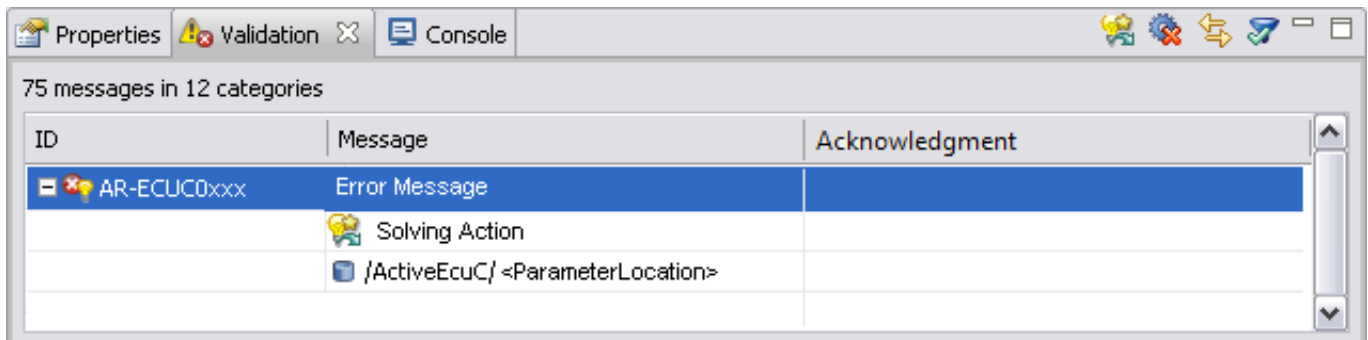
If **Automatic Generation Validation** is activated in **Project Setting Editor**, the configuration of 3rd party modules is also checked while live validation.



Check if the suggested solving action is correct. Double click the error message to open the configuration window of the parameter.

Solving Action is correct?

Start the solving action via double click on the solving action entry in the validation window.



Note

If the solving action is not correct, you can additionally use the link in the solving action to navigate to the parameter and configure the changes manually.



Cross Reference

You find more information about the feature Acknowledgment in the help of the **DaVinci Configurator Pro**.



4 STEP4 Start BSW Configuration

Start the configuration of the BSW modules now. The target of this first description is a basically working configuration, using a minimal necessary configuration.



Expert Knowledge

You need to know more? See section Configuration with Configurator Editors on page 92.



Note

Changes in Configurator Editors could lead to errors during live validation!

If these errors include solving actions, check and start solving action in the validation view

4.1 Start Configuration with Configuration Editors

The configuration editors are use case-oriented and optimized for displaying or configuring those parts of the ECU configuration, which are related to the use case.

4.2 Base Services

The Base Services domain contains configuration editors for the configuration of the development error reporting, general purpose timer channels, microcontroller units and RAM test algorithms.

4.2.1 Development Errors

Open **Development Errors | Enable or Disable Error Tracing** and select necessary modules or domains to enable the development error tracing.

4.2.2 General Purpose Timer (GPT)

Use default settings.

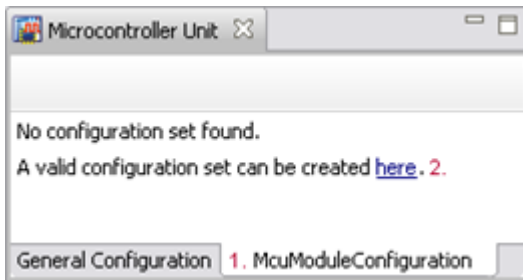


Note

If the editor is deactivated (greyed out), you can activate the General Purpose Timer (GPT) module via click. The Modules Assistant opens to specify the short name of the module to be added. Click finish to add the module.

4.2.3 Microcontroller Unit

1. Open tab McuModuleConfiguration
2. Create new configuration via **[here]**.



3. Open **Clock Config Sets | McuClockSettingConfig | McuClockReferencePoint**, add [+] an **McuClockReferencePoint** and define **Clock Reference Point Frequency**

4.2.4 RAM Test

Some hardware-specific settings are necessary.



Note

If the editor is deactivated (greyed out), you can activate the RAM Test module via click. The Modules Assistant opens to specify the short name of the module to be added. Click finish to add the module.

4.3 Communication

The Communication domain contains configuration editors for the configuration of general parameters of the communication related modules, ECU Bus Controllers, protocol data units, data signals and the transport protocol.



Note

The following setting description are meant for demonstration purposes only and can differ from the settings, that are necessary for your project.

If there are modules describe, you do not have in your project, skip this chapter.

4.3.1 Communication General

Define central settings of the communication based modules.

CAN

1. Open **CAN | Miscellaneous**
2. Set [...] parameter **Counter Ref** to **SystemTimer**.
3. Set **Interrupt Category** to **CATEGORY 2**
4. Set **Interrupt Lock** to **DRIVER**

4.3.2 Bus Controller

1. Open the **CAN Controllers | <CANController>**



2. Define [...] Controller Default Baudrate
3. Define [...] CPU Clock Ref



Note

If no CPU Clock Ref is available, add a new reference via **[+]** in the Select Reference Target view.

4.3.3 PDUs

Configure the PDUs of the modules (depends on your project)

- > CAN/LIN
- > CANIF/LINIF
- > CANTP/LINTP
- > PDUR
- > IPDUM
- > COM

4.3.4 Signals

Configure the communication signals of the ECU.

4.3.5 Socket Adapter Users

Use default settings.



Note

If the editor is deactivated (greyed out), you can activate the SOAD module via click. The Modules Assistant opens to specify the short name of the module to be added. Click finish to add the module.

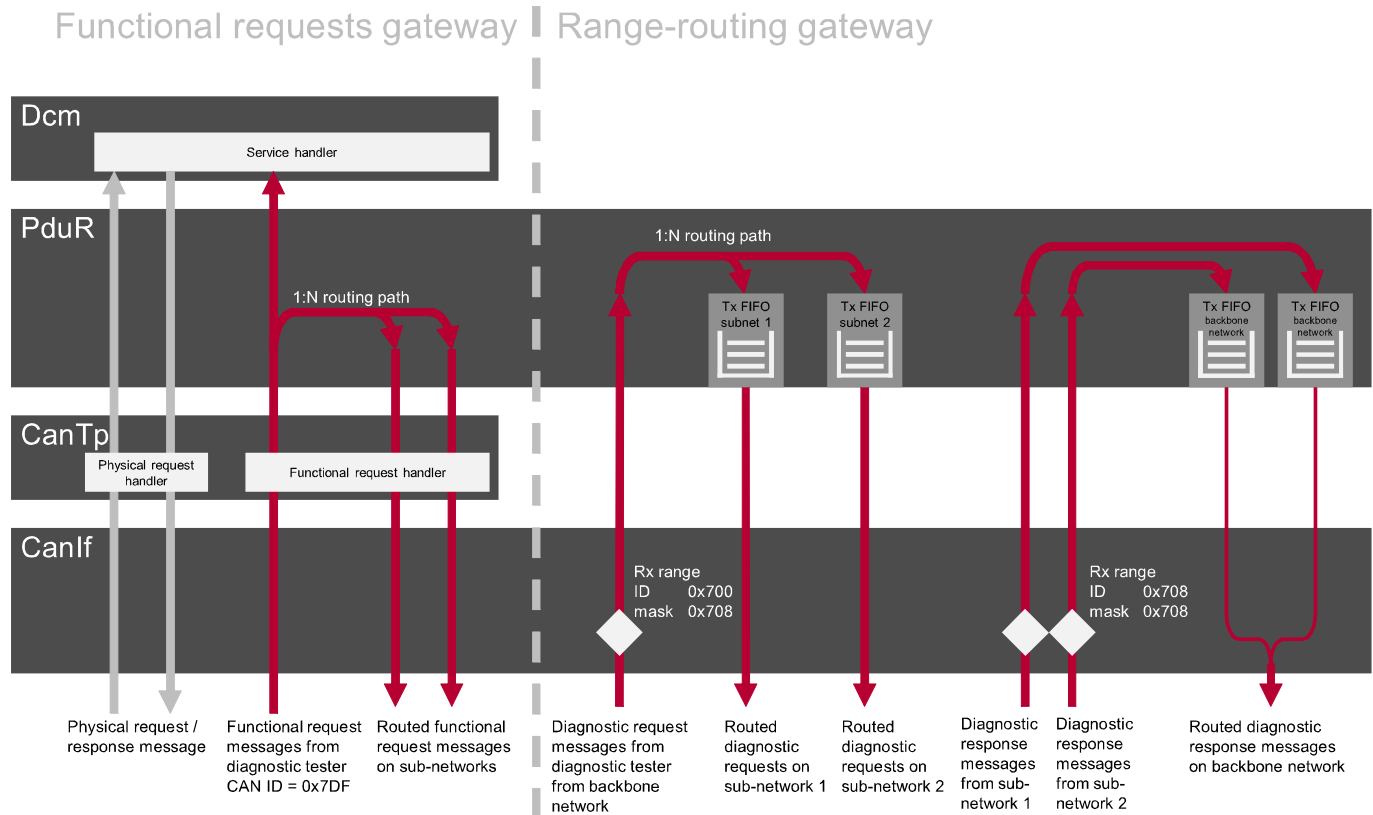
4.3.6 Transparent Diagnostic Gateway

For the configuration of the transparent diagnostic gateway feature, some additional manual configuration steps are necessary.

This gateway functionality can be realized by the following two gateway routing methods

- > **Range-routing gateway routing**
Used for routing of the diagnostic request and response messages, not addressed to the gateway-own diagnostic handler
- > **Functional requests gateway routing**
Used for broadcasting of the functional diagnostic request messages addressed to ECUs including the gateway-own diagnostic handler

The architecture of the transparent diagnostic gateway is visualized in the following figure.



Cross Reference

The detailed step-by-step configuration description of the two gateway routing methods can be found in the PduR technical reference document (TechnicalReference_PduR.pdf):

- > Use Case Configuration: Communication interface range gateway
- > Use Case Configuration: Functional requests gateway routing



Note

As of November 2016, only standard IDs are used by Ford. The use of 29-bit IDs is planned.

4.4 Diagnostics

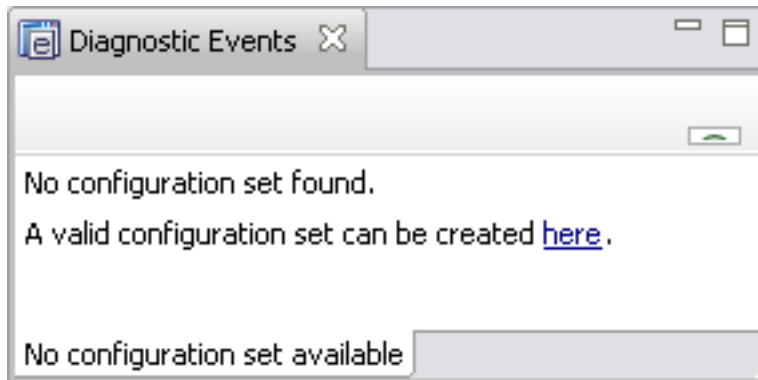
The Diagnostic domain contains comfort editors for the configuration of extended data records and snapshot records, diagnostic events and production error handling of the individual BSW modules and an automatic setup of the NV memory blocks.

4.4.1 Diagnostic Event Data

Use default settings as these have been derived from input files.

4.4.2 Diagnostic Events

If no configuration set is found, use the hyperlink **here** to create a valid configuration set.



4.4.3 Production Error Handling

Use default settings.

4.4.4 Setup Event Memory Blocks

The event memory block configuration needs to be updated. Start the Editor and confirm with **[finish]**.



Note

The automatic setup of the NV memory blocks is required for diagnostic purpose.

4.4.5 FNOS Version Numbers

The version number of the FNOS software (aka package ID, DID F15F) and the NOS message database version number(s) (DID F166/ DID F167) have to be readable via diagnostics. The values of the dbc attributes

- > **VersionDay,**
- > **VersionMonth,**
- > **VersionNumber,**
- > **VersionYear**

are generated as internal Rx signals such that the application can access those values using the RTE (or the Com). You can find the signals in Rx PDUs starting with **FileVersion**.

Parts of the package ID (DID F15F = NOS Generation Tool Version Number) are also generated from the **delivery ID**, which can be found in the delivery description.



Note

The **delivery ID** (10 Byte number) is set up in the following way

17.00.00.01.40.04.12.00.00.00

<xx.yy.zz><0><DeliverySubProjectNumber><DeliveryNumber><00.00>

xx.yy.zz are the FNOS release version numbers:

xx= major version,

yy= minor version,

zz= revision.

The DeliverySubProjectNumber (aka CBD number) and the DeliveryNumber are part of the **NOS Generation Tool Version Number**.



Note

The **NOS Generation Tool Version Number** is set up in the following way:

08.09.00.01.40.04.12.00.00.00

<08.09.00><0><DeliverySubProjectNumber><DeliveryNumber><00.00>

These values are also generated as internal Rx signals. Their names start with **DeliverySubProjectNumber/DeliveryNumber**.

4.4.6 Importing OBD Freeze Frame Data in DaVinci Configurator Pro

In the diagnostic description file the OBD Freeze Frame PIDs are configured at service \$19 05. If service \$19 05 is active in the description file the Diagnostic Importer of Cfg5 will import the OBD freeze frame data and activate automatically the OBD services \$01 and \$02 for the DCM module.

4.5 Memory

The Memory domain contains comfort editors for the configuration of the general parameters of the memory related modules, the memory block in the non volatile memory blocks.

4.5.1 Memory General

If FEE is used, define

- > **BSS Thresholder Reserved**
BSS - Background Sector Switch
- > **FSS Thresholder Reserved**
FSS - Foreground Sector Switch

4.5.2 Memory Blocks

Open **Memory Partitions | PartitionConfiguration** and define



> Partition Device

Add the reference to their device of the partition



Note

If the editor is deactivated (greyed out), you can activate the EA/FEE module via click. The Modules Assistant opens to specify the short name of the module to be added. Click **[finish]** to add the module.

4.5.3 Optimize Fee

If Fee is used, start FEE Optimization. The number of chunk instances (parameter FeeNumberOfChunkInstances) of the Fee blocks will be optimized. Please review the new value or select "skip" to leave the parameter.



Note

If the editor is deactivated (greyed out), you can activate the FEE module via click. The Modules Assistant opens to specify the short name of the module to be added. Click [finish] to add the module.

4.6 Mode Management Editors

The Mode Management domain contains comfort editors for the configuration of BSW Management, ECU Management, initialization and restart sequences of the BSW modules, sleep modes and wakeup sources, supervised entities and watchdog modes.

4.6.1 BSW Management

Auto Configuration: Ecu State Handling

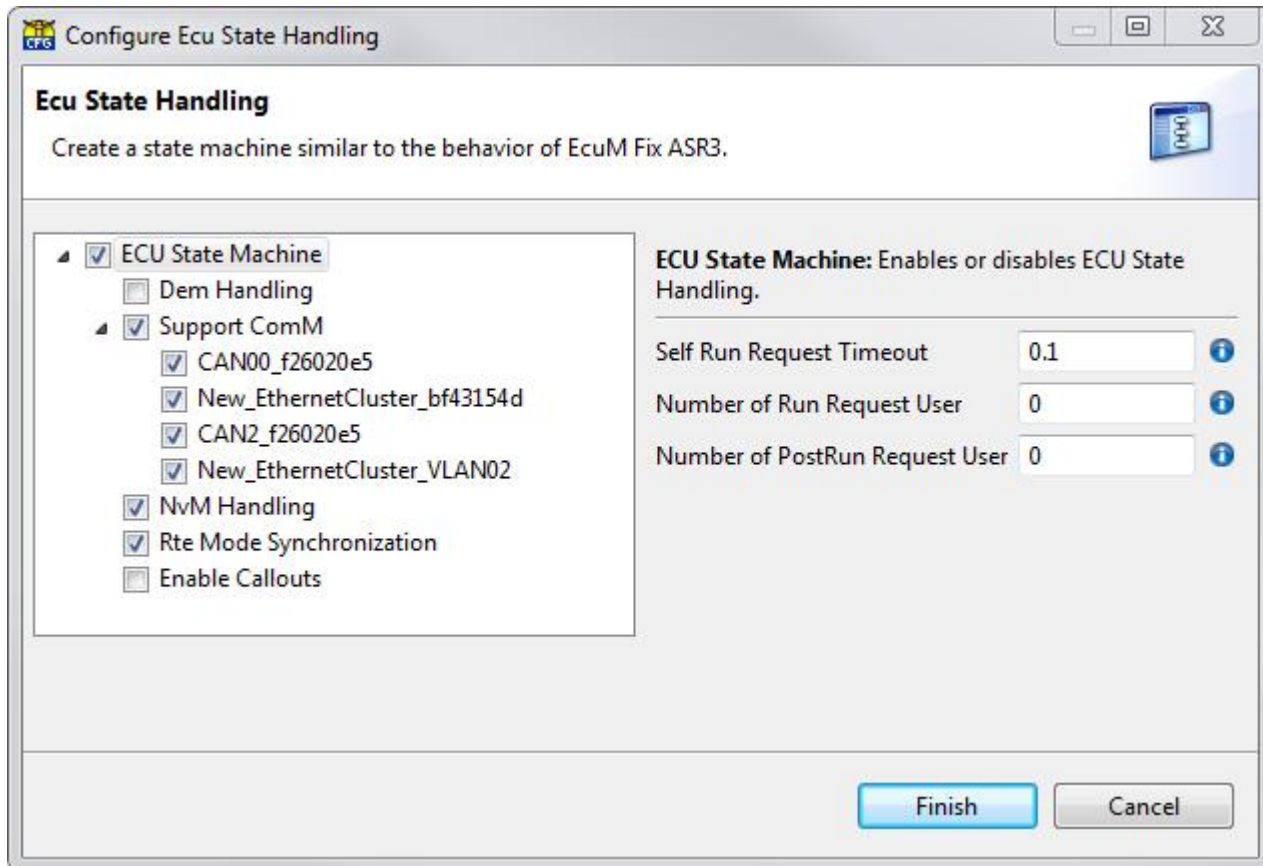
Open Auto Configuration: Ecu State Handling and configure Ecu State Handling settings via **Configure**

Ecu State Handling hyperlink



[Configure Ecu State Handling](#)

. The Configure Ecu State Handling dialog opens.



Select the necessary use case definitions to create a state machine similar to the behavior of EcuM Fix in AUTOSAR3.

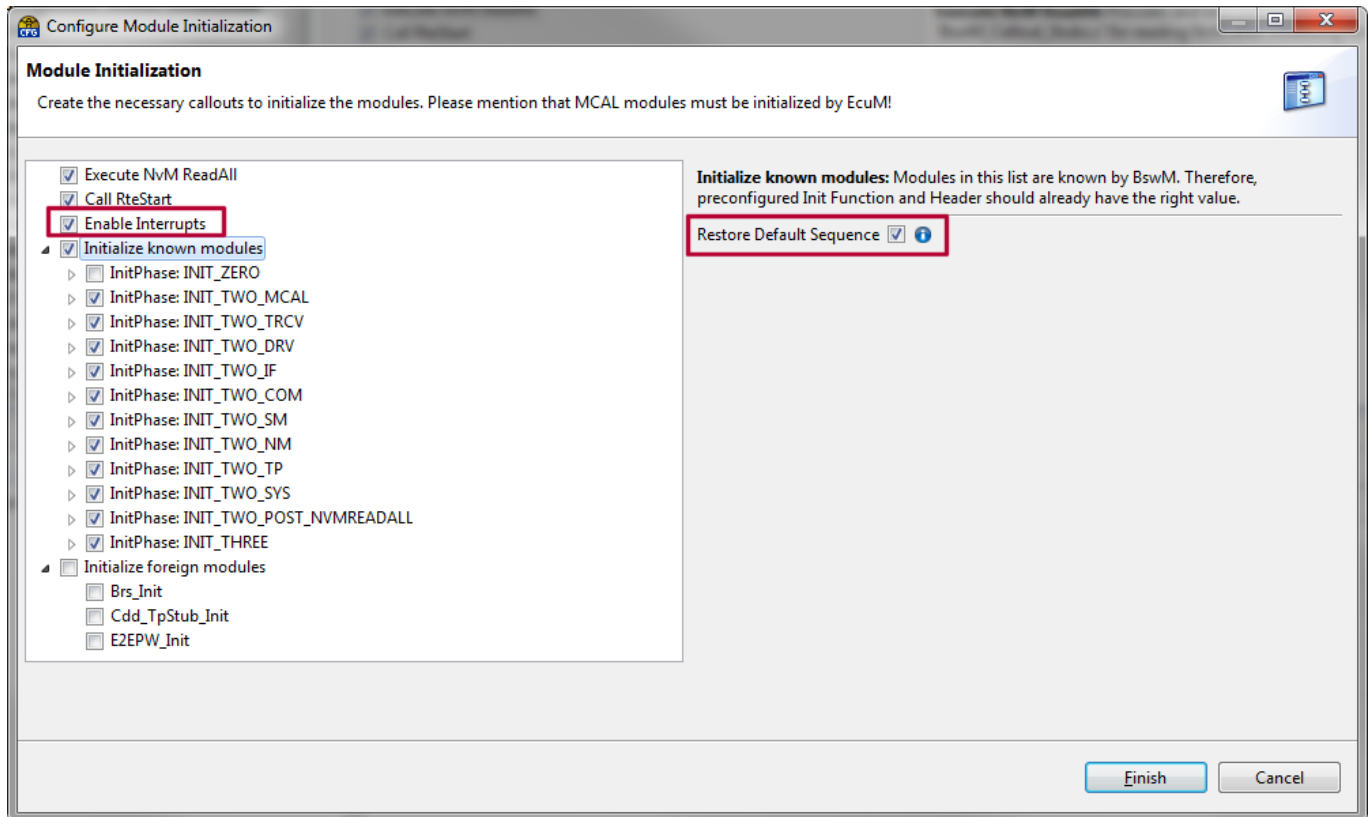
**Note**

If you select Ecu State Machine, you have to define the project-specific settings **Self Run Request Timeout** and **Number of Run Request User**.

Auto Configuration: Module Initialization

Open Auto Configuration: Module Initialization and configure Module Initialization via **Configure Mod-**

ule Initialization hyperlink  [Configure Module Initialization](#). The Configure Module Initialization dialog opens.



Make sure to set **Enable Interrupts** and **Restore Default Sequence**. For the latter setting you have to activate **Initialize known modules**.

Select module to create the necessary callouts to initialize the modules.

4.6.2 Activate Interrupts of Peripherals Devices

Activate **Enable Interrupts** to get the callout `BswM_AL_SetProgrammableInterrupts`. Use this callout to activate interrupts of the peripheral devices. Exceptions from this rule are explained in the technical reference of the drivers.



Note

Configure module initialization at initialization view.



Note

If **Restore Default Sequence** is activated, the lists will be sorted automatically, all new entries will be located at the end of the list.

Auto Configuration: Communication Control

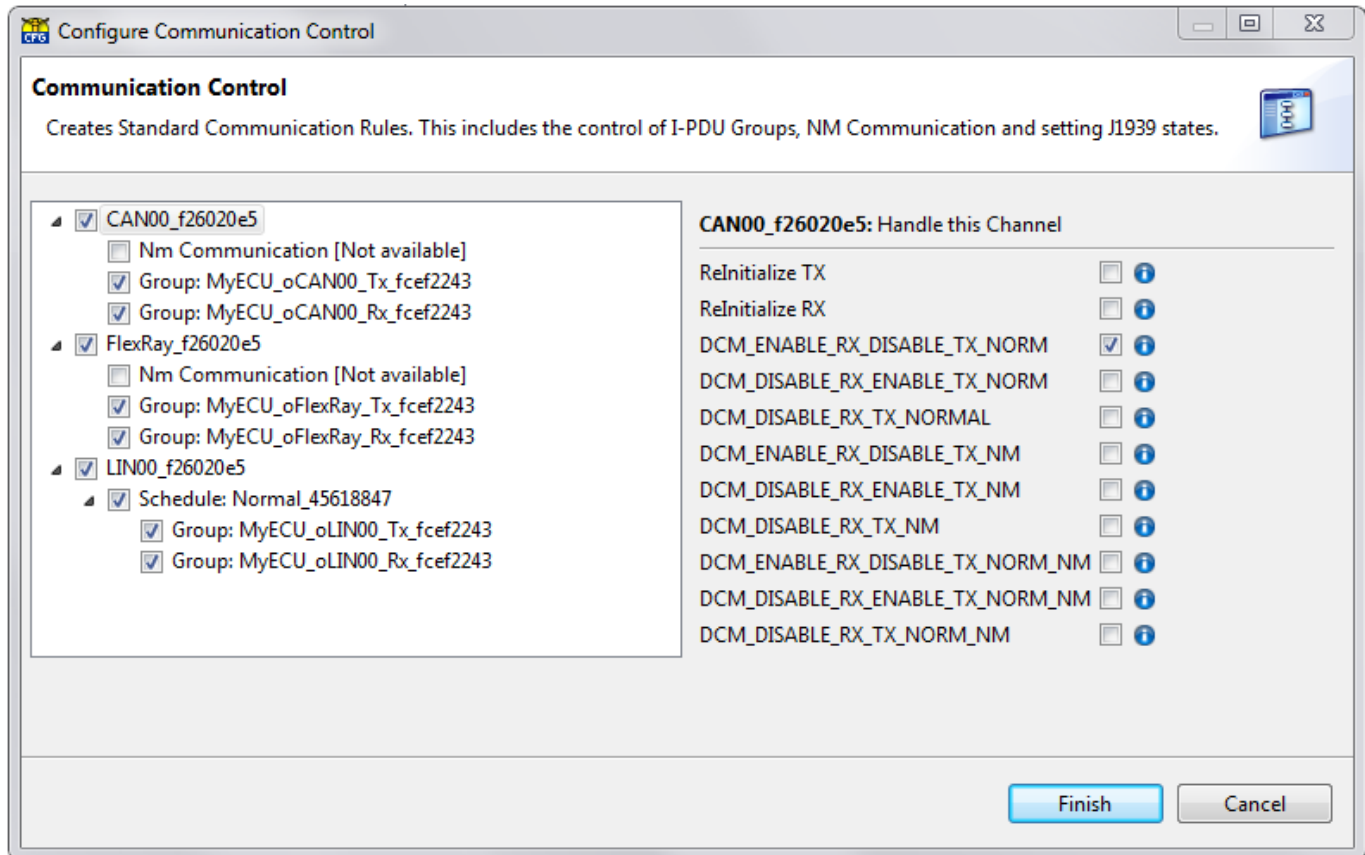
Open Auto Configuration: Communication Control and configure Communication Control via **Con-**

figure Communication Control hyperlink



[Configure Communication Control](#)

. The Configure Communication Control dialog opens.



Select the bus systems for which you would like to create standard communication rules.

Custom Configuration

If necessary you can define additional Custom BswM Configuration information.

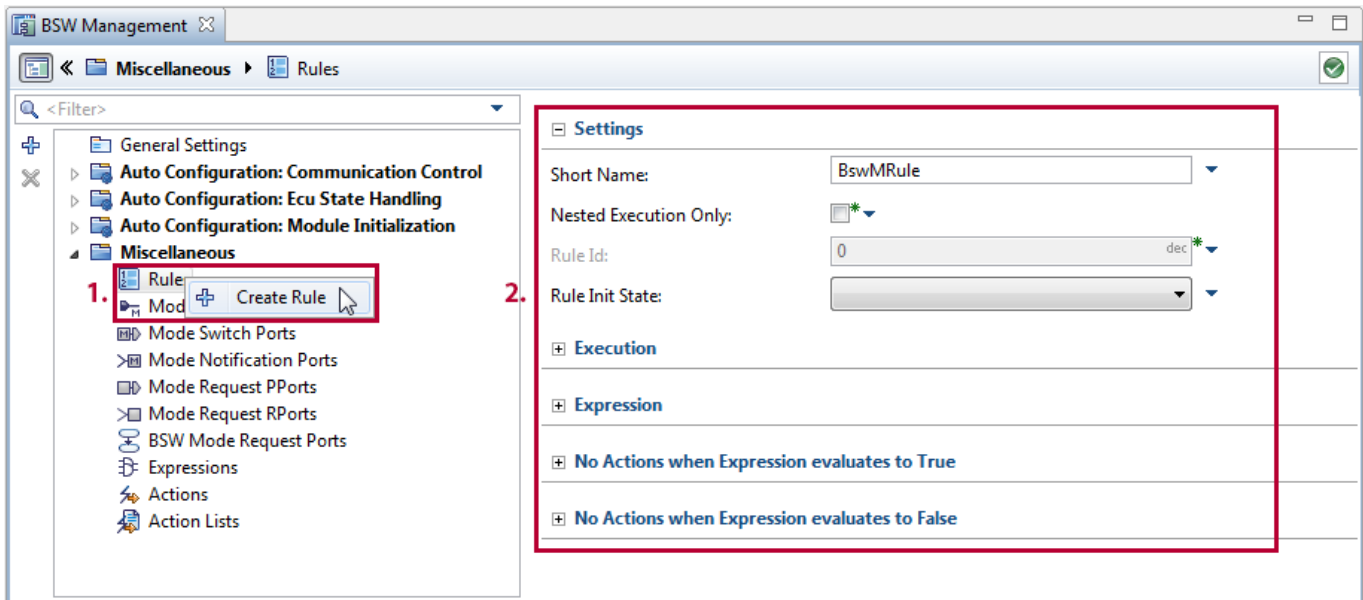


Note

To create a rule, at least one **logical expression** and one **action list** is necessary. If they are not available, you can additionally configure them via **Create Rule** dialog.



1. Open **Custom Configuration**, right click **Rules** and select **Create Rule**.
2. Define **Short Name**
3. Go on with **[Next]**



4. Define a non-empty expression for the resulting condition
5. Configure Action List True, therefore define actions to be executed if the rule's condition evaluates to true.
6. Configure Action List False, therefore define actions to be executed if the rule's condition evaluates to false.



Note

If no actions are configured, no action list will be created.



Note

It is possible to create groups **[+]** subdivide the BSWM configuration into logical sections. The elements can be organized in groups using drag and drop in the editor tree. The groups are only for organization reason, functional usage is not possible. You can't move elements from auto configuration to the created BSWM Group.

4.6.3 ECU Management

1. Open **EcuMCommonConfiguration** and define the **OS Resource** which is used to bring the ECU into sleep mode.
2. Open **EcuMFlexConfiguration** and define **Mcu Mode Ref** which is being restored after a sleep.

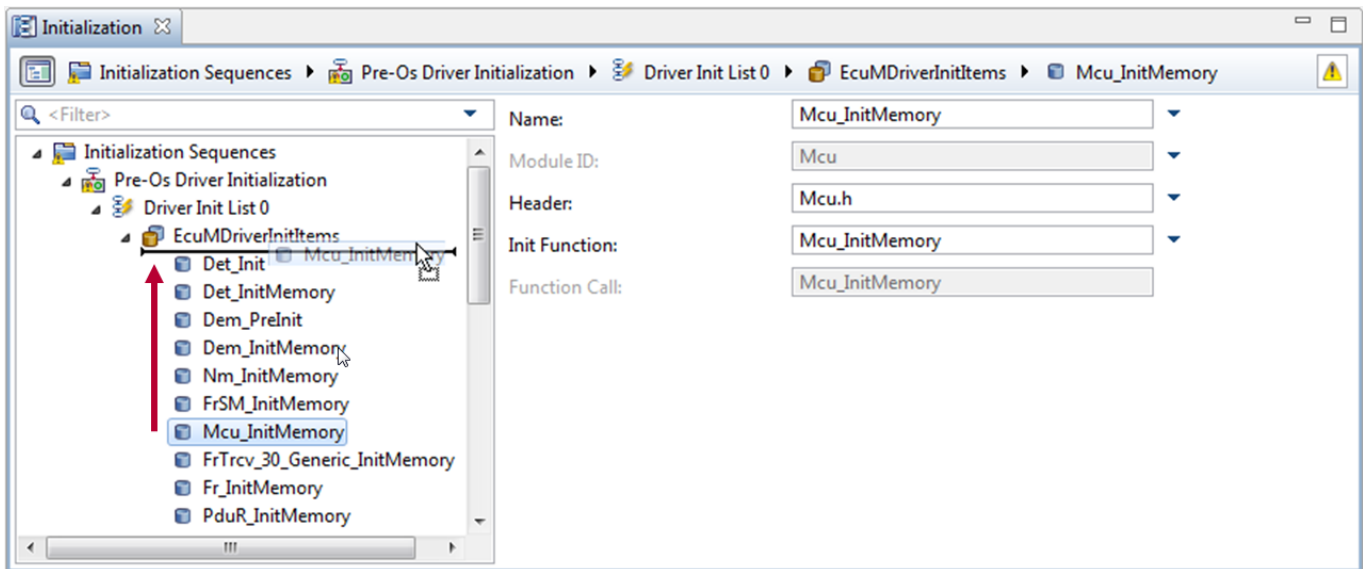
4.6.4 Initialization

Open **Initialization** to configure initialization and restart sequence of the basic software.



Pre-OS Initialization Sequence

1. Open **Initialization Sequences | Pre-OS Driver Initialization** and choose the **Driver Init List** that you want to configure.
2. Open **Driver Init List | EcuMDriverInitItems**. Each module in the list will be called for initialization in the list order. Modify the processing order via drag and drop in the tree:



3. Select all MCAL modules and the DET to call them on ECU startup.
4. Select **EcuMDriverInitItems** and delete via the **Delete containers button**  those modules from the list that shall not be initialized before the OS initialization.

4.6.5 Watchdogs

Use default settings.



Note

If the Editor is deactivated (greyed out), you can activate the Wdg module via click. The Modules Assistant opens to specify the short name of the module to be added. Click **[finish]** to add the module.

4.7 Network Management

The Network Management domain contains editors for the configuration of the general network management parameters and users which are relevant for the ECU communication management.



Note

Please make sure that within the dbc file the Network Attribute **NmType** is set to **Ford-OSEK**. Only one NM is allowed on one CAN bus and it has to be AUTOSAR NM.

If a node doesn't use NM make sure that the node attribute **NmStationAddress** is set to 0 (default).

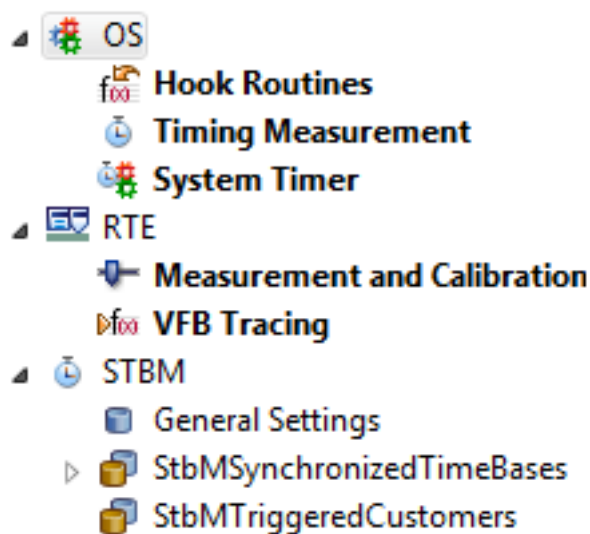
4.7.1 Partial Networking

Partial Networking isn't used by Ford.

4.8 Runtime System

The Runtime System domain contains comfort editor for the configuration of general RTE and OS parameters, operating system and mapping assistants.

4.8.1 Runtime System General



OS

1. Open **Runtime System General | OS** and activate/deactivate the following settings if necessary.
 - > Stack Monitoring
 - > Use GetServiceId
 - > Use Parameter Access
2. Define **Scalability Class, CC** and **Schedule**.
3. Open **OS | Hook Routines** and activate/deactivate the following settings if necessary.
 - > Startup Hook
 - > Error Hook
 - > Shutdown Hook



- > Pre-Task Hook
- > Post-Task Hook
- > Protection Hook



Note

The **Shutdown Hook** is required for shutdown of the ECU to power off.

The **Error Hook** is recommended to inform the application on any error events in the OS.

4.8.2 ECU Software Components

Service Components

Open **ECU Software Components** | **Service Components** and check if all Service Components are added to your project. If not create new component prototypes, based on component types created by the **DaVinci Configurator Pro**.



Note

The component types are stored in **<ProjectFolder>\Config\ServiceComponents**.

4.8.3 OS Configuration

Create Tasks

Open **OS Configuration** | **Task**. Add the following tasks, define Schedule, Priority and Type.

- > Init_Task (Autostart has to be enabled!)
- > Rte_Task
- > Main_Task



Note

All events and alarms which are required are created by the RTE automatically.

Derived from the settings for the runnables and their activation (triggers) some OS events and OS alarms will be created automatically for RTE by the **DaVinci Developer**. The names start with **Rte_**.

4.9 Go on with Basic Editor

Open the Basic Editor and configure additional settings, currently not supported by specific configuration editors.



Note: Bottom Up Approach - from hardware-dependent to hardware-independent modules

Start configuration with hardware dependent modules and continue with the hardware independent modules.



4.10 Start Solving Actions

Changes in Configurator Editors/Basic Editor could lead to errors during live validation!

If these errors include solving actions, check and start solving action in the validation view.

4.11 Start On-demand Validation

The on-demand validation is performed in addition to the automatic validation before starting the code generation or on explicit user request. The on-demand validation calls the specific validation function of the code generators.

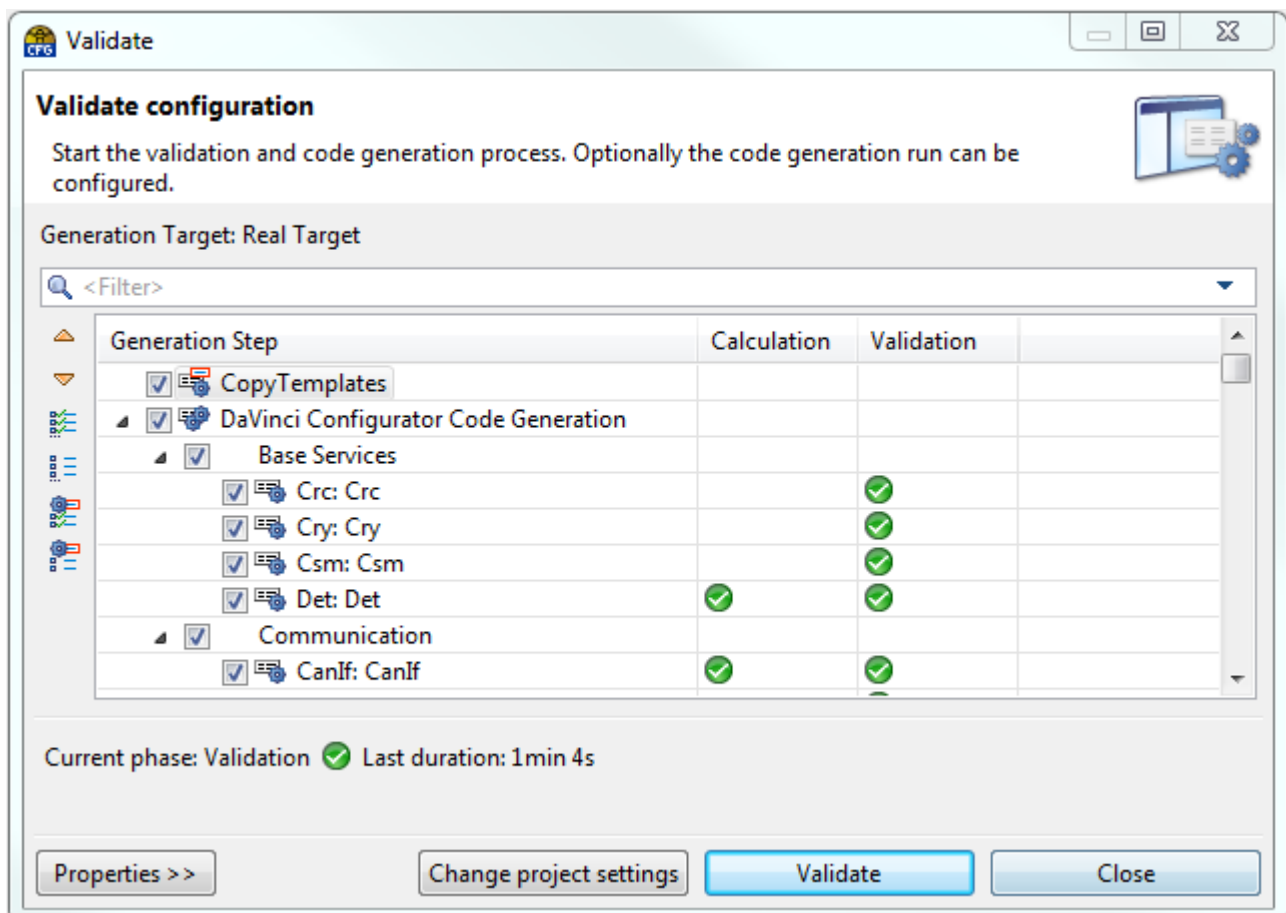
1. Open **On-demand Validation** dialog via **on-demand Validation button** 
2. Deactivate **RTE**



Note

Currently the RTE is not configured completely, this causes errors while validation. The RTE configuration will be finished after SWC Design in the **DaVinci Developer**.

3. Start the Validation via **[Validate]**





Validation finished successfully?

Validation finished NOT successfully?

See error message in validation view and fix the configuration.

4.12 BSW Configuration finished

You have finished your BSW Configuration?

Now it depends on your project, how to go on. You have created a Developer workspace while setting up the project? Then you have to go on with the STEP5 Design Software Components on page 52 using the **DaVinci Developer**. If you have no Developer workspace, you can skip the following chapter and go on with STEP6 Mappings on page 53.



5 STEP5 Design Software Components

Switch to the **DaVinci Developer** to create, configure or modify software components, create or add ports and data elements. Connect software components and define runnables and their activation and interfaces (data access).



Expert Knowledge

You need to know more? See section Software Component Design on page 93.

5.1 Switch to DaVinci Developer

You can start **DaVinci Developer** with already loaded project in two different ways:

- > **Start | Programs | Vector AUTOSAR Projects | <your project name> | Configure SWCs**
(Only if Create entries in the start menu is selected)
- > Right-click on your **<projectname>.dpa** and select **DaVinci Project Assistant | Configure SWCs**

5.2 Design Software Components

If the software components are not delivered by the OEM, use the **DaVinci Developer** to design your software components. How to do this is described more detailed in the expert knowledge part of this manual. Use the link above.



6 STEP6 Mappings

Start project mappings via assistants offered by the **DaVinci Configurator Pro**. It is also possible to perform data mapping within the **DaVinci Developer**.



Expert Knowledge

You need to know more? See section Mappings on page 114.

6.1 Perform Data Mapping within DaVinci Developer or DaVinci Configurator?

Via the data mapping, you connect data elements with network signals. Network signals have been loaded via import of ECU Extract of System Description.

Data Mapping can be done in **DaVinci Developer** or **DaVinci Configurator Pro**. Decide which tool should be used. In the following step, both variants are described.



Note

Data Mapping in **DaVinci Developer** has a higher priority than Data Mapping in the **DaVinci Configurator Pro**.

Data Mapping performed in **DaVinci Developer** cannot be overwritten in **DaVinci Configurator Pro**.



Cross Reference

To perform data mapping within the **DaVinci Configurator Pro**, skip this section and go on with section Switch (back) to DaVinci Configurator on page 56.

6.2 Data Mapping within the DaVinci Developer

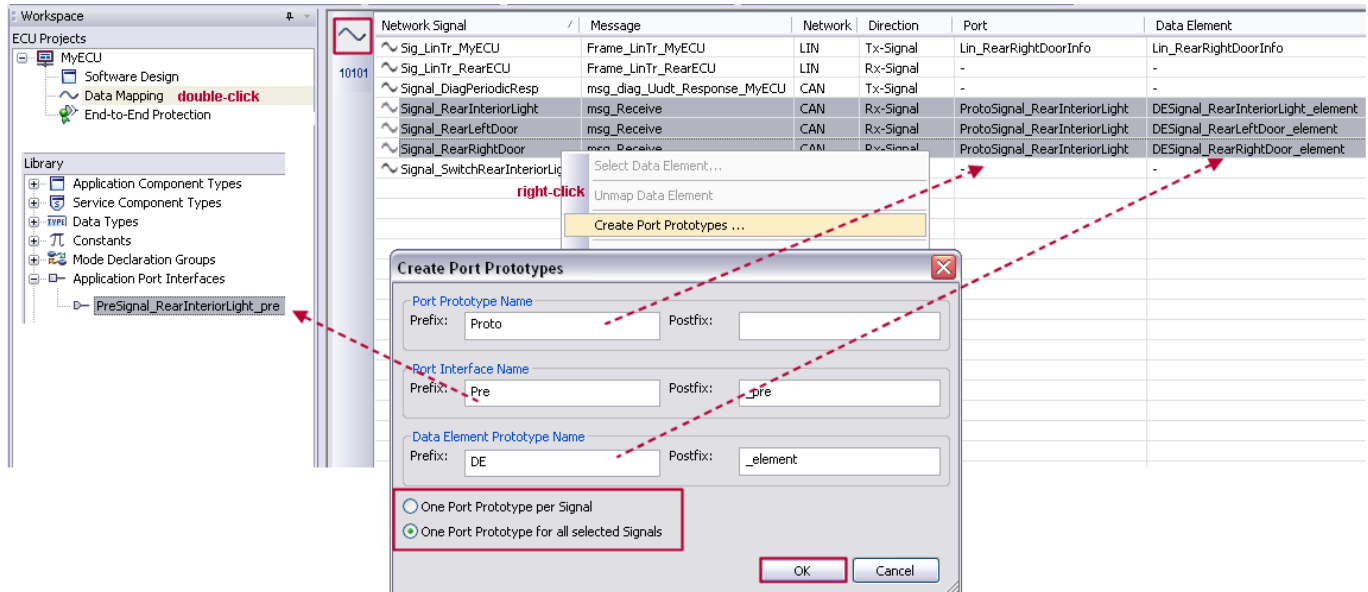
The **DaVinci Developer** offers two ways of data mapping - **automatically** OR **manually**.

6.2.1 Data Mapping Automatically - DaVinci Developer

Network signal oriented approach

Select bus signals and let DaVinci Developer create all necessary data elements, ports and the mapping of the signals to the data elements.

Finally, you only have to drag'n'drop the port interfaces to your software components and create the connections. Here is how this works.



1. Double-click Data Mapping from the ECU Projects view
2. Select the **Signal view mode**
3. Select signals to create Data Elements and Port(s)
4. Right-Click selected signals and select Create Port Prototypes ... to open the Create Port Prototypes window.
5. Define prefixes and postfixes for the Port Prototype Name, Port Interface Name and Data Element Prototype Name.
6. Select whether you want One Port Prototype per Signal or One Port Prototype for all selected Signals.



Note

If you select One Port Prototype for all selected Signals, of course the signal must be of equal direction, only Tx signals or only Rx signals.

7. Confirm your settings with **[OK]**.
8. You see that one Port Prototype, three Data Elements and one Application Port Interface are generated. The pre- and postfixes help to easily find them in the list.
9. Select the Software Design view and see the delegation port that has been also created.
10. Now drag'n'drop the Application Port Interface from the library to your software component (same direction as the delegation port) and connect them.

6.2.2 Data Mapping Manually - DaVinci Developer

You can also connect the data elements with real bus signals manually.



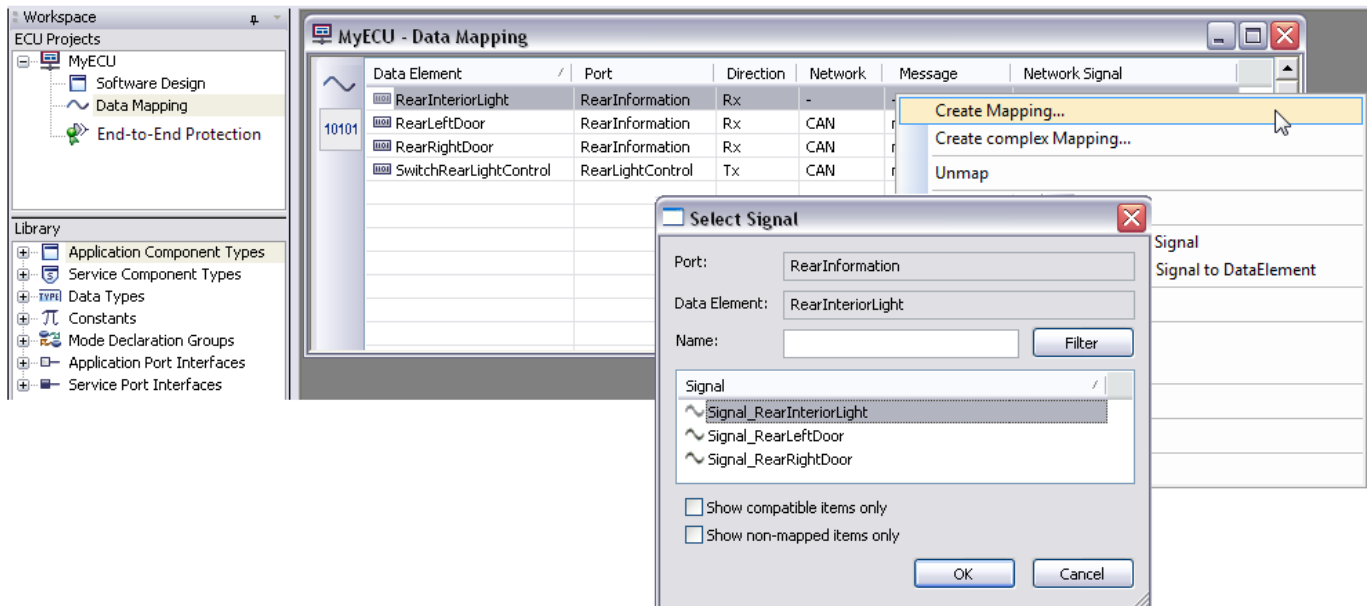
Note

A prerequisite is that you have to create the necessary delegation ports before!

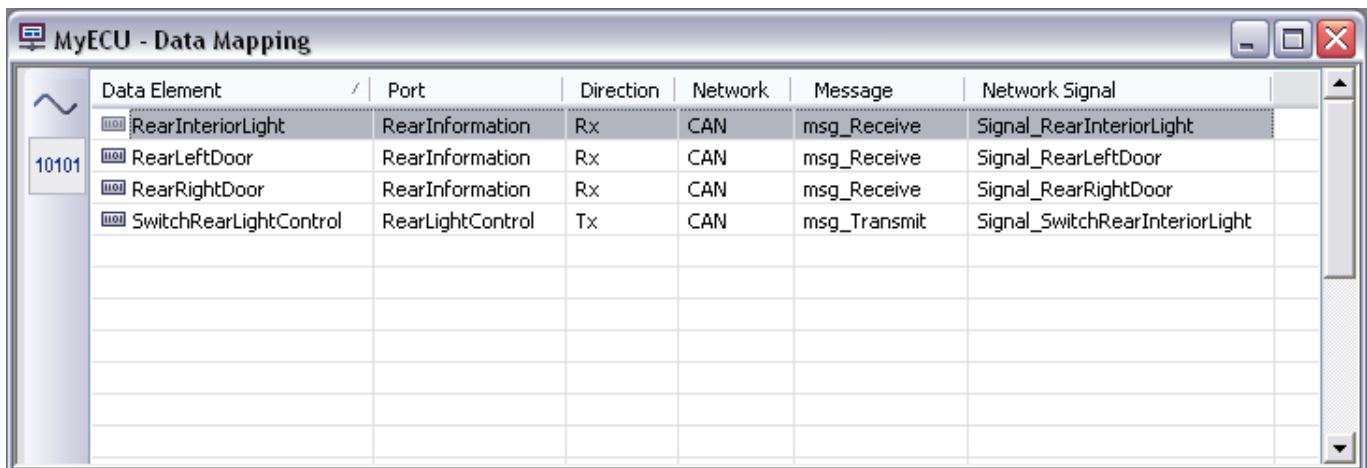


Select Data Mapping in the ECU Projects view and select e.g. the **Data Element View Mode** ¹⁰¹⁰¹ and see a list of data elements. The dash (-) on the right side below Network, Messages and Network Signals shows that there is no assignment between data elements and messages/signals from the ECU Extract of System Description.

Right-click the **Data Element column** and click **Select Signal...**. Select the suitable signal in the Select Signal window and confirm with [OK]. Repeat this until every signal is mapped.



The result of the data mapping should look like this.



6.2.3 DaVinci Developer - Save and close

You have imported your Service Components, configured your Software Components and finished Data Mapping?

Check your workspace. If there are no messages in the **Output** view, save your project and close the **DaVinci Developer**.



6.3 Switch (back) to DaVinci Configurator

Start the **DaVinci Configurator Pro** (again) with already loaded configuration using one of the two possibilities.

- > **Start | Programs | Vector AUTOSAR Projects | <your project name> | Configure BSW**
(Only if Create entries in the start menu is selected)
- > Right-click on your **<projectname>.dpa** and select **DaVinci Project Assistant | Configure BSW**

6.4 Synchronize System Description

Changes by the **DaVinci Developer** require a synchronization of the System Description in the **DaVinci Configurator Pro**.

The necessity of a synchronization is detected by the validation process of DaVinci Configurator Pro and displayed in the **Validation view**. Start System Description synchronization via the validation message **RTE59000 | Synchronize the System Description**.

6.5 Add Component Connection

Data elements can only be transported from one application component to another if their ports are connected via connectors. Create these connections using the **Component Connection Assistant**

 [Add Component Connection](#) (below **Runtime System**).

1. Start Assistant, select connector type and go on with **[Next]**
Select **Application Connector Prototypes** to connect software components or **Service Connector Prototypes** to connect service components
2. Select a **Component Prototype** and go on with **[Next]**
3. Select **Port Prototypes** and go on with **[Next]**
4. If necessary select additional search option and go on with **[Next]**
Initially the Component Connection Assistant will search matching port prototypes based on the port prototype name automatically
5. Check mapping. All Connector Prototypes are mapped with the right ports? Confirm with **[Finish]**

6.6 Service Mapping

6.6.1 Service Mapping via Service Component

If you have connected your service components via **Add Component Connection** in the previous step, the service mapping is basically done.

To define additional mappings manually you have to configure it in the service component directly. Therefore open **ECU Software Components | Service Components** and select the service component you want to map. Open service component and select **Service Connectors**.



Each **client port** should be mapped to a **server port** ! For all unconnected client ports the RTE will generate a **dummy** function returning RTE_E_UNCONNECTED.

- 9 of 16 service port prototypes are connected. Use the [Component Connection Assistant](#) to create service connectors.

© Vector Informatik GmbH



The appropriate port prototype not exists? You like to connect the port to a new port prototype?

Therefore select the service component port and use the button  to open the Connect To New Port Prototype Assistant.

1. Select the **Component** to be connected with the selected port prototypes.
Only those components are displayed which component type is part of the DaVinci Developer workspace.
2. Define the name of the **Port Prototype to be created**. By default the same name is used. Therefore click in the **Port Prototype to be created** column.
If a server port is selected, you can additionally select, if server runnables should created.
3. Click **Finish**

What happens?

The **DaVinci Developer** will open in background, a new port prototype (Application and Service port prototypes are possible) will be created within selected application component. The workspace will be saved and the **DaVinci Configurator** project will be reloaded. The new created port is now available within the **DaVinci Configurator** project and will be mapped automatically to the service component.

6.6.2 Service Mapping (overview)

Here you can do the complete service mapping. Connect or disconnect services to applications, toggle between services view and application view or connect to new port.

Here you also get a complete overview of all connections.

53 of 129 service port prototypes are connected. Use the [Component Connection Assistant](#) to create service connectors.

Component Prototype	Port Prototype	Connected Component Prototype	Connected Port Prototype	Port Interface Mapping
BswM	Switch_ESH_ModeSwitch	BswM	Notification_ESH_ModeNotification	
BswM	Switch_ESH_ModeSwitch	CtFimDemo	BswM_MSL_ESH_Mode	
BswM	Switch_ESH_ModeSwitch	CtDiagDemo	BswM_MSL_ESH_ModeSwitch	
BswM	Switch_ESH_ModeSwitch	CpSaLightDimControl	BswM_MSL_ESH_Mode	
BswM	Switch_ESH_ModeSwitch	CpApMySwc	BswM_MSL_ESH_Mode	
BswM	Switch_ESH_ModeSwitch	CpSaInteriorLightFront	BswM_MSL_ESH_Mode	
BswM	Notification_ESH_ModeNotification	BswM	Switch_ESH_ModeSwitch	
BswM	Request_ESH_RunRequest_0	CtDiagDemo	BswM_SRI_BswM_MSL_ESH_RunReques...	
BswM	Request_ESH_PostRunRequest_0	CtDiagDemo	BswM_SRI_BswM_MSL_ESH_RunReques...	

Connect
Disconnect
Toggles between Services and Application side view
Connect to new port ...

6.7 Add Data Mapping

You have already performed Data Mapping within the DaVinci Developer?

Yes? Skip this section and go on with Add Memory Mapping on page 60.



Cross Reference

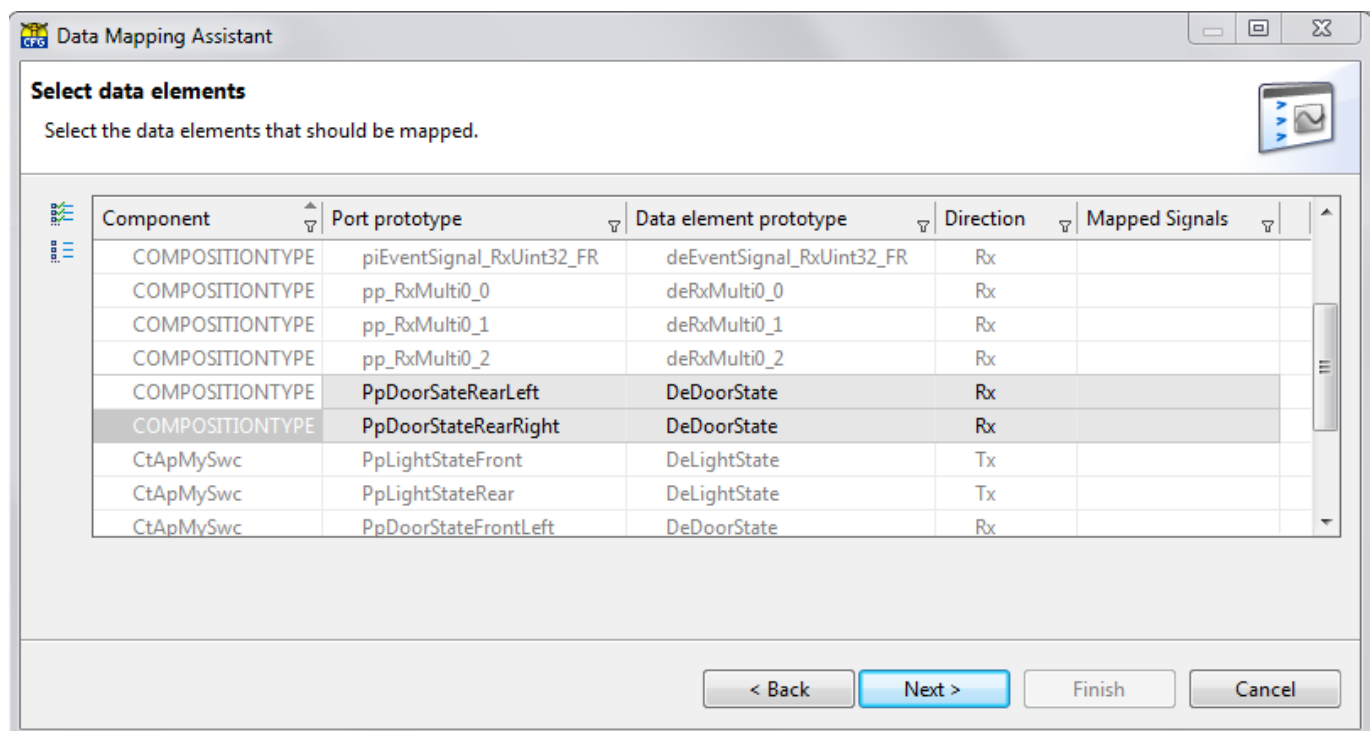
See section Perform Data Mapping within DaVinci Developer or DaVinci Configurator? on page 53



No? Start Assistant via Add Data Mapping  [Add Data Mapping](#) and select mapping direction, i.e. choose whether **Signals** or **Data Elements** should be mapped.

Choose **Find matching signals for the data elements** to assign your signals based on a selected data element and go on with **[Next]**

Now you can see a list of data elements. The empty row **Mapped Signals** shows that there is no assignment between data elements and messages/signals from the ECU Extract of System Description.



Data Mapping Assistant

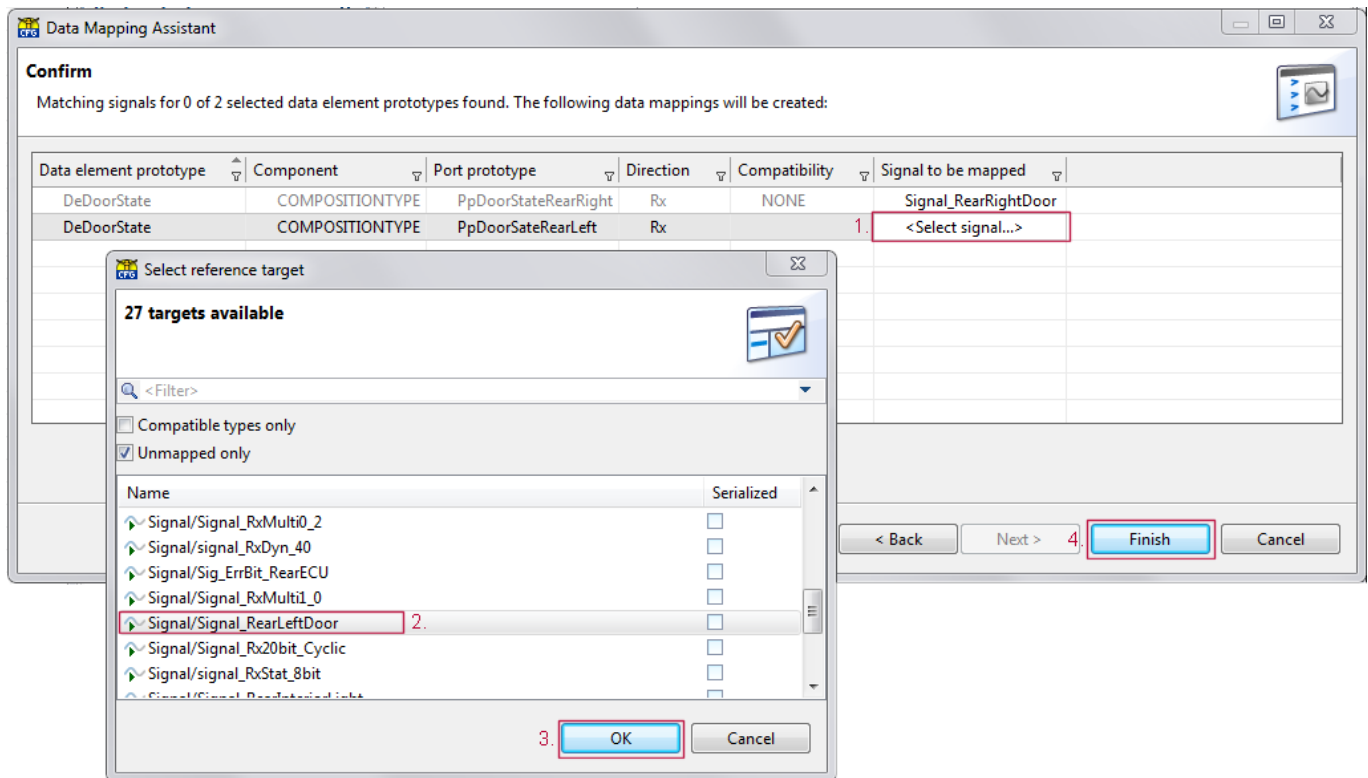
Select the data elements that should be mapped.

Component	Port prototype	Data element prototype	Direction	Mapped Signals
COMPOSITIONTYPE	piEventSignal_RxUint32_FR	deEventSignal_RxUint32_FR	Rx	
COMPOSITIONTYPE	pp_RxMulti0_0	deRxMulti0_0	Rx	
COMPOSITIONTYPE	pp_RxMulti0_1	deRxMulti0_1	Rx	
COMPOSITIONTYPE	pp_RxMulti0_2	deRxMulti0_2	Rx	
COMPOSITIONTYPE	PpDoorStateRearLeft	DeDoorState	Rx	
COMPOSITIONTYPE	PpDoorStateRearRight	DeDoorState	Rx	
CtApMySwc	PpLightStateFront	DeLightState	Tx	
CtApMySwc	PpLightStateRear	DeLightState	Tx	
CtApMySwc	PpDoorStateFrontLeft	DeDoorState	Rx	

< Back Next > Finish Cancel

Select the Data element prototypes you want to map and go on with **[Next]**.

Now you can see if matched signals are found automatically. If there are no entry in row **Signal to be mapped**, you have to define the signal manually.



1. Click **<Select signal...>** to open the **Select reference target** window.
2. Select the signal you want to map.
3. Confirm with **[OK]**.

Repeat these steps until all data elements you have selected for mapping are mapped to a signal.

4. Close Data Mapping with **[Finish]**.

6.8 Add Memory Mapping

You have to map all per-instance memory objects of the application components to NV memory blocks. Therefore use the **Memory Mapping Assistant**  [Add Memory Mapping](#) .



Note

A per-instance memory object is mappable if it is referenced by a service need object. The definition of per-instance memory objects or service needs is part of the component type definition and not yet editable by **DaVinci Configurator Pro**.

1. Start Assistant, choose **Use Case** and go on with **[Next]**
2. Select component prototype and go on with **[Next]**
3. Select per-instance memory objects
Choosing Use Case Find existing NV memory blocks?
Go on with **[Next]**, Check the memory mappings and confirm with **[Finish]**



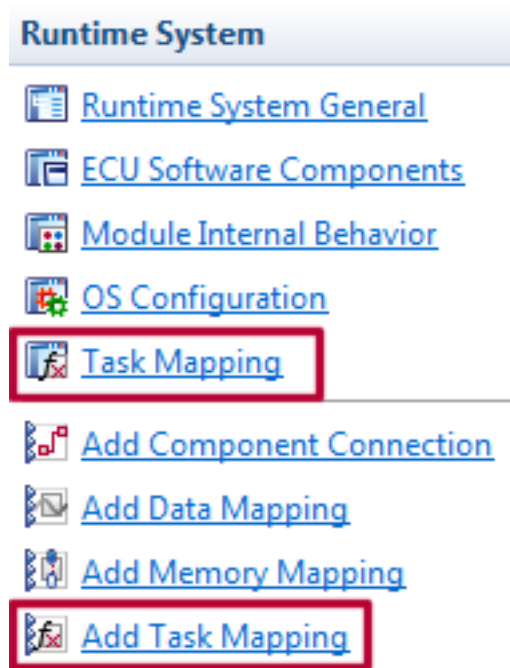
Choosing Use Case Create new NV memory blocks?

Confirm with **[Finish]**

6.9 Add Task Mapping

You have to map all runnable entities and schedulable entities (e.g. Main Functions of the BSW modules) to a task. There are two ways to perform the task mapping.

Where to activate the two ways of task mapping



Either the **Task Mapping Assistant**  [Add Task Mapping](#) or the Task Mapping view .

Our advice is to use the assistant for the first mapping and then switch over to the Task Mapping view  to do the fine tuning.

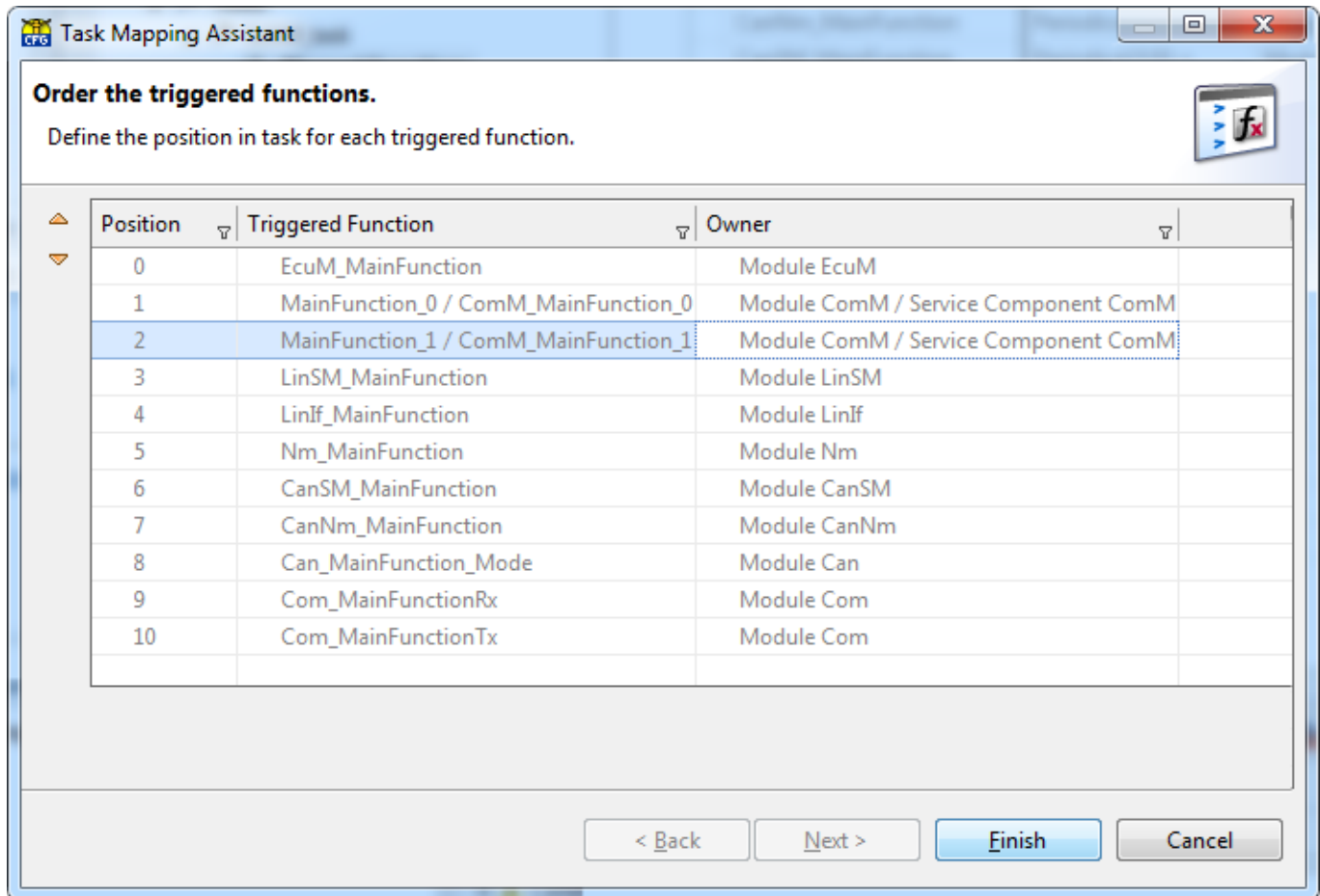


Note

A runnable has to be mapped to a task if it is not re-entrant but could be called re-entrantly during system operation.

6.9.1 Task Mapping Assistant

1. Start Assistant, select all Triggered Functions with Function Trigger On Init and go on with **[Next]**
2. Select Init_Task and go on with **[Next]**
3. Define the position in task for each triggered function and confirm with **[Finish]**



- Repeat the steps and select all **Main Functions** of **Service Components** and **Modules** and map them to **Main_Task**.
- Repeat the steps and select all **Runnables** of your **Application Components** and map them to **Rte_Task**.



Note

Runnables with Function Trigger **On Operation Invocation** are normally not needed to be mapped to a task, as they result in direct function calls.

6.9.2 Task Mapping

The task mapping is one view where you can

- > map runnable entities per drag and drop,
- > move mapped runnable entities from one task to another,
- > filter the list on the left side for mapped mandatory or unmapped runnable entities,
- > get information of the mapped runnable entities concerning their triggers, etc.



Task Mapping

Application Components: CpApMySwc, CpSaDoorFrontLeft, CpSaDoorFrontRight, CpSaInteriorLightFront, CpSaLightDimControl, CtCdd_TgfTestApplication, CtE2eCommunication, IoHwAb

<Filter>

Application Components

- Application Components
 - CpApMySwc
 - CpSaDoorFrontLeft
 - CpSaDoorFrontRight
 - CpSaInteriorLightFront
 - CpSaLightDimControl
 - CtCdd_TgfTestApplication
 - CtE2eCommunication
 - IoHwAb
- Service Components
- Schedulable Entities

<Core unassigned>					
<OsApp unassigned> TgfBackgroundTask Priority 30	<OsApp unassigned> TgfMainTask Priority 0	<OsApp unassigned> BrsTcc1msTask Priority 6	<OsApp unassigned> Brs_Task Priority 40	<OsApp unassigned> DemoMainTask Priority 3	<OsApp unassigned> DemoInitTask Priority 6
	Tgf1Ms_Runnable		BrsCtrl_Tasklevel	RCtApMySwcWdgMTTrigger1	CtApMySwc_Init
				RCtApMySwc_20ms	CtSaLightDimControl_Ir
				RCtSaDoor_100ms	CtE2eCommunication_I
				RCtSaDoor_100ms	
				RCtSaInteriorLight_100ms	
				RCtSaLightDimControl_100ms	
				ResetEcu	
				IoHwAb_IoHwAbRunnable	
				CtE2eCommunication_500ms	
	Periodical/1 ms		Periodical/200000 ms	Mode Entry, Periodical/10 ms, Periodical/100 ms, Periodical/20 ms, Periodical/50 ms, Periodical/500 ms	Init

More details about the features of this dialog you find in the **Online Help** of the **DaVinci Configurator Pro**.

Mappings finished? Validation successful? Then go on with project generation!



7 STEP7 Code Generation

Now you can go on with code generation.



Expert Knowledge

You need to know more? See section Generation on page 121.

7.1 Generate SWC Templates and Contract Phase Headers

Start the generation of the SWC templates and/or contract phase headers via  from **DaVinci Configurator Pro** toolbar.



Cross Reference

How to select your settings for the generation of the software components templates and contract phase headers? See section STEP2 Define Project Settings on page 31.

7.2 Start Code Generation

1. Open the Generation dialog via **Generate button** 



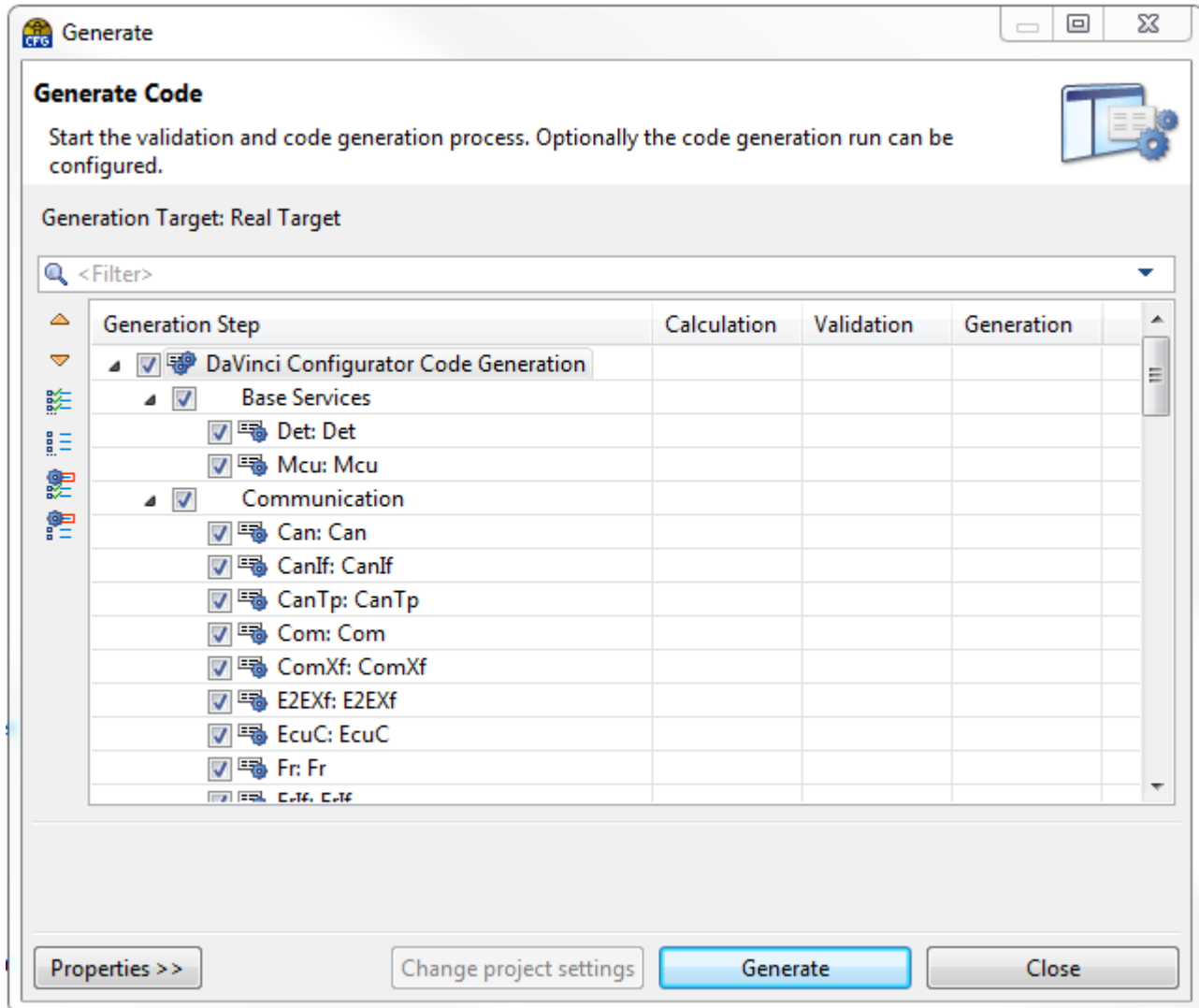
Note

The activated generation steps depend on the project settings. If you want to take over your selection to the project settings press **[Change project settings]**.



2. Start Generation Process via **[Generate]**

If you select **Start immediately** the generation starts immediately after opening this dialog.



Note

Before the **Generation** process starts, **Calculation** and **Validation** has to be finished successfully.

If code generation canceled because of calculation and validation errors, start solving actions at the validation window of the **DaVinci Configurator Pro** (see section STEP3 Validation on page 34).



7.3 Generation Process finished!

Generation Process finished and all generation steps are marked with Phase finished successfully ?



Note

The **DaVinci Configurator Pro** stores all generated *.c and *.h file in the modules files folder, given during project setup (the default is **<ProjectFolder> \Appl\GenData**).

Go on with implementing your runnables using the runnable skeletons created during the generation process.



8 STEP8 Add Runnable Code

If configured, the **DaVinci Configurator Pro** code generator generates the skeletons for your runnables into the software component template files. Now you have to implement your code to the runnable skeletons.



Expert Knowledge

You need to know more? See section Runnable Code on page 122

8.1 Component Template

The software component templates contain all runnables of the according software components. The RTE Template Generator generates skeletons for the runnables. The structure of those skeletons looks as shown in the illustration below.

Generated Skeleton for a Runnable

Name of runnable

When is runnable executed

Interfaces

- ☐ Input Interfaces
- ☐ Output Interfaces
- ☐ Service calls
- ☐ Available application errors
- ☐ Inter Runnable Variables

Start Comment

Your code

End Comment

Summary of environment conditions:

Contains Name and information when this runnable will be executed

Available Interfaces:

Lists all available interfaces of this runnable. All those listed interfaces can be used by your code.

Start Comment:

Only add code after this comment to prevent the code from being overwritten with the next generation process.

Your Code:

Enter the code for your runnable here. This code will not be overwritten with the next generation process.

End Comment:

Only add code before this comment to prevent the code from being overwritten with the next generation process.



8.2 Implement Code

Implement your code using the runnable skeletons. Make sure to enter your code between the start and end comment. Only this section is protected against modification by a further generation process.

```
FUNC(void, RTE_AP_MYSWC_APPL_CODE) MySCW_Code(void)
{
    /*-----< Start of runnable implementation >-----*/
    * DO NOT CHANGE THIS COMMENT!      << Start of runnable implementation >>      DO NOT CHANGE THIS COMMENT!
    * Symbol: MySCW_Code
    /*-----*/
    Boolean dataLeft, dataRight, dataLeftRear, dataRightRear;

    Std_ReturnType value;

    Rte_Read_FrontLeftDoorState_OpenClose(&dataLeft);
    Rte_Read_FrontRightDoorState_OpenClose(&dataRight);
    Rte_Read_RearInformation_RearLeftDoor(&dataLeftRear);
    Rte_Read_RearInformation_RearRightDoor(&dataRightRear);

    if (dataLeft || dataRight || dataLeftRear || dataRightRear)
    {
        Rte_Call_UR_ComUser_CAN_GetRequestedComMode(&value);
        if (value != COMM_FULL_COMMUNICATION)
        {
            Rte_Call_UR_ComUser_CAN_RequestComMode(COMM_FULL_COMMUNICATION);
        }

        Rte_Write_FrontLightState_OnOff(TRUE);
        Rte_Write_RearLightControl_SwitchRearInteriorLight(TRUE);
    }
    else
    {
        Rte_Write_FrontLightState_OnOff(FALSE);
        Rte_Write_RearLightControl_SwitchRearInteriorLight(FALSE);

        Rte_Call_UR_ComUser_CAN_GetRequestedComMode(&value);
        if ((value != COMM_NO_COMMUNICATION) && (lightDimControl == 0))
        {
            Rte_Call_UR_ComUser_CAN_RequestComMode(COMM_NO_COMMUNICATION);
        }
    }
    /*-----< End of runnable implementation >-----*/
    * DO NOT CHANGE THIS COMMENT!      << End of runnable implementation >>      DO NOT CHANGE THIS COMMENT!
    /*-----*/
}
```

All other sections like execution information or interfaces can be changed depending on your settings in the **DaVinci Developer**. If access to e.g. a port interface is missing, go back to the **DaVinci Developer** and adapt your configuration, regenerate and then you can use this port interface. There is no other way to do it properly.



Info

A backup (*.bak) file will be generated before a component template C file is modified to make sure that your previous version is not deleted in case of any generation problem caused by non-predictable reasons.

And there is a backup section at the end of the template file to rescue the code of runnables that have been deleted by the **DaVinci Developer**.



9 STEP9 Compile, Link And Test Your Project

Now you can compile, link and test to get your executable for the download to your hardware.



Expert Knowledge

You need to know more? See section Compile and Link on page 124.

9.1 Finish your project with compiling and linking

1. Compile and Link your code.

You can start compilation of your code from the vVIRTUALtarget tool. Enable the checkbox Build Software (DLL) in the VTT tool and select  **[Build]**. You can also open the Visual Studio solution and start the build process by **Build Solution (F7)**.

2. Test your code

A proper compile and Link process is just the beginning. The final step is to test what you have done until now. The ideal tester would be **CANoe** from Vector Informatik.

To test your application with **CANoe**, the following steps are necessary:

- > Create a new **CANoe** configuration
 - > Import the communication matrix (e.g. *.dbc file)
 - > Right-click on the ECU to be simulated, and click on **Configuration**
 - > Open the **Components** tab, add another node layer DLL and select the *.dll file, you have created with the **Microsoft Visual Studio** solution in the previous steps.
This completes the basic setup for testing your application. See CANoe references for further information.
 - > Start the simulation.
 - > Have a look at the trace window and observe bus communication on your virtual ECU.
3. Debugging the application
- When the simulation in CANoe is started, switch to Microsoft Visual Studio and select **Debug | Attach to process ...**. Select **RuntimeKernel.exe** and make sure that the code type (Attach to) is set to Native code. Now you can debug your code. For example, try to set a breakpoint in a cyclic runnable!

9.2 No error frames? Congratulations, that's it!

The basic step is done, the Ford SLP1 software is basically working together with your application.

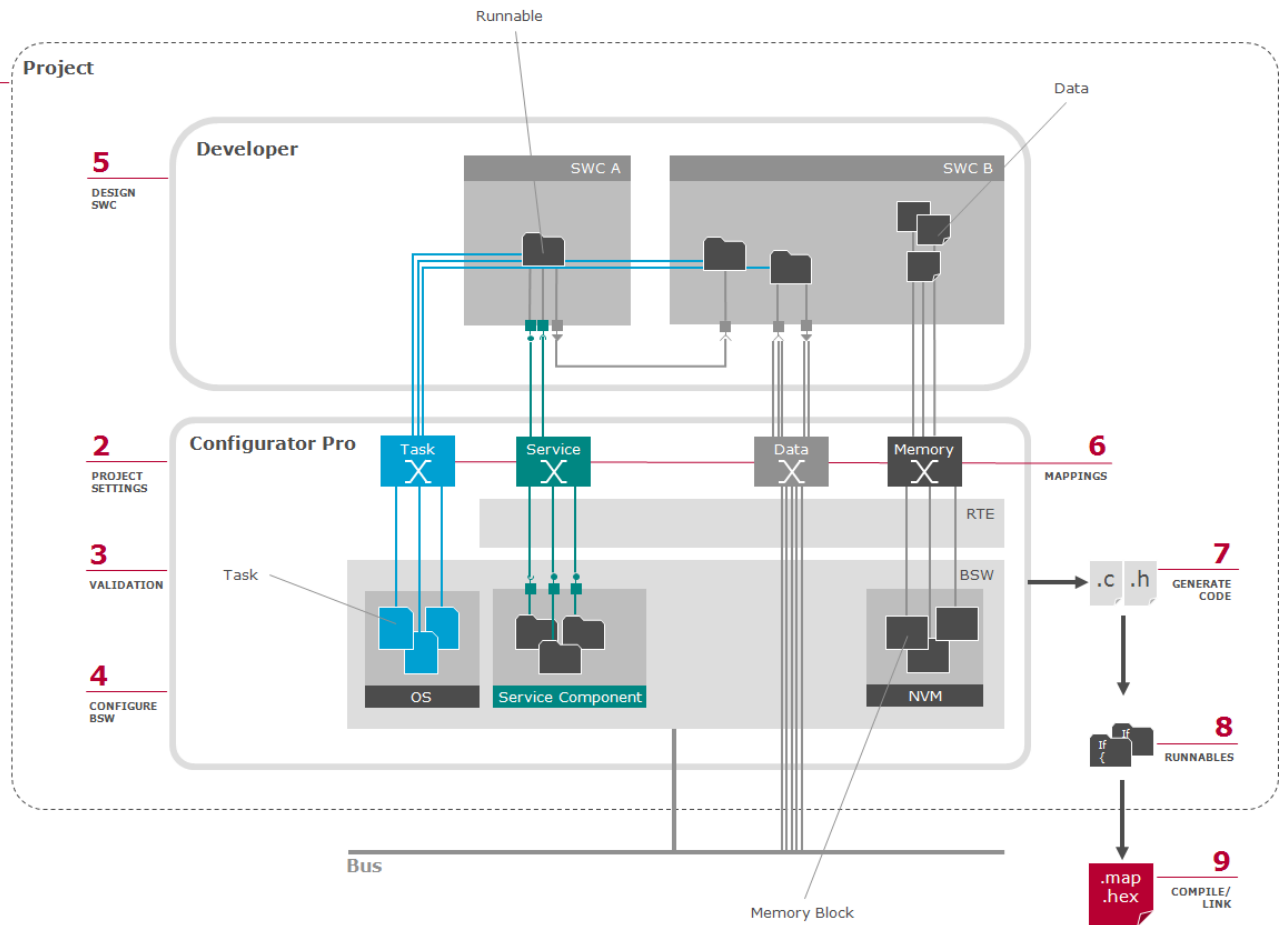
That is the base you can start from to optimize your system. Refer to Technical References of the BSW modules for more detailed information.



Cross Reference

You need additional information? See section Additional Information on page 125.

II Concept



1 General Overview

The following illustration shows the basic idea behind AUTOSAR and can be divided up into 3 parts, upper, middle and lower part, named as

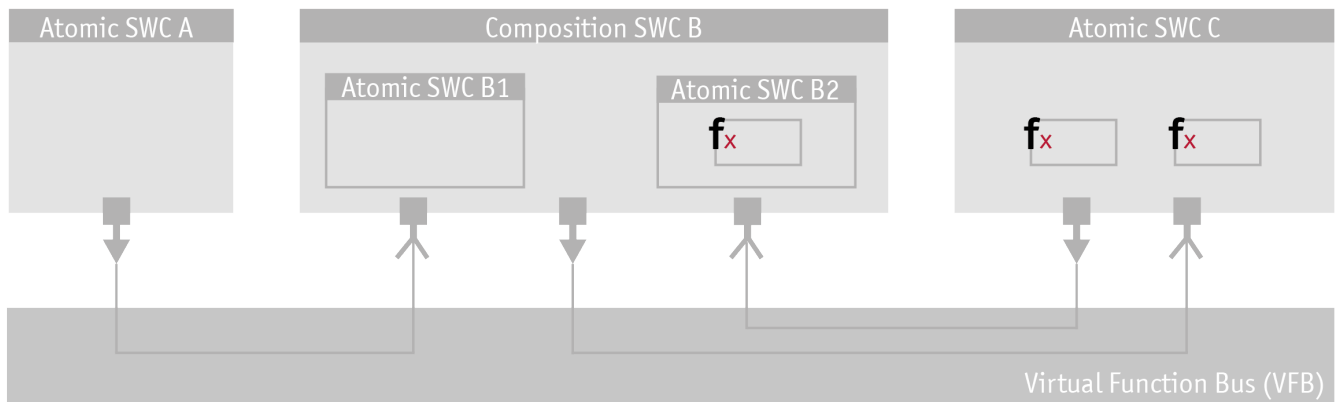
- > Creation of Software Components
- > Software Components and ECUs
- > Development of ONE ECU

The upper part deals with software components, runnables, ports and connections. It is the design level of functions and applications.

During the transition to the middle part, the software components are assigned to ECUs. Software components that are assigned to the same ECU can communicate internally, via so-called internal connections. Software components, that communicate with each other and that are mapped to different ECUs communicate via so-called external connection, i.e. via a real bus system.

In the last transition towards the ECU sight, the BSW is put in-between the software components and the hardware. And this is the starting point for the ECU development.

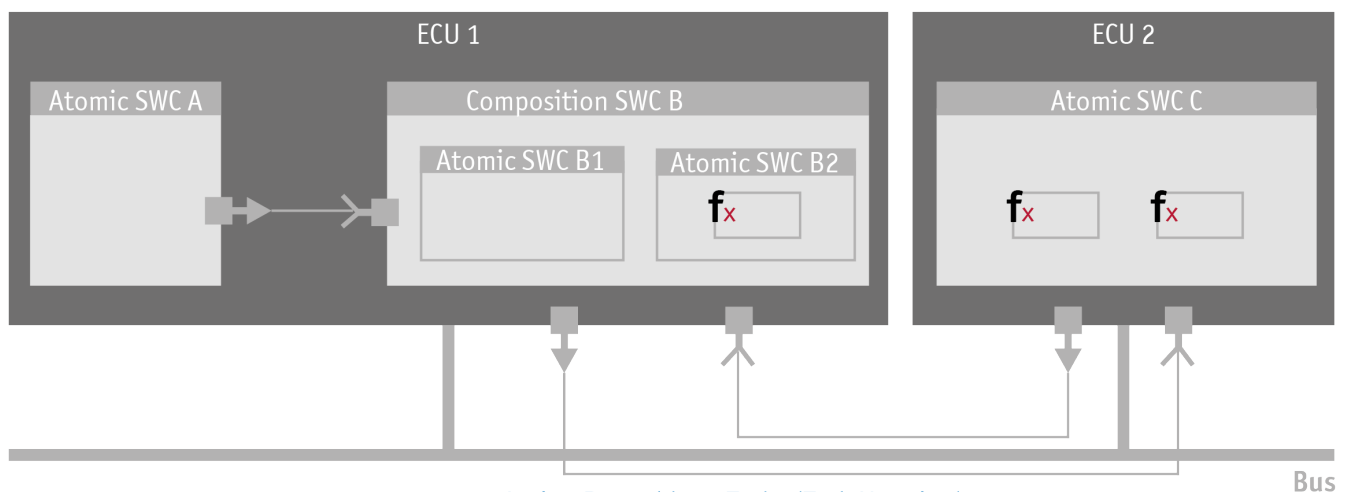
Creation of Software Components



Mapping of SWC A and SWC B to ECU 1

Mapping of SWC C to ECU 2

Software Components and ECUs

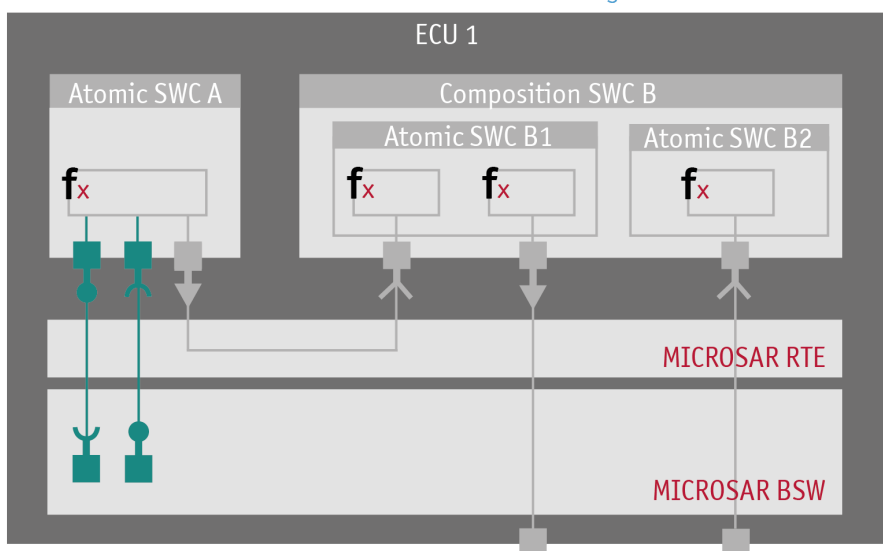


Assign Runnable to Tasks (Task Mapping)

Service Components (Service Mapping)

BSW - Basis Software modules

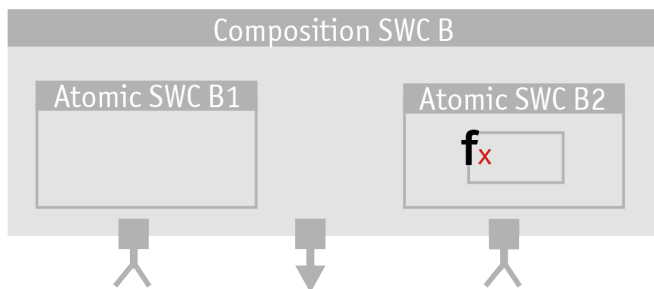
Flattening

Development of **ONE** ECU

The following sub chapters explain all the means that belong to AUTOSAR and that make it work.

1.1 Software Component

Software Components are described by their SWC file - a file in XML format. By definition SWC files are independent from any ECU except for the sensor and actuator software components. Those software components are dependent on the sensors and actuators attached to a specific ECU.



There are several kinds of software components

1.1.1 Atomic components

- > Application
- > Sensor Actuator
- > Calibration
- > I/O Hardware Abstraction
- > Complex Driver
- > Application (End-to-end Protection)
- > Non-Volatile Memory Block
- > Service components

1.1.2 Compositions

Use compositions to group software components.

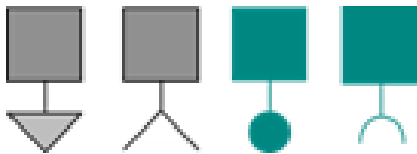
1.2 Runnables

A runnable entity, runnable for short, is a C function that carries your code. It depends on the configuration of the runnable when it is called and what data your code is allowed to access within the runnable. Runnables are triggered by the RTE.



1.3 Ports

Via ports, a software component can communicate with its outside world. This outside world can be another software component within the same ECU or a software component within another ECU. There are several kinds of ports:



1.3.1 Application Port Interfaces

To communicate between different SWCs

- > Sender port
- > Receiver port
- > Client port
- > Server port
- > Calibration port
- > Mode port
- > Interface to non-volatile data

1.3.2 Service Port Interfaces

To communicate between BSW and SWCs

- > Sender port
- > Receiver port
- > Client port
- > Server port
- > Calibration port
- > Mode port
- > Interface to non-volatile data
- > NV Data Interface

1.4 Data Element Types

Data elements types determine the kind of data that can pass a port. There are predefined data element types and you can define data elements on your own.

1.5 Connections

To define which software components communicate with each other, their ports have to be connected via so-called connectors. The connectors realize the sender/receiver and client/server communication

between the software components. A connector can only be drawn between two ports, if their port interfaces are compatible.

1.6 RTE

The RTE is the interface between the SWCs and the BSW. In essence the RTE

- > controls the execution of the Runnables (your code),
- > controls access of Runnables to the basic software modules,
- > controls the access of Runnables to Services of the BSW,
- > knows about external and internal transmission of data and
- > provides data consistency

1.7 BSW – Basic Software Modules

The BSW contains configurable modules that offer services to the SWCs dealing with:

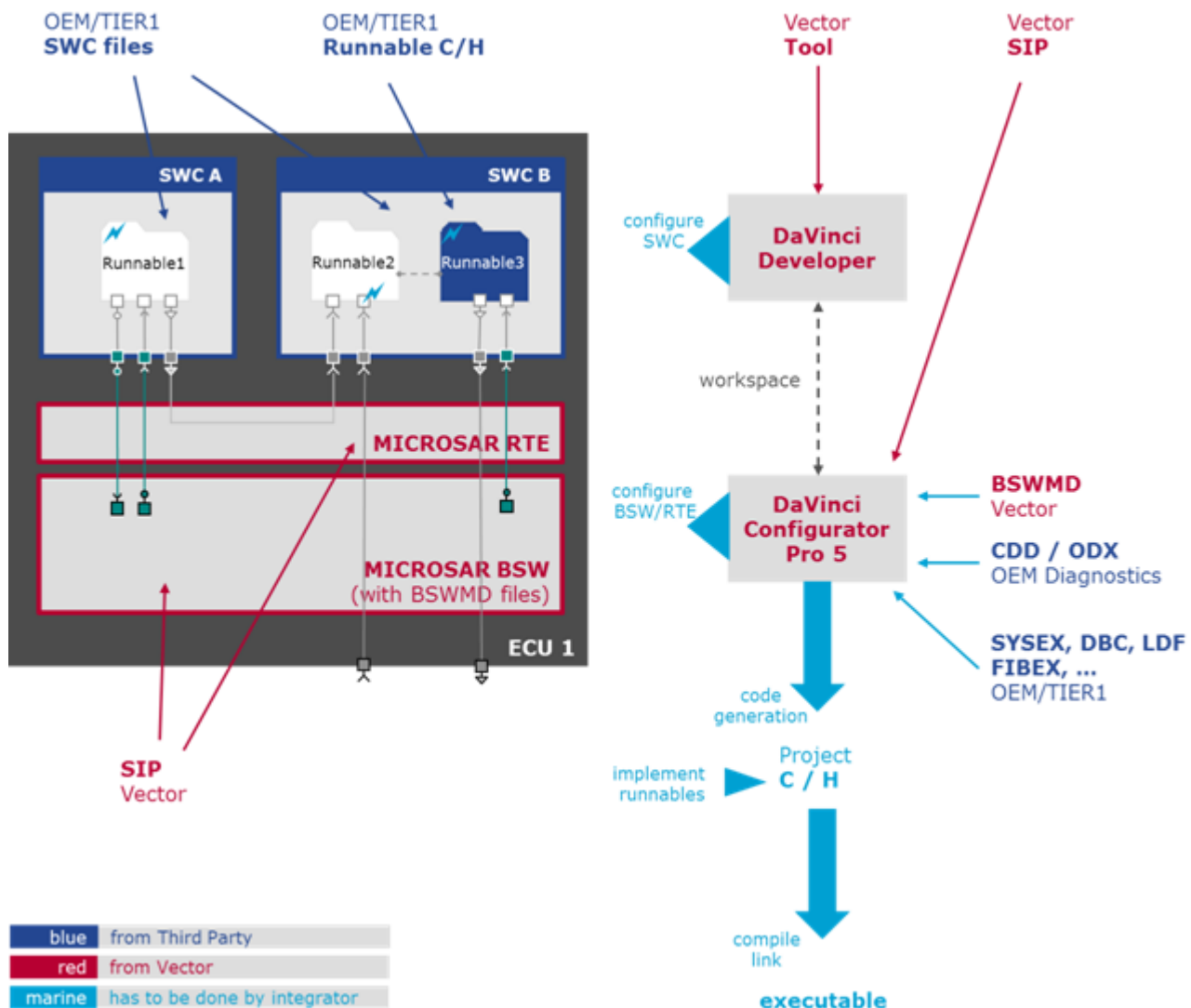
- > Communication
- > Diagnostics
- > Mode management
- > Memory management
- > Network management

1.8 Software, Tools and Files

When setting up a new project using the **DaVinci Configurator Pro**, many things that are not necessary to know for your basic work with the tool happen in the background. But sometimes it might be necessary to know more. Then this part of the document is the place to get a deeper insight.

This chapter gives you an overview what kind of files you need to set up a new project, where the files normally come from and how to work with the **DaVinci Configurator Pro** and, if necessary for software component design, with **DaVinci Developer**. Get familiar with what is installed where and how to work with it and to know, how to update if necessary.

Starting point is the development of a single ECU. The focus is set on the MICROSAR Solution and where all the necessary software and files do come from.



To build up a project according AUTOSAR with MICROSAR from Vector you need the following tool(s), software and files:

> Software Components (SWC) and Runnables

Software components are created by the OEM or the TIER1. A software component can be an SWC file, where you have to perform the task mapping and decide about runnables and the code they should contain. Or the software component is delivered ready-made, together with C and H files. Then you only have to map it to a task and compile and link the files together with your project.

> DaVinci Developer (Vector Tool, has to be ordered separately - NOT included within the SIP)

The **DaVinci Developer** is the Vector Tool to create and configure Software Components. Via a graphical view you can draw software components, add ports, draw connections, etc. **DaVinci Developer** and **DaVinci Configurator Pro** exchange information via the **DaVinci Developer Workspace**.

> BSW and RTE (included in SIP)

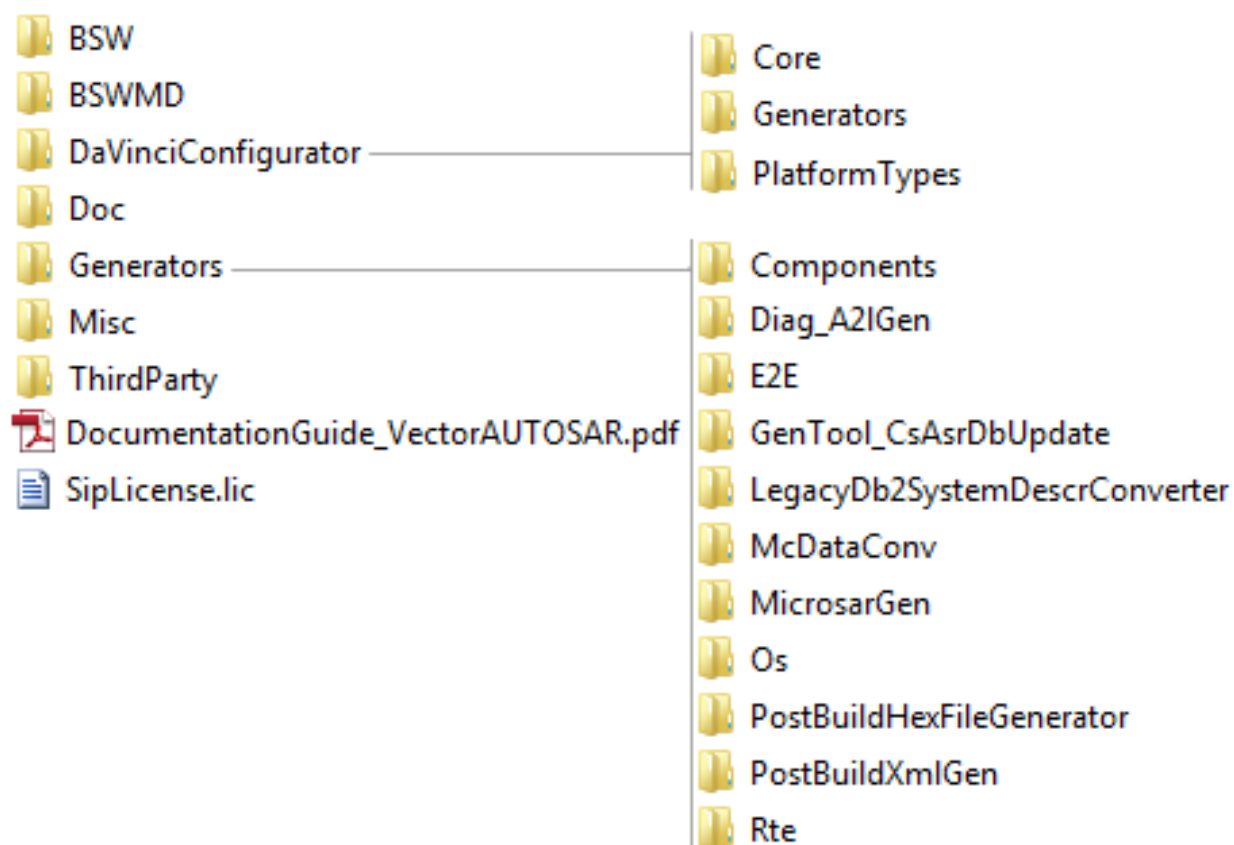
The BSW modules together with their BSWMD files (dependent on your order) and the RTE are part of the Vector delivery that is called SIP. **SIP** stands for **Software Integration Package**. A SIP is a ready-made software from Vector, already tailored and tested for your hardware, derivative, compiler, compiler version (as you filled-out the questionnaire) and can be put to work within a few steps (see step by step description).

> DaVinci Configurator Pro (included in SIP)

The **DaVinci Configurator Pro** is the Vector Tool to configure the BSW and the RTE. It is part of the SIP delivery. It contains a comfort editor to configure the BSW cluster-oriented, a powerful validation concept to check your settings against constraints and it also offers to directly edit AUTOSAR parameters.

1.9 Structure of the SIP Folder

The screenshot below shows the file structure that is created by installation of the SIP.



> BSW

Contains the code files (*.C and *.H) files for each Basic Software Module (BSW).

- > **BSWMD**
Contains the BSWMD files for each Basic Software Module, which include necessary information for **DaVinci Configurator Pro**.
- > **DaVinciConfigurator**
Contains the **DaVinci Configurator Pro** tool itself. This is also the place where your tool is updated when an update is available.
- > **Doc**
Contains SIP relevant documentation e.g. Technical References, User Manuals, Startups (containing Release Notes) and Application Notes.
- > **Generators**
Contains Generators and additional tools
- > **Misc**
Contains SIP-specific additional data
- > **ThirdParty**
Contains MCAL-specific data

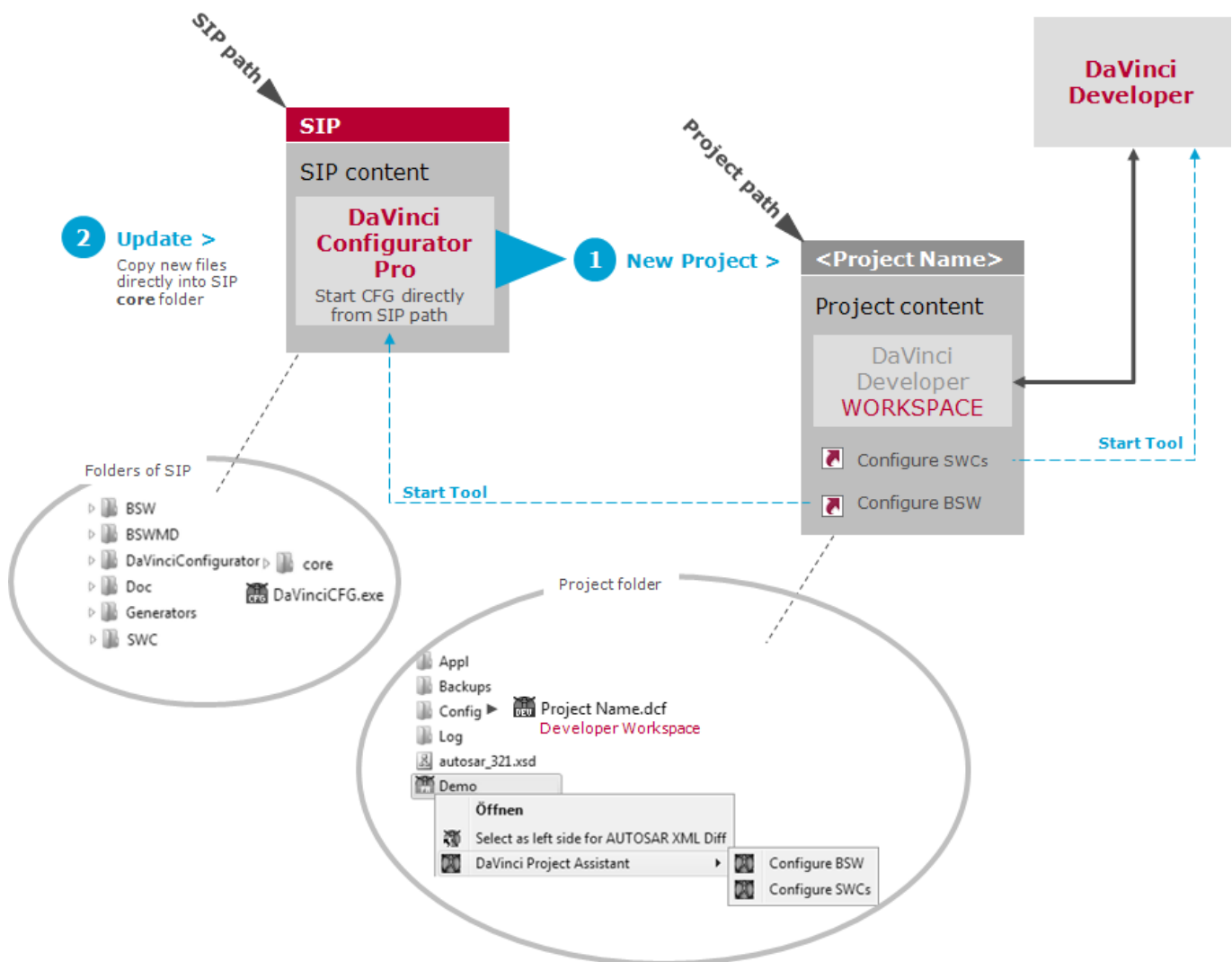
**Note**

Each MCAL is listed within a separate folder (<MCAL Short Name>).

This folder includes:

- > **\Supply**
Location of the 3rd party MCAL delivery in its original structure
- > **\VectorIntegration** (optional)
Location of scripts for the integration into the MICROSAR SIP
- > Legal notes (optional)

See **TechnicalReference_3rdParty-MCAL-Integration.pdf** for detailed information.



Vector MICROSAR for AUTOSAR 4 now has one major tool – the **DaVinci Configurator Pro**. **DaVinci Configurator Pro** is the starting point, coming with the SIP. To start the tool, use the file **DaVinciCFG.exe** located in the **CORE** folder of the **DaVinci Configurator** folder.

2 Set-Up New Project

DaVinci Configurator Pro offers a Project Assistant to set up that helps you setting up a new project file in just a view steps. The assistant guides you through several windows to enter necessary project information.

The **DaVinci Configurator Pro** creates your new project to the defined folder. The project folder contains:

- > **Appl** folder for generated files and source files.
- > **Config** folder for software components, BSWMD files, ECUC files, System description and **DaVinci Developer** workspace
- > **Backup** folder to store older DaVinci Developer workspaces
- > **Log** folder
- > Access to **DaVinci Developer** and **DaVinci Configurator Pro** via context menu on the *.dpa file.

Your development project consists of a folder structure on your disk, where all the configuration files, template files, etc. are generated to and where you implement your runnables using the empty runnable skeletons. Then you compile and link and get your executable that you can download to your hardware.

The DaVinci project file (DPA) is the central file which references all the related folders and files. It also references the SIP, which is used for the project.

2.1 DaVinci Configurator

As already mentioned the **DaVinci Configurator Pro** is the major tool for Vector MICROSAR for AUTOSAR 4.

2.1.1 The Main Window of DaVinci Configurator Pro

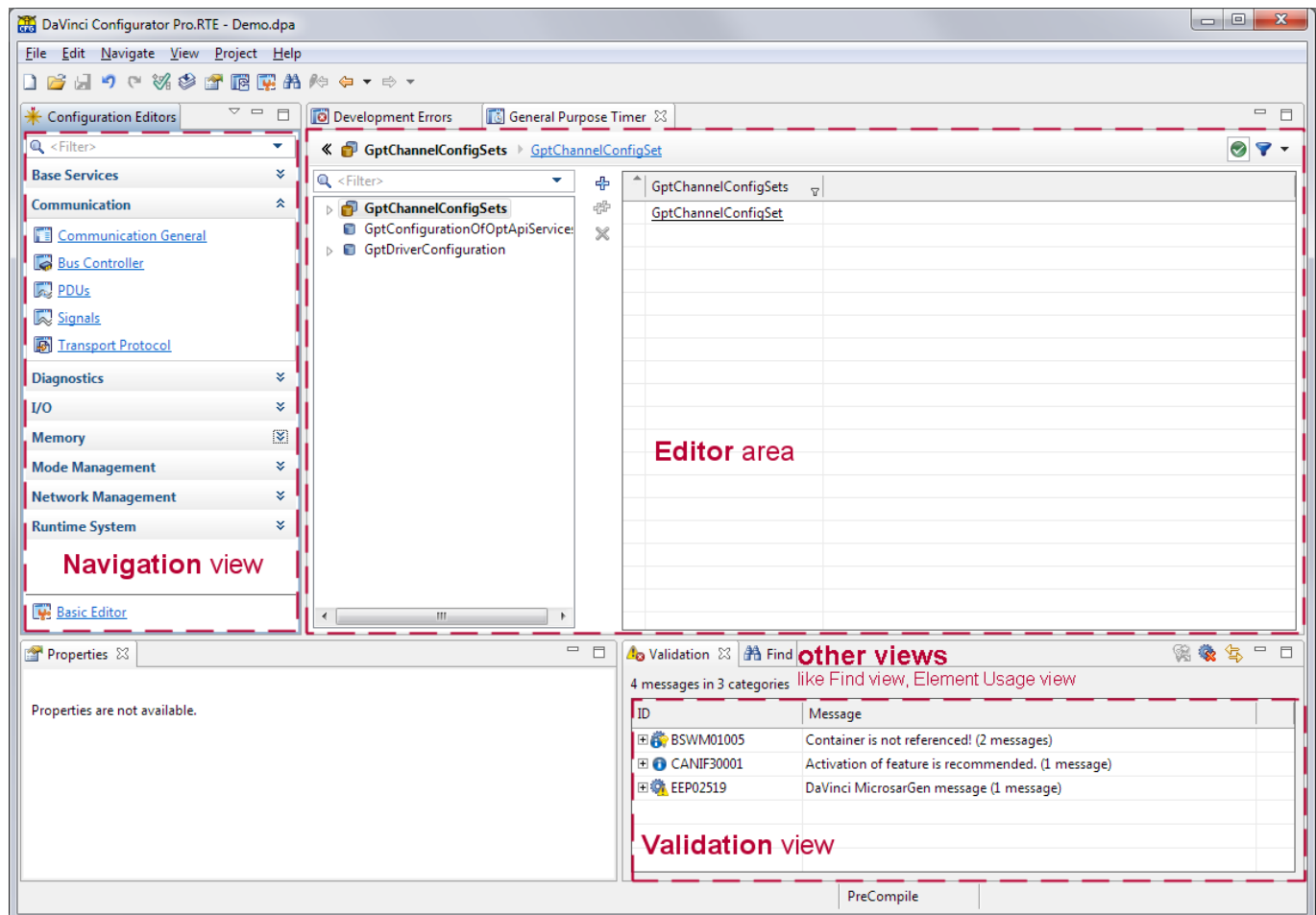
The illustration below shows the main appearance of the **DaVinci Configurator Pro**. It consists of 3 main areas or views. The positions of the views can be defined manually and depend on your habits in handling with tools.

- > Navigation view
- > Editor Area
- > Validation, Output and Properties view



Cross Reference

For more information about simple tool handling and icons refer to the help of the **DaVinci Configurator Pro**.



Navigation view

The Navigation view displays all available editors and assistants sorted by domain. What you select in the Navigation view will be displayed in the Editor view.

Editor area

The Editor area is the place to configure the modules or whatever you have selected in the Navigation view.

Validation view

The Validation view displays the overall list of validation messages. The messages are grouped by their ID. You may expand such a group to display all message of the same ID. By expanding an individual message, the related items are displayed as well as the proposed solutions, the so-called solving actions.

Using according commands in the shortcut menu you can navigate to the editors displaying the related items, or execute one of the proposed solutions for solving the problem.

You can use the Validation view with the following buttons:

> Solve All

Calls the default solving action of all messages that provide a default solving action.

> **Clear On-Demand Validation Messages**

Deletes all validation messages issued by the external generators during the on-demand validation.

> **Link with Editor**

Filters the Validation View to display only messages related to the currently focused editor.

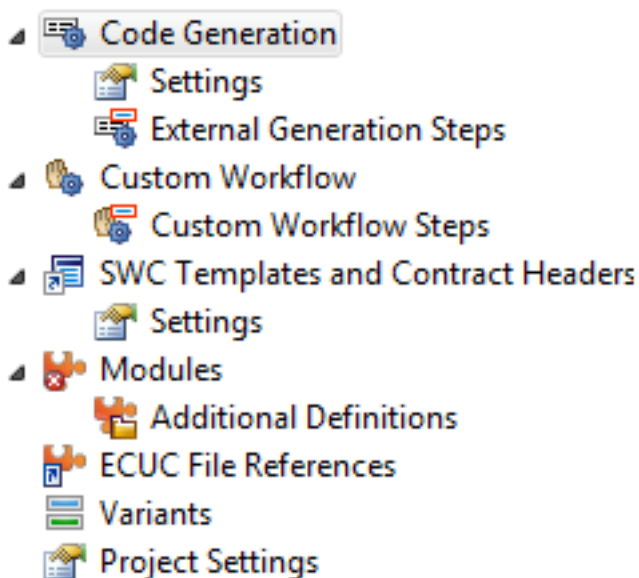
Properties view

The Properties view displays details of the object (parameter or container) selected in an editor. The view contains the following tabs:

- > **Definition:** Displays the definition of the selected object
- > **Status:** Displays information about the parameter state
- > **Description:** Displays the description of the parameter definition

Project Settings

This Overview shows all necessary project data and enables the configuration of customer workflow steps, the definition of external configuration steps and the activation of additional BSW modules.



Code Generation: Configure the code generation sequence

Settings: The code generation settings can be changed within this section

External Generation Steps: Define and execute a sequence of custom workflow step

Customer Workflow: Configure and execute a sequence of **custom workflow steps**

SWC Template and Contract Headers: Definition of additional paths to BSWMD files relevant for the project.

Modules: Activate or deactivate modules

Additional Definitions: Definition of additional paths to BSWMD files relevant for the project.

Additional Definitions: Definition of additional paths to BSWMD relevant for this project

ECUC File Reference: Include mechanism for external ECU configuration files from other projects.

Variants: Handling of Variants when used.

Project Settings: The project settings can be changed within this section.

2.1.2 Editors and Assistants

Generally spoken there are two different configuration editor concepts in the **DaVinci Configurator Pro**.

- > **Domain-specific** configuration editors, which provide an optimized view on the ECUC
- > **Basic** Editor, which provides a native view on the ECUC

Assistants

An assistants guides you through special tasks like connections, data mapping, memory mapping or task mapping.

[Add Component Connection](#)

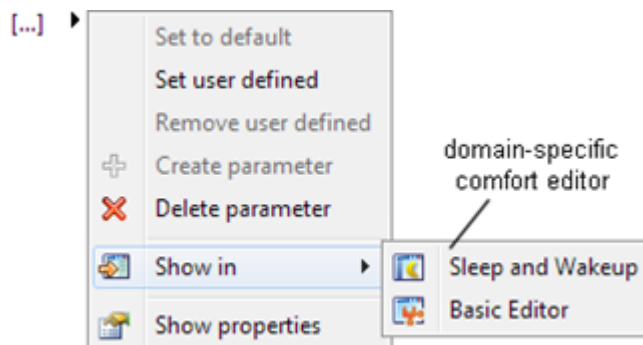
[Add Data Mapping](#)

[Add Memory Mapping](#)

[Add Task Mapping](#)

Switch Between the Two Editors

You can switch from the basic editor to the domain-specific comfort editor and back. Move to a setting or a value and click the little triangle to open the context menu. If there is a pendant in the other configuration editor, you can switch from one to the other via **Show in**.

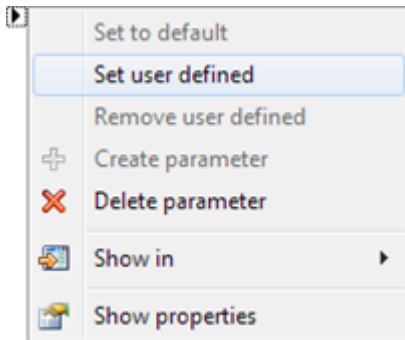


Set User Defined



It is very important to understand that you are responsible for these modifications!!

Move to a setting or a value and click the little triangle to open the context menu.



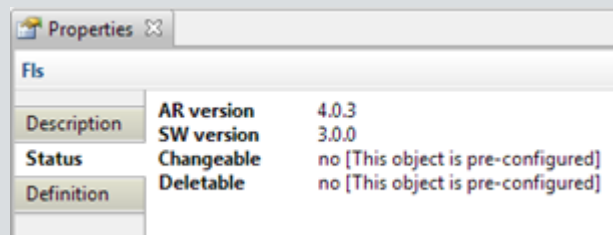
With **Set user defined** you can change values even when they are grayed because their values are derived from the input files or locked by a configuration dependency.

When you use **Set user defined**, the parameter will be marked with a little blue pin on the left side. In the example illustration below the upper check box is marked as user defined, the lower one is not.



Note

Set user defined will not work for pre-configured parameters. The setting in the context menu is grayed and cannot be used. The information about predefined or not can be seen in the **Properties** view. Select **Status** and look at **Changeable** or **Deletable**.



Remove user defined

To set back the user defined status, use the context menu and select **Remove user defined**.



Note

Every parameter set to user defined will provoke a warning during validation with the information, that this parameter was set to be user defined.

**Caution!**

The feature **set user defined** enables settings that might be terribly wrong. As soon as you use this feature, you have to be sure that you know what you do and you have the complete responsibility.

3 Define Project Settings

Your first steps after the new project is set up, are:

- > Define Input files
- > Define your specific workflow steps and external generation steps (if needed)
- > Activate all necessary BSWs

3.1 Input files

The **DaVinci Configurator Pro** needs information about your system, the bus communication, etc. This information is stored in the SYSEX or when using legacy file formats dependent on the bus system, in the **DBC**, **LDF** or **FIBEX** files. One or more (e.g. for systems with multiple channels) of these files must be given to the **DaVinci Configurator** as input files.

The necessary diagnostic information is stored in **CDD** or **ODX** files and is also needed by the **DaVinci Configurator Pro**.

3.1.1 System Description Files

You need one of the following description files:

3.1.2 SYSEX

ECU-specific extract of the System Description. This file is typically provided to the TIER1 by the OEM. It contains

- > ECU composition
- > Atomic SWCs
- > Compositions
- > Communication
- > Data mapping
- > typically NO service SWCs

3.1.3 ECUEX

Base for the ECU development is the ECU Extract of System Description, ECUEX for short. It is created from the SYSEX by flattening the SWC architecture. This means that SWC compositions have been removed, only the atomic SWCs are left. And they now communicate directly with each other. Additionally to the SYSEX, the ECUEX contains:

- > Service SWCs
- > Service mapping, i.e. service connections between the ports of the SWCs and the ports of the service SWCs.

3.1.4 Legacy Data Base files (DBC, LDF, FIBEX, ...)

The legacy data base files are normally not in ARXML format. They also contain information about EUCs, messages, signals, attributes, etc. Good AUTOSAR tools can read them and convert their information into AUTOSAR-conform notation. The missing information when using legacy file formats must be given to the configuration tool by the software developer.

3.1.5 Diagnostic Data Files

Your Project uses Diagnostic? Therefore you need one of the following diagnostic description files:

3.1.6 CDD / ODX

These files contain the diagnostic description for the ECU.

> ***.CDD**

The **CANdelaStudio** Document (*.cdd) format is specified by Vector for describing the diagnostic feature set of an ECU.

> ***.ODX/*.PDX**

ODX (Open Diagnostic Data Exchange) is an ASAM standard which defines a unique, open XML exchange format for diagnostic data. A packaged ODX file (*.PDX) comprises all files of an ODX project.

3.1.7 State Description

Diagnostic session and security level management is modeled as a state machine which describes the transitions between sessions and states triggered by the execution of diagnostic services.

If a CANdela Document (*.CDD) is used as diagnostic description file, all diagnostic session and security level related information will be imported from the input file.

If a Packaged ODX file (*.PDX) is used as diagnostic description file, it will depend on the used ODX version if an additional state description is required.

> **ODX 2.2.0**

For ODX 2.2.0, the state machines are defined within the ODX file, using the data model intended for this purpose.

> **ODX 2.0.1**

As ODX 2.0.1 does not provide means to explicitly model diagnostic sessions, security levels and their corresponding transitions, dedicated SDGs (special data groups) are used to annotate the ODX data with supplemental information about the diagnostic session behavior. This information is not sufficient in all cases and an additional state description will be mandatory to augment the ODX data with the required information during the import of the diagnostic description.

A state description is a *.CSV (comma separated values) file which defines the transition matrices of the diagnostic session and security level state machines.

3.1.8 Standard Configuration Files

In some cases a OEM-specific preconfiguration is necessary, therefore additional Standard Configuration Files provided by your OEM are necessary. Add fragments of ECUC modules which will be used as project specific mandatory configuration.

3.2 External Generation Steps

In STEP7 Code Generation on page 64, after the configuration, **DaVinci Configurator Pro** generates code. You can additionally let **DaVinci Configurator Pro** call any external tool during this step in a free configurable order. If something is to be done before code generation, if you use some command line

tools or whatever, add these tools to the list and define the call order. With STEP7 Code Generation on page 64, all your mentioned tools will be also called.

3.3 Activate BSW

Which modules are activated and which are not depends on the information in the input files. Activate or deactivate the modules.

4 Validation

**Nothing done but imported some files and already many mistakes in the configuration?
Why is that?**

The **DaVinci Configurator Pro** provides powerful validation routines that run in the background named **Live Validation**. At start-up of your project, there is only the information from the input files of STEP2 Define Project Settings on page 26. The content of these input files normally has leaks, sometimes errors or the information of the system is still missing and has to be accomplished during configuration. This all leads to a number of errors detected by the validator of **DaVinci Configurator Pro**. But with the powerful solving algorithms and assistants, it is very easy to solve off the first warnings and errors.

4.1 Validation Concept

The validation concept comprises:

- > Detailed validation of the ECU configuration
- > Comprehensive set of domain-specific validation rules
- > Navigation from validation message to the editors
- > Tool makes proposal for solving an error (solving actions)
- > Automatic consistency (auto-solving action) after changing the configuration via Comfort Editors or via the Basic Editor

Live Validation

The Live Validation is performed in the background after loading a project or after changing the configuration. The Live Validation directly gives feedback after entering wrong values.

On-Demand Validation

This validation is performed when you start a generation process. You can start the validation via **Project | on-demand validation**. The on demand validation will also launch external generators or execute more complex validation tasks that cannot be executed live.



Note

The result of a validation is displayed in the **Validation View**.

5 BSW Configuration With Configuration Editors

5.1 DaVinci Configurator Pro Editors

To configure the BSW modules you can use the **Basic Editor** or the more comfortable and cluster-specific **Configuration Editors**.

Basic Editor [Basic Editor](#)

The Basic Editor is a generic configuration editor (GCE) and includes the native view of the ECU configuration. It is based on the basic software module description (BSWMD) format and displays all modules of the ECU configuration.

Configuration Editors Configuration Editors

The Configuration Editors offer a use-case oriented view of the ECU configuration. They are optimized for displaying or configuring those parts of the ECU configuration, which are related to the use case.

Use the Configuration Editor first. To get the necessary settings for your first step in dependency of your OEM and its use cases refer to [Start Configuration with Configuration Editors](#) of the step by step description.

6 Software Component (SWC) Design

6.1 Data Exchange between DaVinci Developer and DaVinci Configurator Pro

The data exchange between **DaVinci Developer** and **DaVinci Configurator Pro** is done via the **DaVinci Developer** workspace, file with **DCF** format.

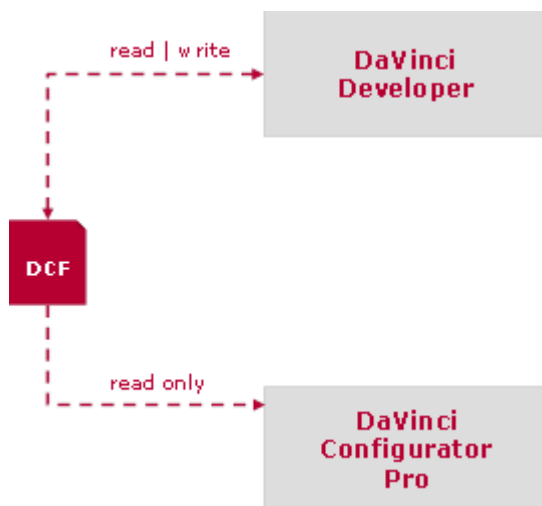
DaVinci Developer uses this file to store the complete project information. It can read and write this file.

DaVinci Configurator Pro only reads this file and only at tool start-up.



Note

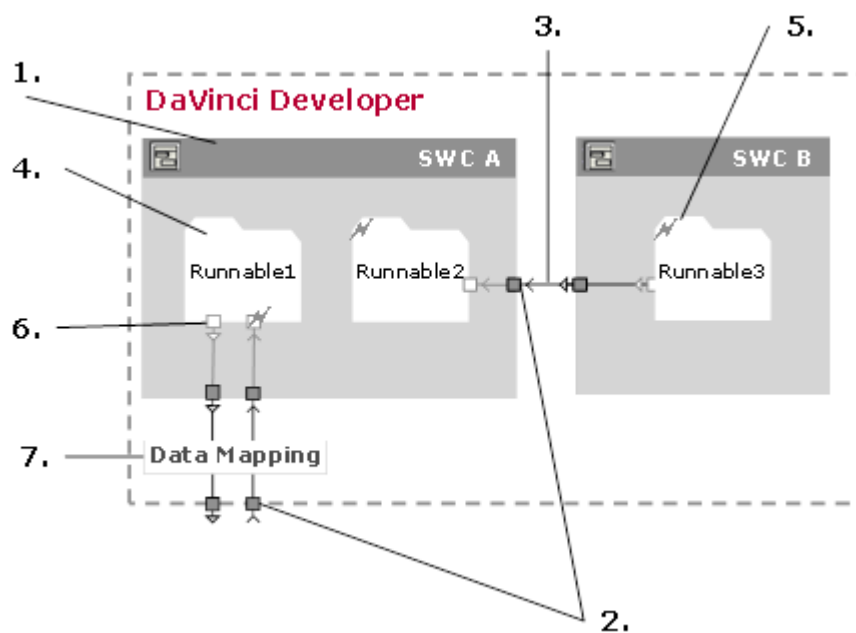
Please re-open the **DaVinci Configurator Pro** if you have changed the DCF file via the **DaVinci Developer** to get the current data from the DCF file.



Keep in mind that the DaVinci Configurator Pro is not able to write to DCF file.

6.2 About Application Components, Ports, Connections, Runnables and More...

Learn all about application components, ports, connection and runnables, how they work and how to exchange information between application components.



1. Application Components
2. Ports, Port Init Values and Data Elements
3. Connections
4. Runnables
5. Triggers
6. Port Access
7. Data Mapping



Cross Reference

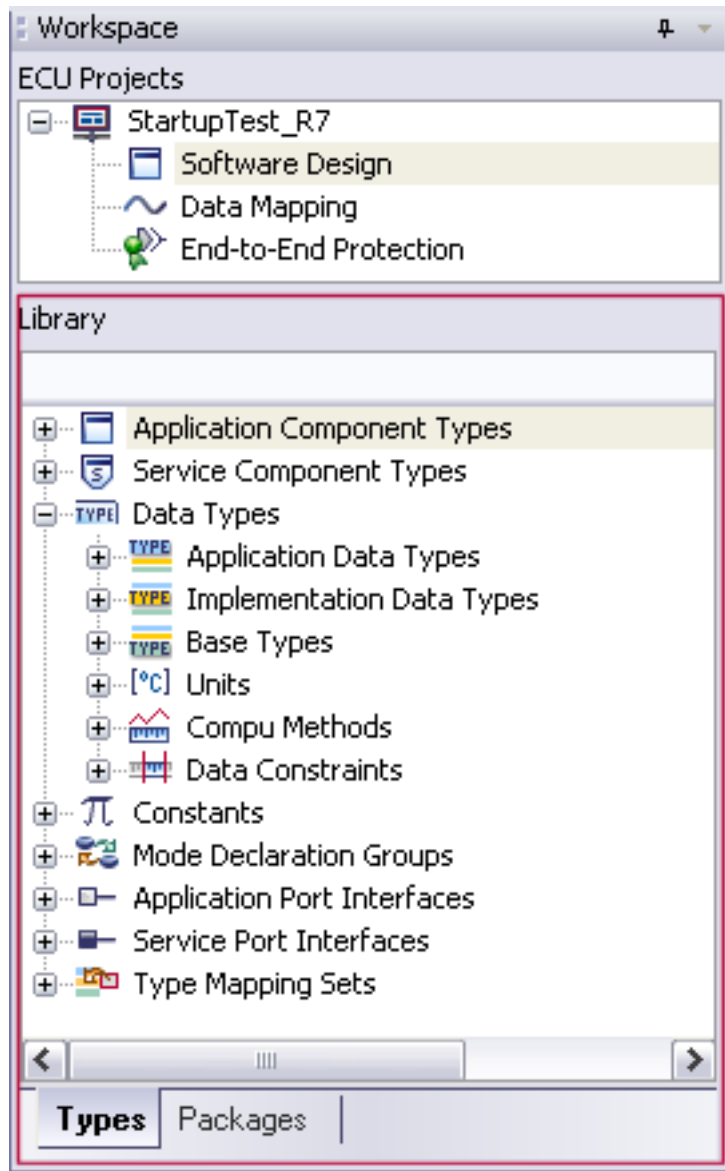
To get basic tool handling information, refer to the online help of the [DaVinci Developer](#).

6.3 Application Components

To create application components use the [DaVinci Developer](#).

6.3.1 The Library – Type and Package

You find the Library of the DaVinci Developer (in standard configuration) below the ECU Projects.



Types or Package?

The Library you use in the following steps can basically be used in two ways.

- > Type-oriented
- > Package-oriented

The type-oriented workflow is shown in the following steps. But you can also choose the package-oriented one. This concept will now be described briefly.

Packages

With Packages you can pack elements together like:

- > Application Component Types
- > Service Component Types

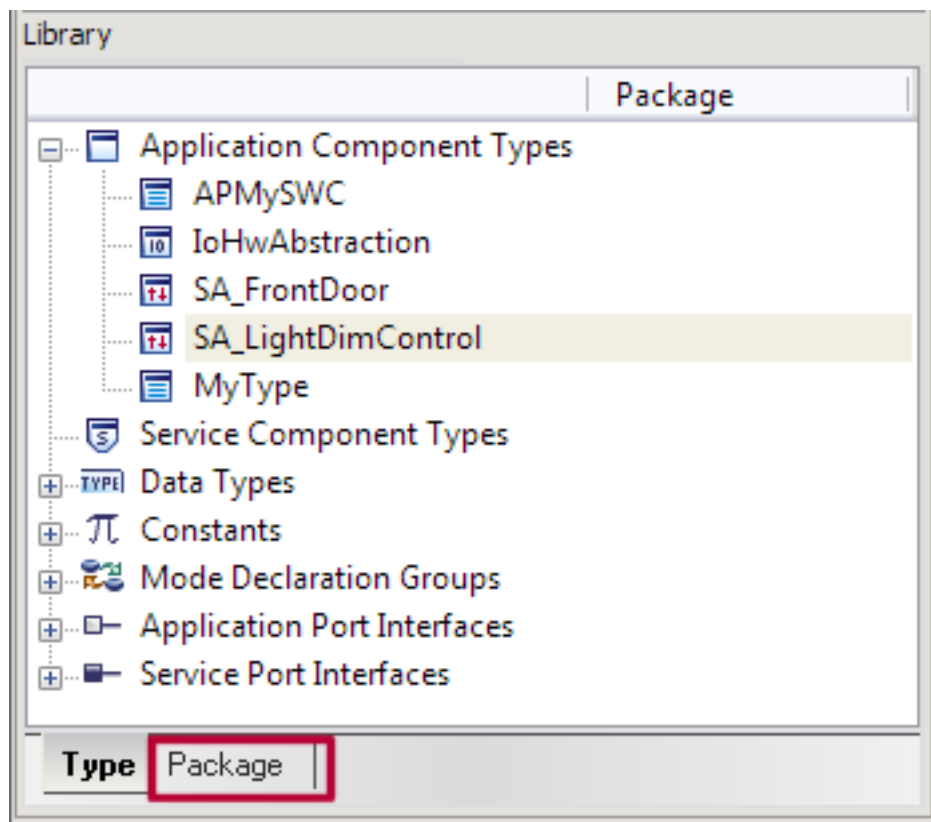
- > Data Types
- > Constants
- > Mode Declaration Groups
- > Application Port Interfaces
- > Service Port Interfaces

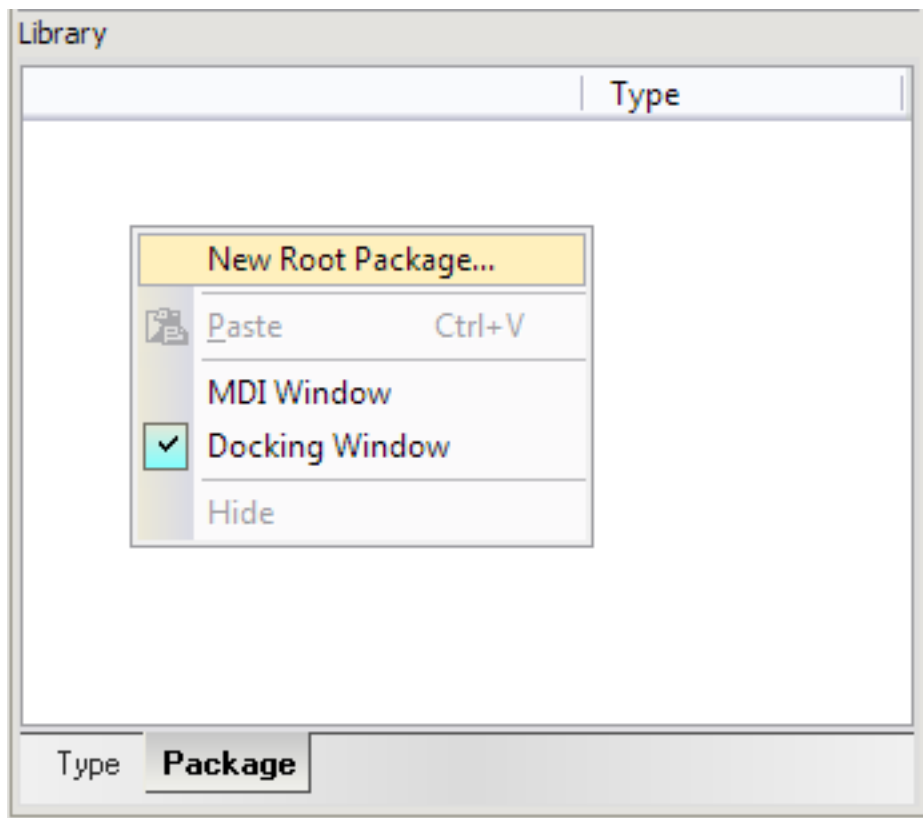
It is like a kind of grouping with its own name space. A name of an object has to be unique within a package but can be used more than once within the project.

These packages can be exported and imported for round-tripping with other tools also supporting the packages.

Switch to package view

At the bottom of the Library you can select Type view or Package view. Select the Package. You need a Root Package first. Create it via right-click in the empty Package view.

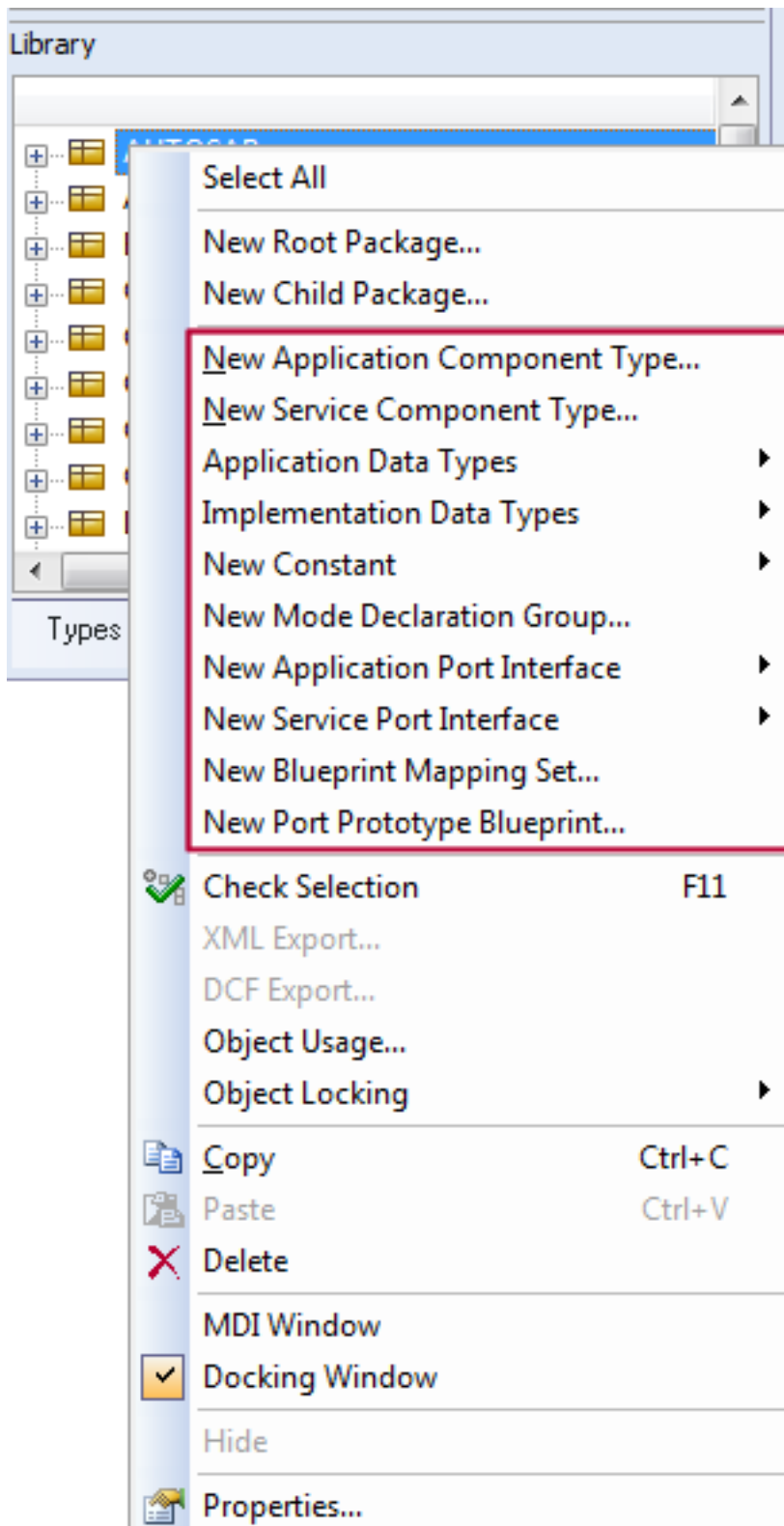




New Child Package...

Then right-click the root (e.g. MyECU_PartA) and you will get to the context menu. Use New Child Package to nest the packages or you put the objects plain below the root package.

The context menu for the packages



Add Objects

Via Add Objects you open a window to select and add already existing object to your package.

New ...

The section New... of the context menu comprises all available objects which you can also create via the Type view of the Library (explained in the following chapter).

XML Eport..., DCF Export...

Export your package as XML file or as DCF file.

Object Usage...

Shows dependencies between used objects in a tree-view.

Object Locking APMysWC

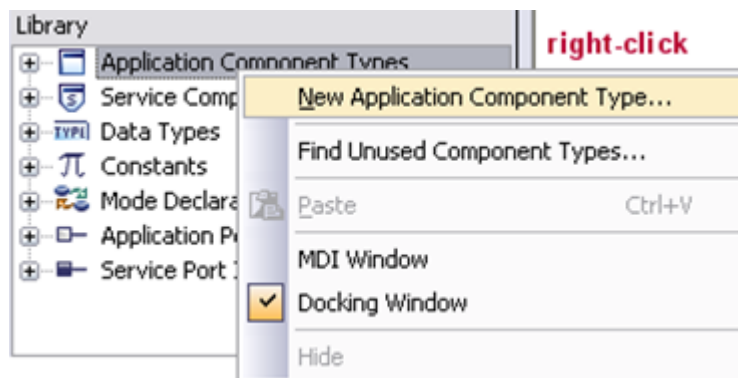
Objects can be locked and unlocked. A locked object is displayed with a little lock. Locked objects cannot be deleted without being unlocked before.

Define new Component Types in the Library

Right-click on Application Component Types in the Library and select New Application Component Type.

6.3.2 New Application Components

Right-click on Application Component Types in the Library and select New Application Component Type.



It is good to give a speaking name to see what kind of component it is. e.g. AP – Application, SA – Sensor / Actuator

Enter a Name for the Application Component Type. Select Composition if the SWC should contain other SWC or select Atomic. Select its Type and confirm with [OK]. All application software component types are displayed in the library.

New Component Type

Properties Mapping Sets Included Data Type Sets Package Attributes Description

Name:

☐ Composition

☒ Atomic

Type:

☐ Sup

Implement

☒ Application (End-to-End Protection Proxy)

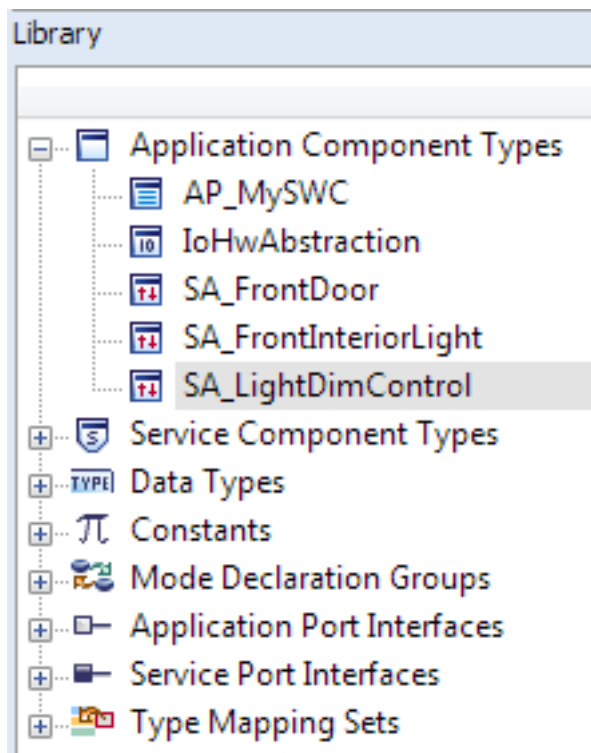
☐ Object code

Symbol

Short Name:

Code Symbol:

OK Cancel Apply now



Note

To get fast information about any element in the library just select one and see its Properties, Port Prototypes (if available) and Description in the Properties view (bottom left).

Component Type		
Name	AP_MySWC	
Package	/ComponentTypes	
Realization	Atomic(Application)	
Code Type	source code	
Multiple Instances	no	

Properties

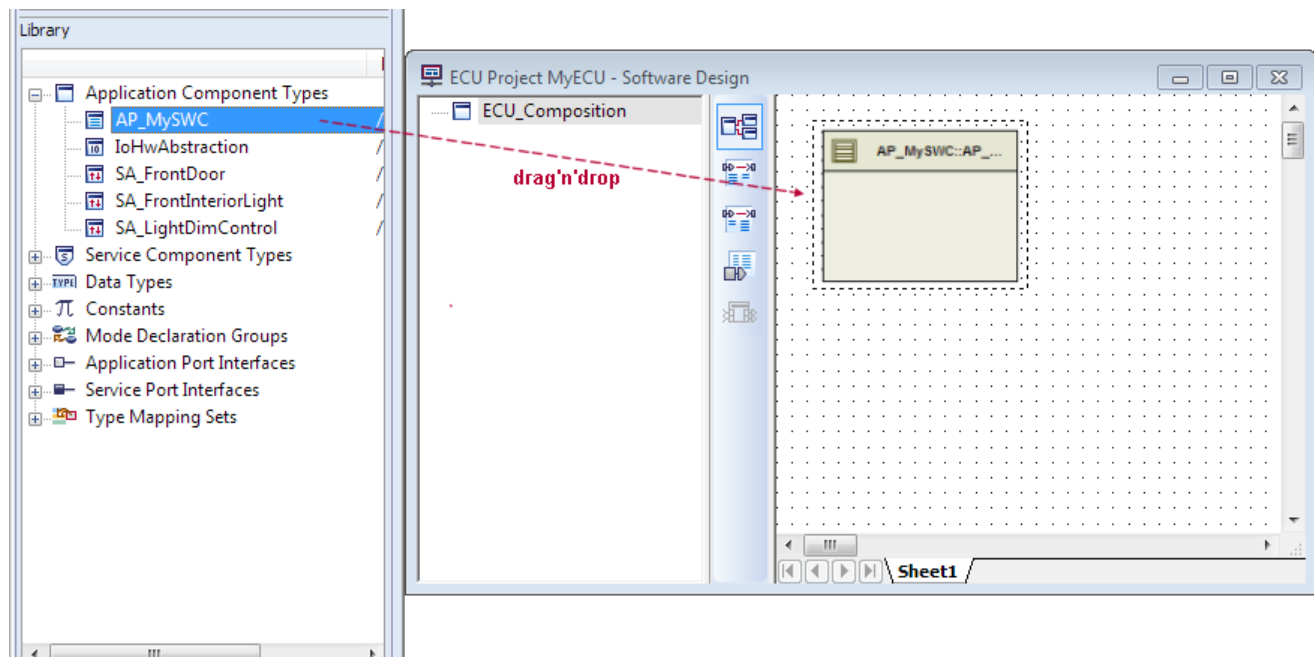
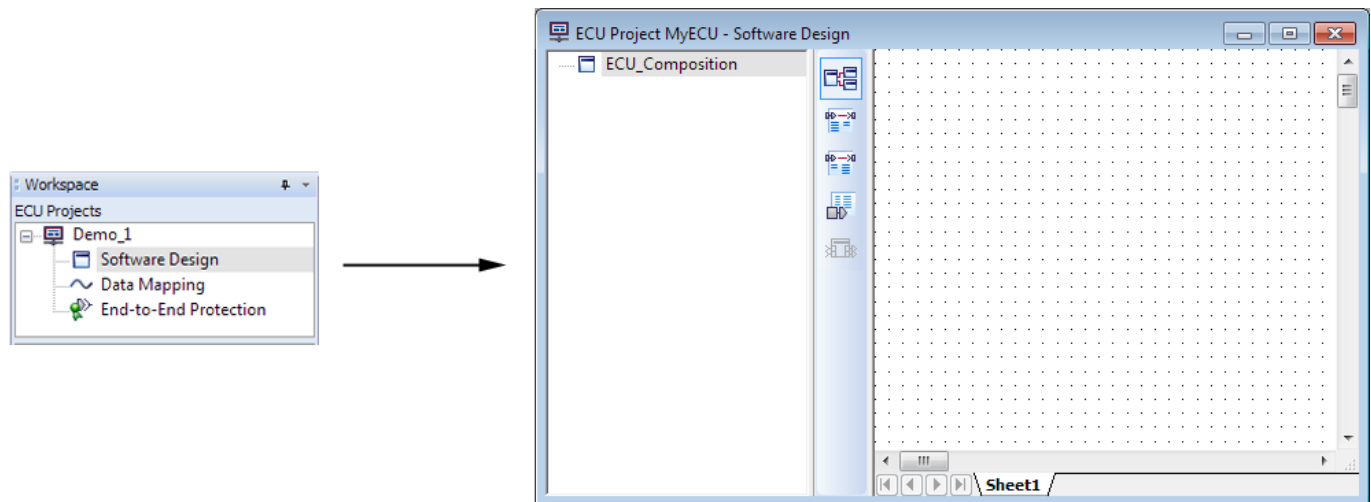
Properties

Port Prototypes

Description

Component types become component prototypes by being used.

Double-click Software Design in the ECU Projects view to open the Software Design view for your ECU project (e.g. MyECU).



Note

In the graphical representation of an application component prototype you can change its size using the little squares shown at the angles and in the middle of the sides.

An application component type becomes an application component prototype when it is used. It also needs a name. Using drag'n'drop, both names are the same. Open the properties window from the context menu for a component prototype to change the prototype name.

Define and use as many application software components as you need for your ECU.

**Note**

The notation of the software component starts with the software component prototype followed by the software component type.

6.3.3 Understand Types, Prototypes and Interfaces

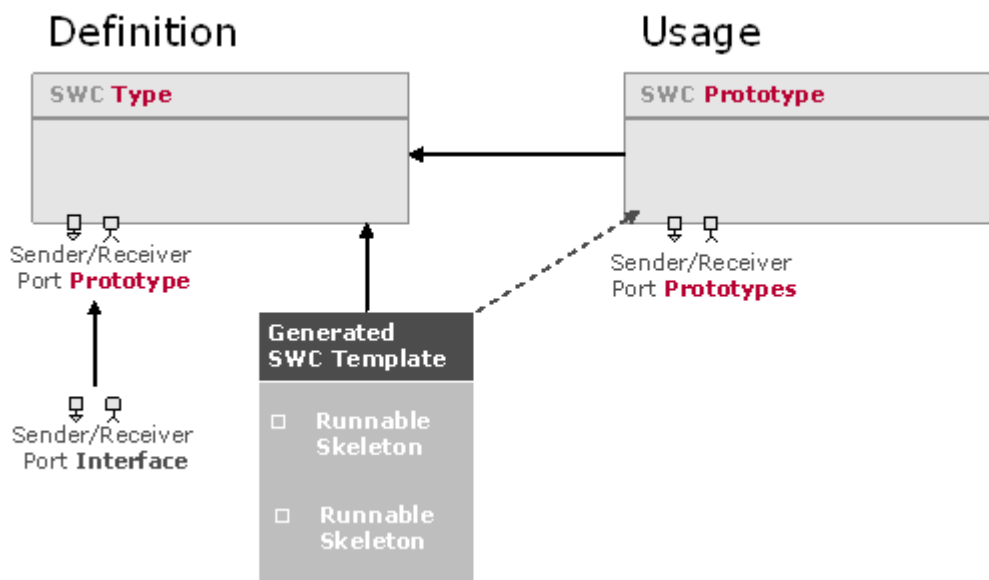
This illustration will help you to understand working with the **DaVinci Developer**. You have to deal with

- > Prototype
- > Type
- > Interface

When is a software component a software component type, when a software component prototype?
When is a port a port interface, when a port prototype?

In the library, the software components are types, the ports are interfaces. As soon as you use them, they become prototypes.

- > Port Interface used by a component type → Port Prototype
- > Component Type in library used in software design view → Component Prototype



Runnables are always connected to the component type.

6.4 Ports, Port Init Values and Data Elements

To communicate and to exchange information the components need so-called ports (S/R ports).

For communication between software component ports have to be defined.

There are different kinds of application/service ports,

- > **Sender Ports**  to provide information
- > **Receiver Ports**  to receive information
- > **Sender/Receiver Ports**  to provide and receive information within one port
- > **Server Ports**  to provide services (operations)
- > **Client Ports**  to use services (operations)

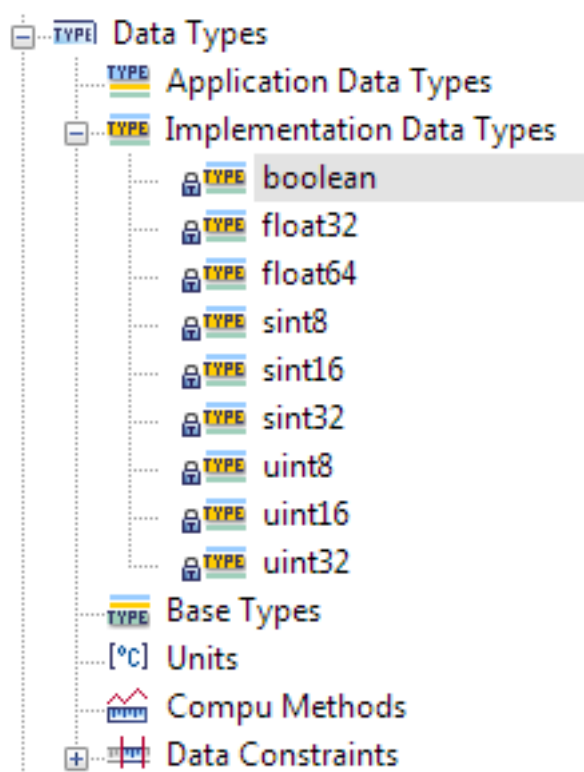
The following ports are listed here for completeness.

- > Calibration Ports to hand over calibration parameters
- > Mode Ports to e.g. trigger or not trigger runnables within certain modes

Before you can use application ports you have to define application port interfaces. To completely define the port interfaces you have to define data types first, if you don't want to use the predefined ones.

Predefined data types in library

Just right-click and select from the list.

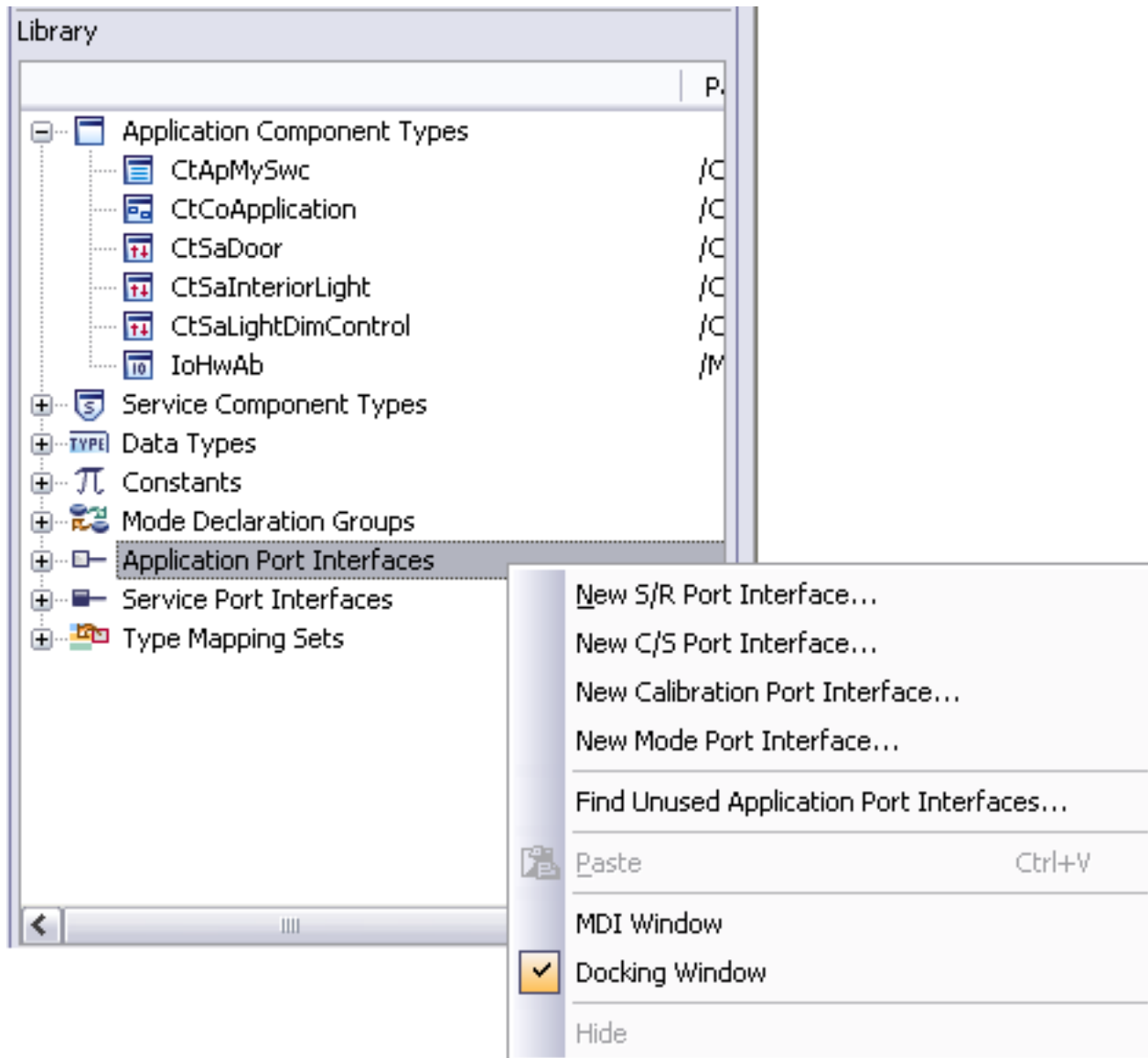


Caution!

You can handle application port interfaces like application component types. Define the application port interfaces in the library and then use them as application port prototypes for each application component. The same applies for data types and data elements.

Create Port Interface

To define a new application port interface, right-click the Application Port Interface in the library and select New S/R Port Interface....

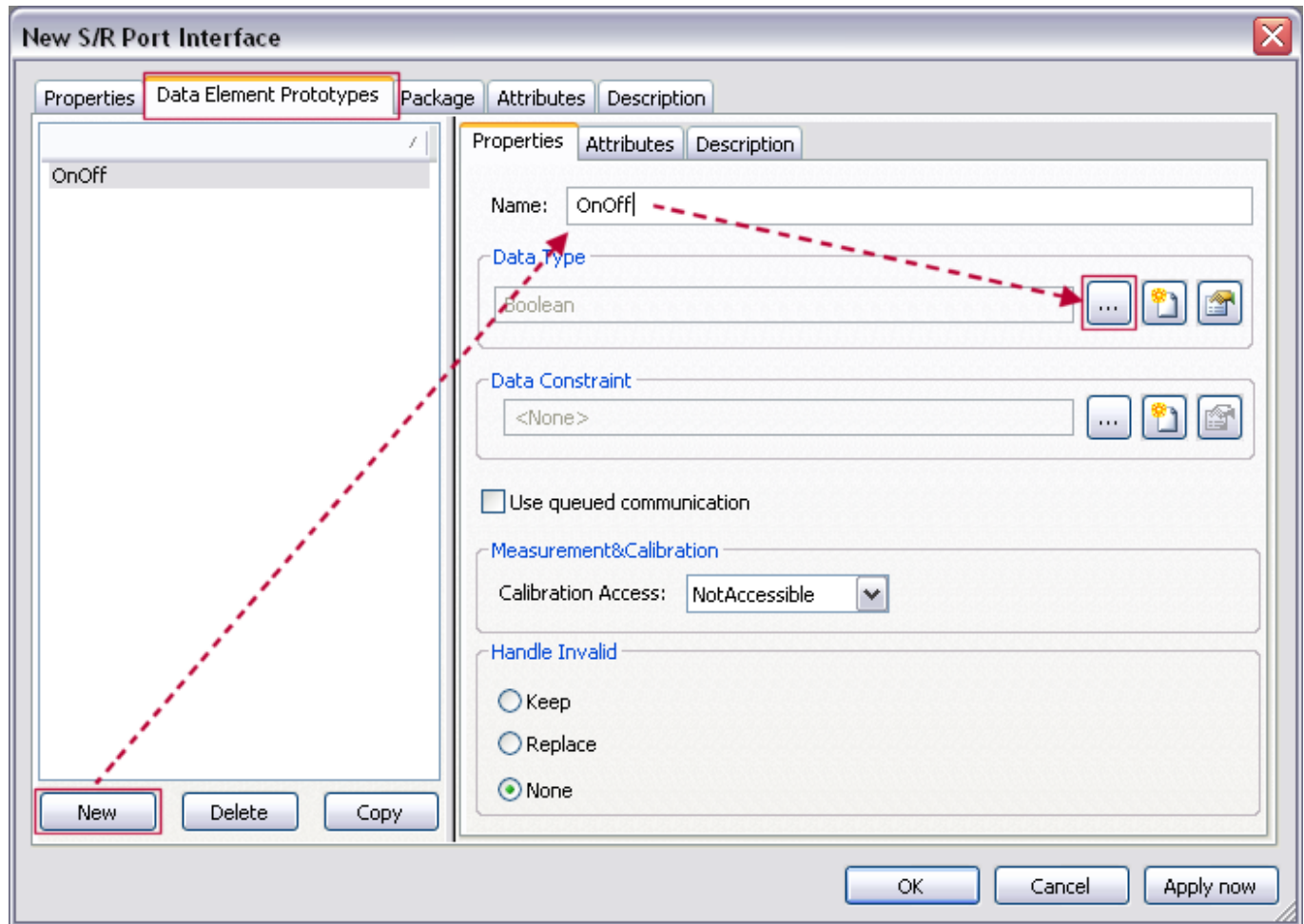


Then the window for the S/R Port Interface opens. Enter a name, select the tab **Data Element Prototypes** and define the content of the port. In this case it is just one data element called OnOff with the Data Type Boolean, use the button [...] to select Data Type.



Note

Data elements are the contents of ports.

**Note**

A port interface can carry many data elements of different data types.

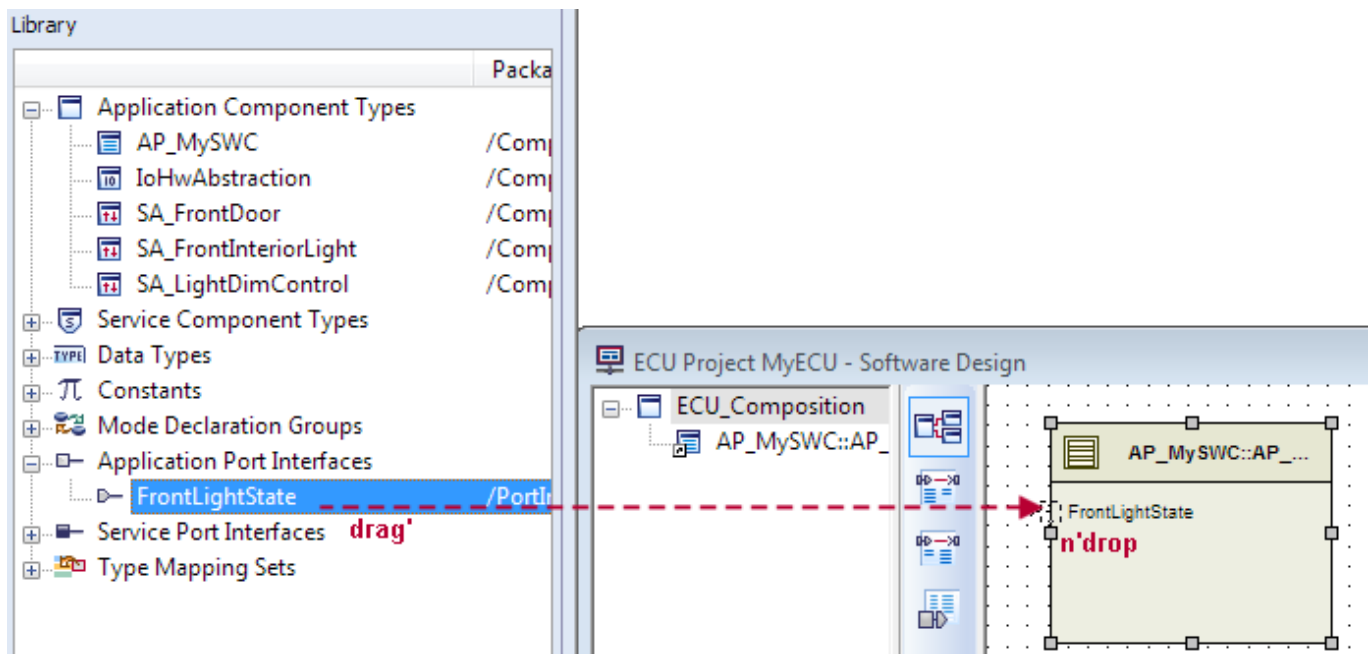
Provide software components with ports

- > You have created your application software components.
- > You have created necessary application ports interfaces and their content (data element prototypes).

Now you have to define which application component needs which application ports.

Add ports to the application components via drag'n'drop

Select an application port interface from the library and place it onto the application component via drag'n'drop. This transfers a port interface into a port prototype.



Note

Press <Ctrl> while drag'n'drop to change between sender or receiver port.

Interfaces

The ports are added to the components as graphical element together with the name of the port.

- > **Sender port**
- > **Receiver port**

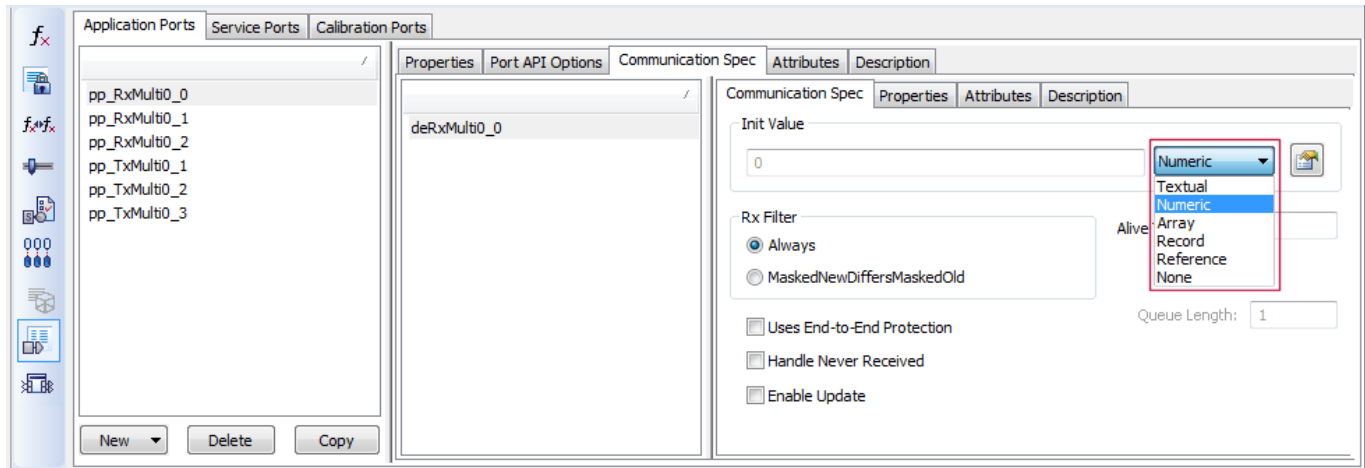


Note

Ports need Init Values.

Port Init Values

You have to assign an initial value to every used application port.



6.5 Configure Service Ports within your Application Components

Your Service Components will be loaded automatically to the **DaVinci Developer** workspace .



Note

The **DaVinci Configurator** stores the service components within the folder **<Project Folder> \Config\ServiceComponents**.

The content of the service components are not part of the **DaVinci Developer** workspace.

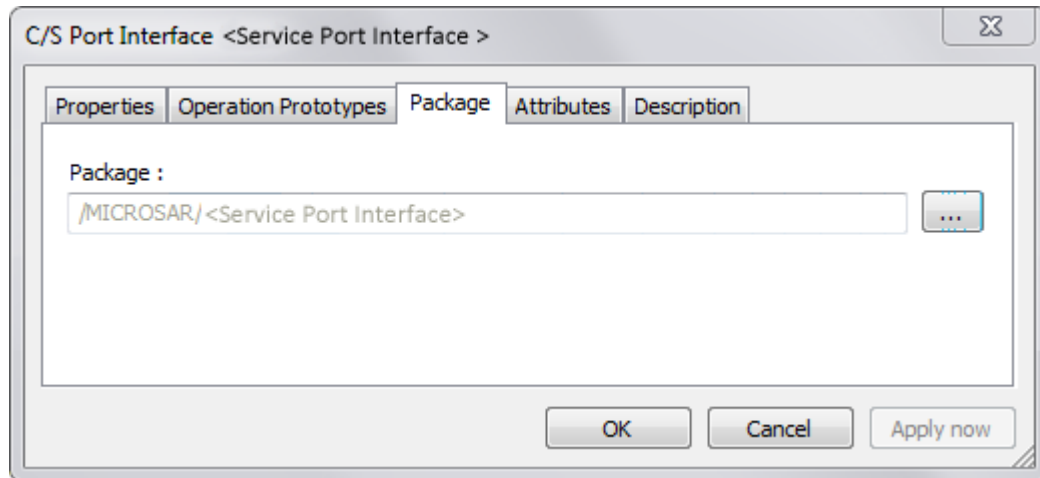
If you want to connect one of your Application Component to a Service Component, you have to define a Service Port Prototype based on a Service Port Interface.

Service Port Interface name and content depend on the BSW configuration and version of the service component within the **DaVinci Configurator**, so it is not stable. An Application Component shall always define its own or a general Service Port Interface definition.

The Service Port Interface definition from the loaded Service Component is always in read-only status. These Service Port Interfaces are marked with a **read-only icon**  within the library.

You have currently not defined your Service Port Interfaces?

Then copy and paste the definition from the Service Component. Assign the definition to an own package, so it is possible to use the same name as before. Therefore open the Service Port Interface, select **Package** tab and choose a package via [...].



Assign your copied Service Port Interface to your Application Component. Therefore open your Application Component Prototype, select **Port Prototype List** , open **Service Ports** tab and add your Service Port via **[New] | From Port Interface**. Now the Service Port used by your Application Component is independent from the Service Component description.



Note

If incompatible changes apply these will be reported by RTE checks.

6.6 Define your Runnables

Now configure runnables that will carry your code.

There are four important topics for a runnable:

- > **SYMBOL** and **NAME** – what the runnable skeleton is called in the template file
- > **TRIGGER** – when is the runnable executed
- > **PORT ACCESS** – what data can the runnable access
- > **MAPPING** – in which task context does the runnables work?



Caution!

Runnables can only be defined for atomic application components types.



Note

The runnable skeletons are generated by the **DaVinci Configurator Pro**/the RTE and can then be accomplished with your code. Find more details follow in the next chapters.

Check also later hints to template generation

1. Open a software component via the library or the software design view.
2. Click on the runnable icon [fx].
3. Click [New] and select Runnable.
4. Enter Name and Symbol for the new runnable on the Properties tab.

If you leave the Symbol field empty, the runnables (functions) will be named according to the entry in the **Name** field.

Minimum Start Interval

Define the minimum time interval between the successive triggering of this runnable.

Can Be Invoked Concurrently

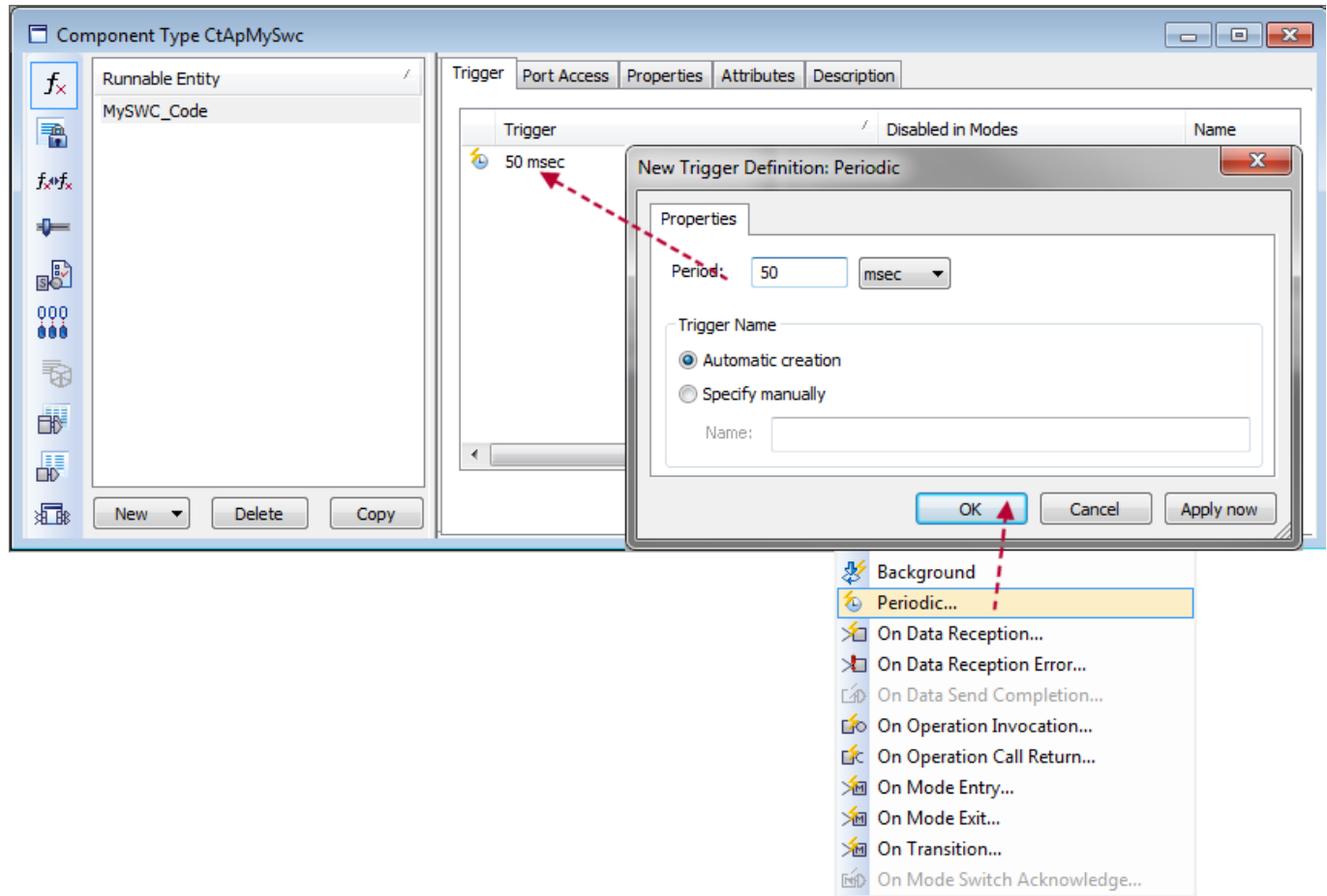
A runnable can be marked as **Can be invoked concurrently** if it can be executed while it is already running. In other words it can be safely executed concurrently (re-entrant). There are a few criteria for your implementation code:

- > No static (or global) non-constant data
- > No return of address to static (or global) non-constant data
- > Only work on data provided by caller
- > No modification of own code at runtime
- > No call of non-re-entrant runnables

6.7 Triggers for the Runnables

The trigger decides when a runnable is executed.

Select the tab Trigger. On this tab you select when your runnable should be executed.



Note

A runnable can be triggered periodically or via an event.

Periodical: Select the checkbox and enter the cycle time.

On Data Reception: As soon as data is received at the appropriate port the runnable is activated. (Indication)

The trigger **On Operation Invocation** belongs to service ports and will be explained later.

The runnable of the demo application only needs to be called when the state of the doors is changed. This can be realized via a cyclic polling or directly by a trigger from the doors. In the demo, **Periodical...** is chosen.

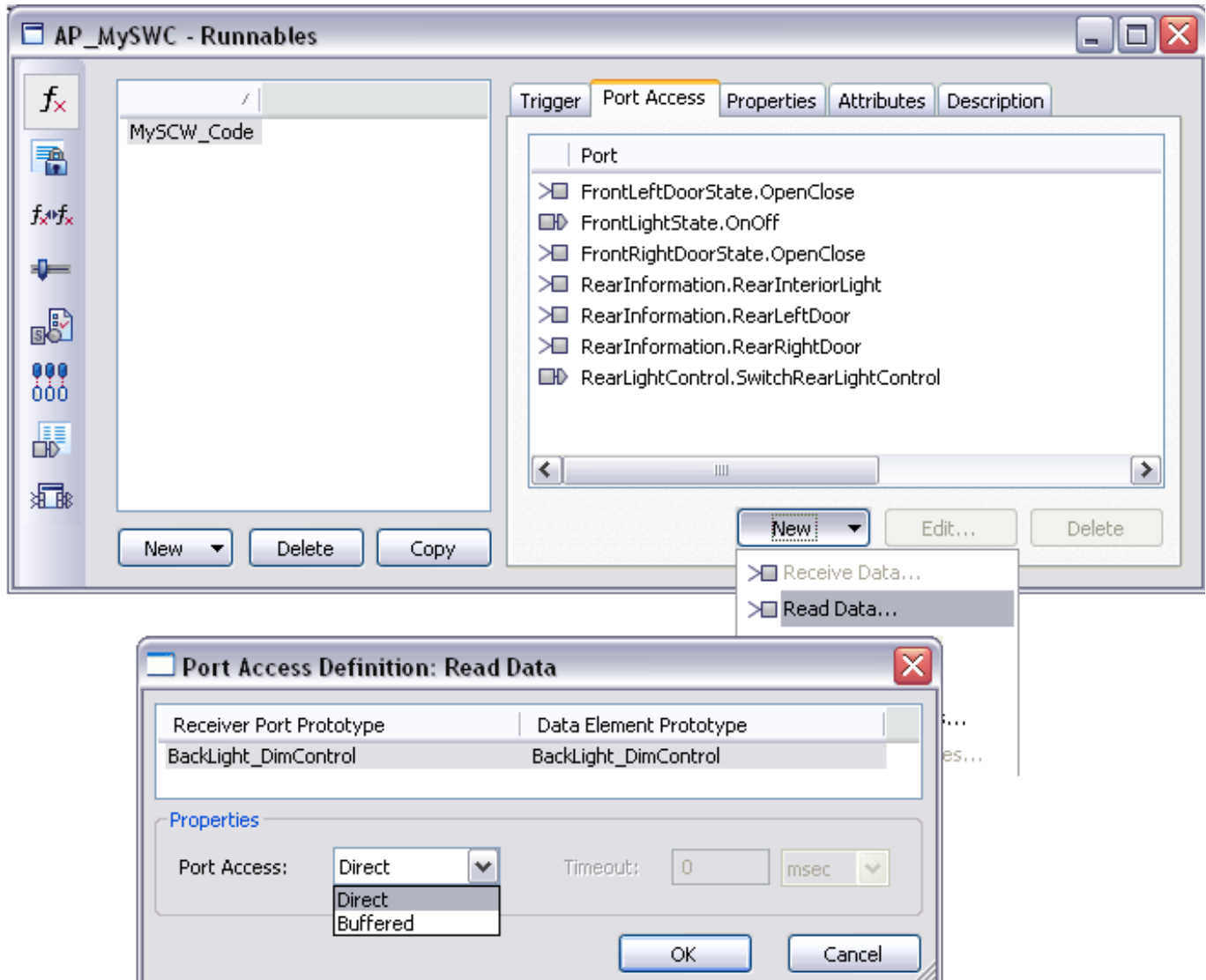


Note

When experimenting with the demo, replace the cyclic trigger with two triggers On Data Reception..., one for the left and one for the right front door.

6.8 Port Access of the Runnables

Define which port information each runnable should be able to read or write. Select the tab Port Access. You only get displayed accessible ports.



You can define access to

- > Read Data... and
- > Write Data...
- > and later when using client server port also to operations (Invoke Operations).

Click e.g. **Read Data...** and the Port Access Definition:Read Data window will open.

**Caution! Direct – Buffered**

Direct Write: as soon as you write the information to the data, it is changed immediately.

Buffered Write: the data information is changed at the end of the runnable runtime just before leaving it.

Direct Read: if you access to the data multiple time, it could be changed in the meantime. You always read the current information.

Buffered Read: with the start of the runnable, the data is copied to a buffer. Every time you access the data, it has the same value until the runnable is left.

**Note**

In the header of the runnable's skeleton that will be generated by the **DaVinci Developer**, you will find a list with all available API functions for this runnable. If any access is missing, go back to the **DaVinci Developer** and check whether the Port Access is set correctly.

Summary

Summary of the settings for your runnable MySWC_Code:

- > The runnable is called MySWC_Code
- > Its functional representation is called: MySWC_Code.
- > It is triggered periodically
- > The runnable has access to the state of the left and right door and to the data of the front interior light. It also has access to the LightDimControl.

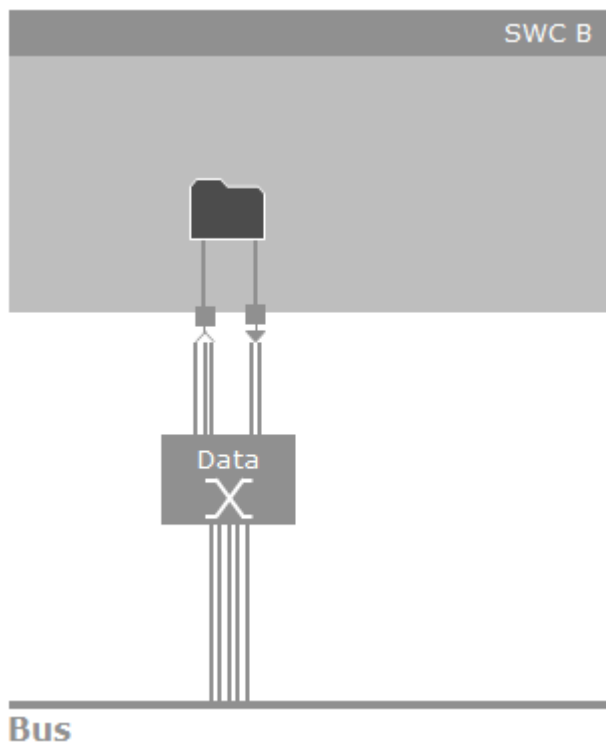
7 Mappings

Generally spoken Mapping stands for assignment of one object to another. There are different mappings in the context of AUTOSAR. What is meant here is:

- > Data Mapping
- > Task Mapping
- > Service Mapping
- > Memory Mapping

7.1 Data Mapping

Your ECU has to communicate with other ECUs, the external communication. Signals have to be sent and received via the connected bus system. The signals and the types of data transported via the signals is defined in the data base like DBC, LDF, FIBEX, and also in the SYSEX.



From the view of software components, communication information is exchanged via ports that carry so-called data elements. The definition of a port contains the assignment of data elements that can pass the port. Via the Data Mapping you define, which data element belongs to which signal. For short, you assign data element to bus signals. That's called Data Mapping.

**Note**

Data Mapping can also be done in **DaVinci Configurator Pro**!

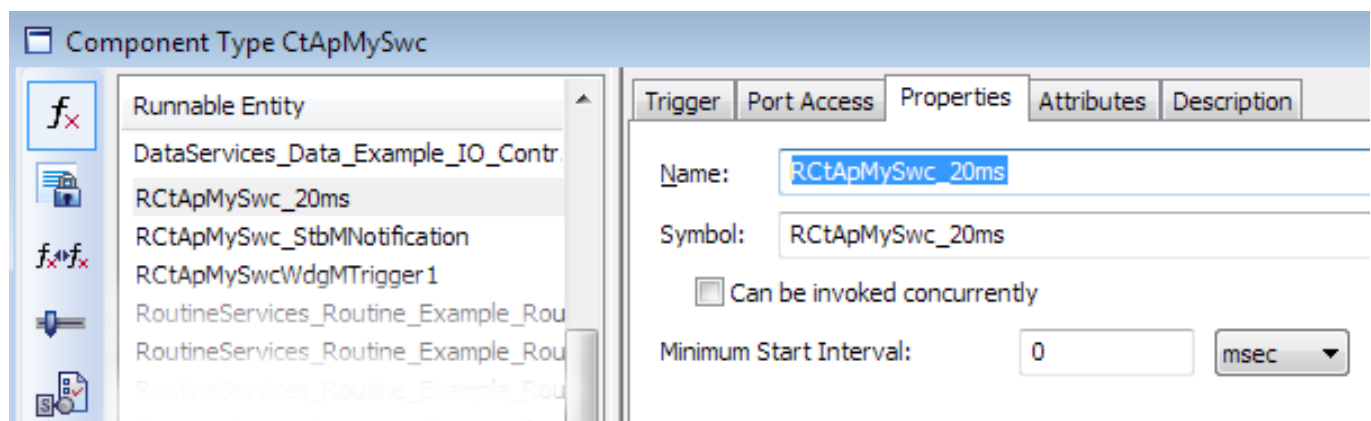
Data Mapping in **DaVinci Developer** has a higher priority than Data Mapping in the **DaVinci Configurator Pro**.

Data Mappings done via **DaVinci Developer** cannot be overwritten by the ones done via **DaVinci Configurator Pro**.

7.2 Task Mapping

Tasks are means of the operating system. You can define as many tasks as you like and need. You define the name of the tasks, its priority and its type like **Auto**, **Basic** or **Extended**. You can also define the task as non- or full preemptive.

With Task Mapping, you have to map all runnables to tasks, except **On operation invocation** triggered runnables. Those are normally called only from one task and have no problem with reentrance.



A runnable has to be mapped to a task if it is not re-entrant but could be called re-entrantly during system operation.

What happens if you do not map a runnable that should be mapped?

The Vector AUTOSAR Solution provides an intelligent RTE generator. The RTE generator checks necessary conditions and will warn you if there are unmapped runnables that have to be mapped.

What happens if you map a runnable that does not have to be mapped?

Nothing will happen. It will still work. But the code efficiency and RAM consumption could be less good.

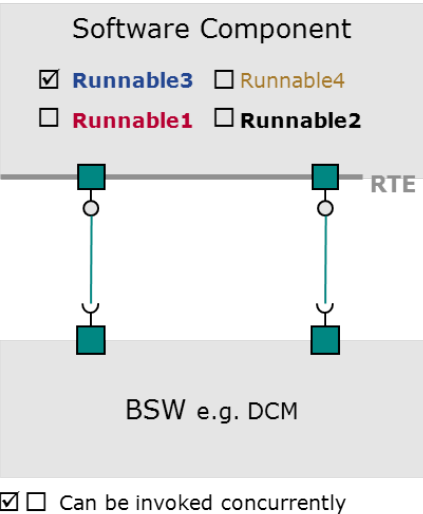
Is there a disadvantage of the RTE generator automatism?

It could be less comfortable to figure out the call context of the runnables and the stack consumption could increase.

7.2.1 Information about Interaction between Runnable, Re-entrance and Task Mapping

Your goal is a configuration of the system that results in a highly runtime-optimized and memory consumption-optimized code. Here is a little information to estimate what the RTE generator does.

Software Design



Task Mapping

Task A	Task B
Runnable1	Runnable2
Runnable5	Dcm_MainFunction
Runnable9	Runnable11
	Dem_MainFunction

In context of Task A

↓

RTE

```
tmp = A
SetEvent R1
...
```

TASK

```
{
  tmp = A
  waitEvent
  if Event = R1
    Runnable1(A)
  ...
}
```

In context of Task B

↓

```
Rte_Call Runnable3
Rte_Call Runnable4
```

This illustration shows a software component with four runnables. Only runnable 3 is set to **Can be invoked concurrently**. On the right-hand side you see which runnable is mapped to which task and the result of this mapping for the generated code. Here is the details.

Can be invoked concurrently is set and the runnable is not mapped to a task

Example: Runnable 3

Runnable 3 is called directly in the context of the calling BSW (DCM in this example – DCM_MainFunction, TASK B).

Caution

Make sure that your runnable is really re-entrant (see section **Can Be Invoked Concurrently**). If not, errors can occur that are difficult to debug.

Can be invoked concurrently is NOT set and the runnable is not mapped to a task



Example Runnable 4

If the RTE generator finds out that the runnable is not called multiple times in parallel, Runnable 4 is called directly in the context of the caller BSW (DCM). Otherwise an error message will be shown.

Can be invoked concurrently is NOT set and the runnable is mapped to a task different from the task where the main function of the caller (e.g. Dcm_MainFunction) is mapped.



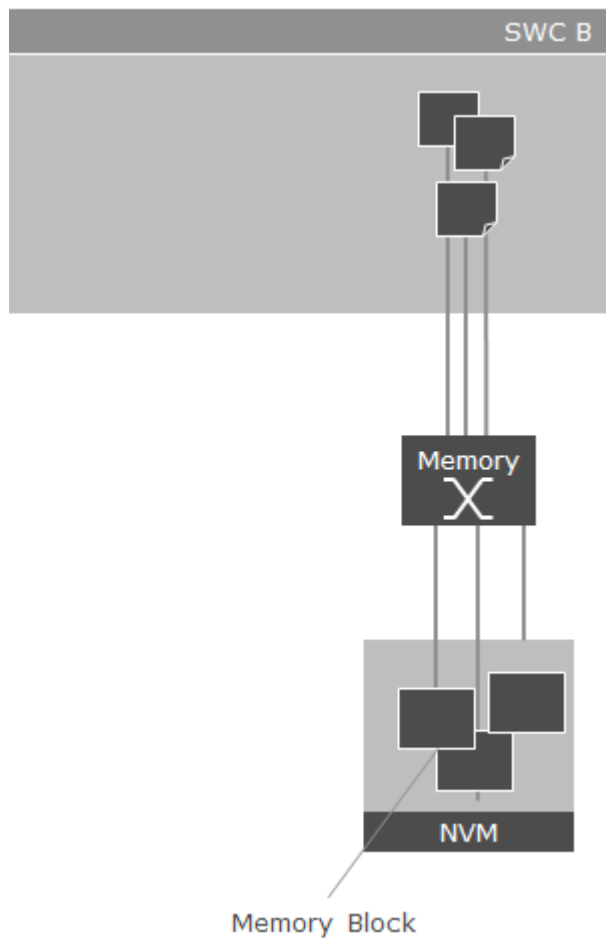
Example: Runnable 1

When the caller BSW now wants to activate the runnable via the RTE call, an event is set and the temporary variables of the runnables are stored to RAM (context of calling BSW, e.g. Task B). When the point in time has come, the WaitEvent in Task A is triggered by the event and starts the Runnable1(A) and hands over the parameters previously stored to RAM.

As you see, runtime is longer than using direct call and the RAM consumption is higher. E.g. for diagnostics: DCM runnables always have array types that are completely stored to RAM. For e.g. 100 DIDs à 4 bytes you need 400 bytes!

7.3 Memory Mapping

Within ECUs there is data that has to be persistent during power-off. Therefore this data cannot be stored simply to RAM, it must be stored to EEPROM or Flash, i.e. to non-volatile memory. As an example for this data, think of the DEM information.



There are two possible ways how this non-volatile data will be used. Either your application always accesses a RAM copy of the NV data where the data is copied at start-up. Or data can be read/written directly from/to NV memory on request.

The Memory Mapping assigns your defined PIM, per instance memory to the memory blocks of the NVM.

A per-instance memory object is mappable if it is referenced by a service need object. The definition of per-instance memory objects or service needs is part of the component type definition and is not yet editable by **DaVinci Configurator Pro**.

7.4 Service Mapping

The base for the service mapping is the theory about clients and servers. The client uses a service of the server. The server itself provides the operation to the client. The client server communication between your application software component and a service component is done via service ports – client ports and server ports. A server port provides services (one or more operations) and a client port uses these services.

A service component can have server ports and client ports.

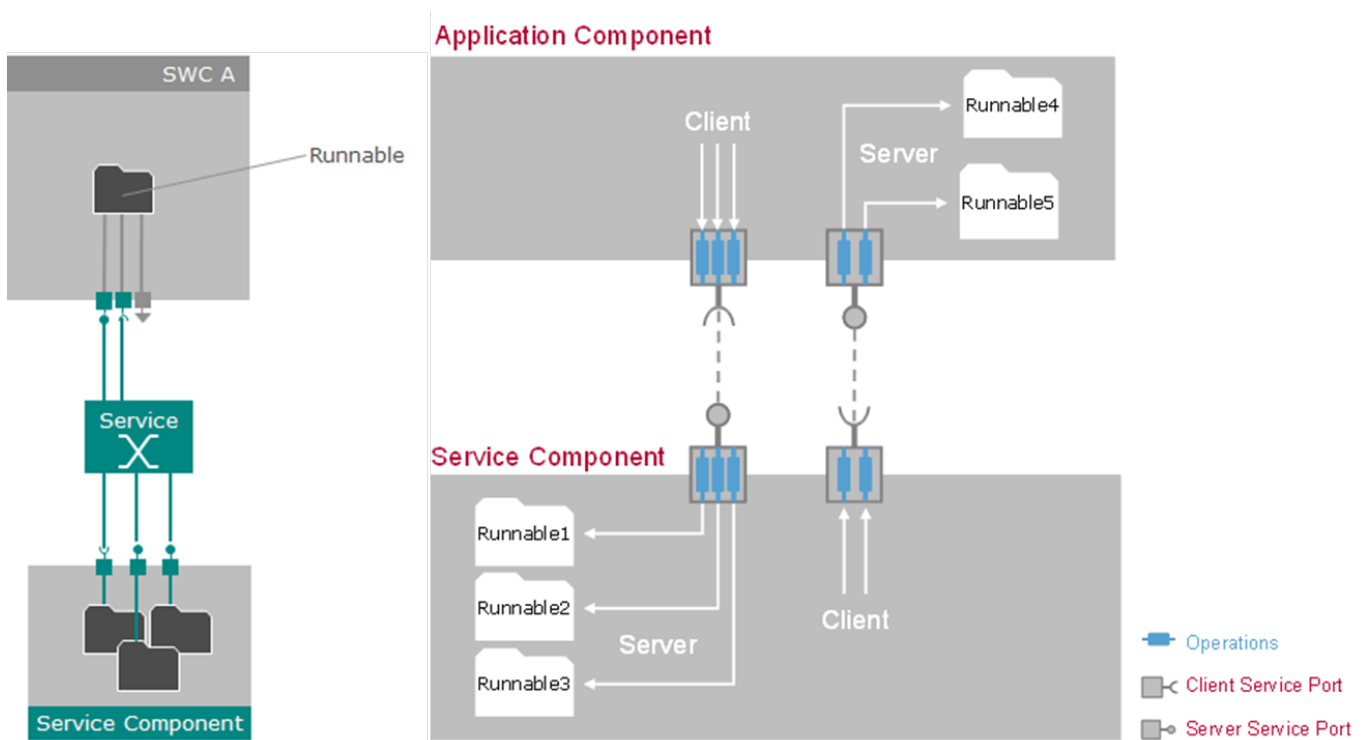
> **Service port of the service component is a server port**

The service is provided by the service component and your application software component just uses the operations (services, functions)

> **Service port of the service component is a client port**

Your application software component must be server and has to provide the operation (service, function). In this case, you have to program code, i.e. a runnable. This runnable must be created by you and it is triggered by the request of the service.

Besides the sender and receiver ports, there are also client and server ports. Sender and receiver ports carry data elements. Via client ports you can access so-called operations (or functions) of the servers.



Within one service port there could be n operations. Every single operation has to be assigned to a runnable.

The servers provide the runnables that contain the code. Here Runnable1, Runnable 2, Runnable 3 of the service component and Runnable 4 and Runnable 5 of application component.

When performing the service mapping, it is almost the same as the data mapping – but now you deal with services instead. To be able to access the services of a service component, you have to add this service component to your software component.

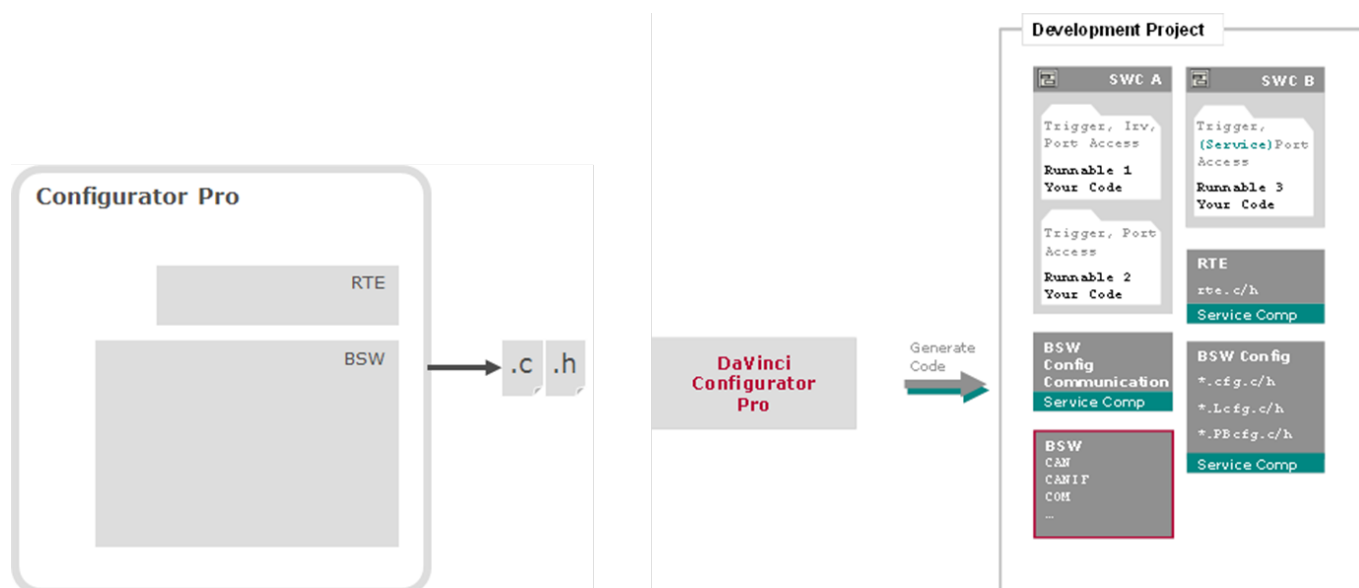
Then you can use all services that are provided by the service component. In this case your application is the client and the service component is the server.

But using a service component not only brings benefit – it also costs. Almost every service component does not only provide services – there can also be client ports on the service component side. And if you add this service component to your software component, you have to provide all the services where the software component provides a client port interface.

8 Generation

With the generation step **DaVinci Configurator Pro** generates the code for your project and also concerns your settings in STEP2 Define Project Settings on page 26. **DaVinci Configurator Pro** clings to your settings and works off the “to be generated” list.

The **DaVinci Configurator Pro** provides an automation interface for remote-controlled validation and code generation.



8.1 MICROSAR Rte Gen

The RTE is generated by the **MICROSAR Rte Gen**. Its generation depends on many settings in the configuration tools and how runnables are mapped to tasks. The files start with **Rte_** or **SchM_**.

9 Runnable Code

Until now you don't have written one line of C code. And that's typical for the AUTOSAR concept of software development. There is much more configuring work with tools than programming or implementing as you are normally used to from previous ECU software projects with e.g. CANbedded.



Example

This is an example showing the runnable called MySWC_Code with the function name (Symbol) MySWC_Code

```

/*****
 *
 * Runnable Entity Name: SwitchFrontLight
 *
 *****/
-----
 *
 * Executed if at least one of the following trigger conditions occurred:
 * - triggered on DataReceivedEvent for DataElementPrototype <OnOff> of PortPrototype <FrontLightState>
 *
 *****/
 *
 * Input Interfaces:
 * =====
 * Explicit S/R API:
 * -----
 * Std_ReturnType Rte_Read_FrontLightState_OnOff(Boolean *data)
 *
 * Client/Server Interfaces:
 * =====
 * Server Invocation:
 * -----
 * Std_ReturnType Rte_Call_FrontInteriorLight_DEFECT_OP_GET(IoHwAb_BoolType *signal)
 * Synchronous Server Invocation. Timeout: None
 * Returned Application Errors: RTE_E_env_FrontInteriorLight_DEFECT_E_NOT_OK
 *
 * Service Calls:
 * =====
 * Service Invocation:
 * -----
 * Std_ReturnType Rte_Call_Event_DTC_0x000002_SetEventStatus(Dem_EventStatusType EventStatus)
 * Synchronous Service Invocation. Timeout: None
 * Returned Application Errors: RTE_E_DiagnosticMonitor_DEM_E_QUEUE_OVERFLOW, RTE_E_DiagnosticMonitor_E_NOT_OK
 *
 *****/

FUNC(void, RTE_SA_FRONTINTERIORLIGHT_APPL_CODE) SwitchFrontLight(void)
{
/*****
 * DO NOT CHANGE THIS COMMENT!          << Start of runnable implementation >>          DO NOT CHANGE THIS COMMENT!
 * Symbol: SwitchFrontLight
 *****/

IoHwAb_BoolType lightDefect;

Rte_Call_FrontInteriorLight_DEFECT_OP_GET(&lightDefect);

if (lightDefect)
{
/*Light is detected to be defect. So any light information from MySWC_Code is turned to light off.*/
Rte_Call_Event_DTC_0x000002_SetEventStatus(DEM_EVENT_STATUS_FAILED); /*set event*/
}
else
{
/*Light is working well. No special treatment necessary*/
Rte_Call_Event_DTC_0x000002_SetEventStatus(DEM_EVENT_STATUS_PASSED); /*set event*/
}
}

```

This is the header of a runnable showing all necessary information about the runnable like:

- > When it is triggered
- > Input and output interfaces
- > Service interfaces

**Note**

If some access is missing, go back to the **DaVinci Developer**, add the necessary port access for your runnable, go back to **DaVinci Configurator Pro**, synchronize the system description and generate the component templates again.

Then the missing interface should be there and can be used.

10 Compile And Link

10.1 Using your "real" hardware

This step is highly dependent on your compiler / linker / project settings. Compile and link everything together and create a file that can be downloaded to your hardware.





III Additional Information

This section includes additional information dealing with special topics like e.g. multiple user concept or support request package.

- > [Update Input File](#)
- > [Update Project Settings](#)
- > [Support Request via DaVinci Configurator](#)
- > [Multiple User Concept](#)
- > [Update Configuration](#)
- > [Update DaVinci Tools](#)
- > [Non-Volatile Memory Block](#)
- > [Basic Software Modules](#)
- > [Variant Handling](#)
- > [Add Module Stubs from AUTOSAR definition](#)
- > [Project Migration](#)



1 Update Input Files

Open Input Files editor via **Project | Input Files** or **Input Files icon**  at **DaVinci Configurator Pro** toolbar.



Note

Any change of the input files requires an update of the configuration. Update configuration updates all input files, whether they are changed or not.

1.1 System Description Files

If necessary, replace your File at Input Files | System Description File.

Select the Input File, click **Replace button**  and choose the new **Input File**.



Note

Any change of the input files requires an update of the configuration. Update the configuration via **Update Configuration icon** .

1.2 Diagnostic Data Files

If necessary, update your Diagnostic Description file and select ECU and Variant.

In case of **ODX 2.0.1** as Diagnostic Description file, it is necessary to update the state description. Therefore click **[Synchronize State Description...]**.



Note

Any change of the input files requires an update of the configuration. Update the configuration via **Update Configuration icon** .



2 Update Project Settings

Open Project Settings Editor via **Project** | **Settings** or **Project Setting icon**  at **DaVinci Configurator Pro** toolbar. Select **Project Settings** to open the view.



3 Support Request Via DaVinci Configurator Pro

If you request support for Vector products, you have to provide project and environment information. Therefore, use the **Support Request Package** function of the **DaVinci Configurator Pro**.

The **Support Request Package** compiles the necessary project and environment information in a ZIP-file. Send this file to the Vector Support via E-mail. Just follow the description below.

**Note**

The **DaVinci Configurator Pro** does not send any data automatically!

1. Open the Support Request Editor via **Help | Create Support Request Package...**

**Note**

The entry **Create Support Request Package...** is only enabled if a project is loaded.

2. Add your First Name, Last Name and E-Mail address.
3. Click **[Next >]**
4. Select project information which has to be added to the support request ZIP-file.

**Note**

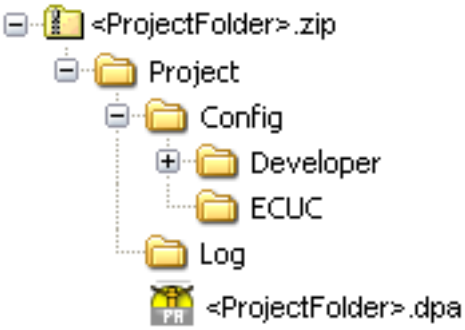
PC Info, Tool Version Info and SIP Info are default and can not be deselected.

5. Click **[Next >]**
6. Enter the path where the created ZIP-file has to be stored.
7. Click **[Finish]**

3.1 Result

In addition to the **Zip-File**  <Target Path>.zip, the Project Assistant creates the following folders and files.

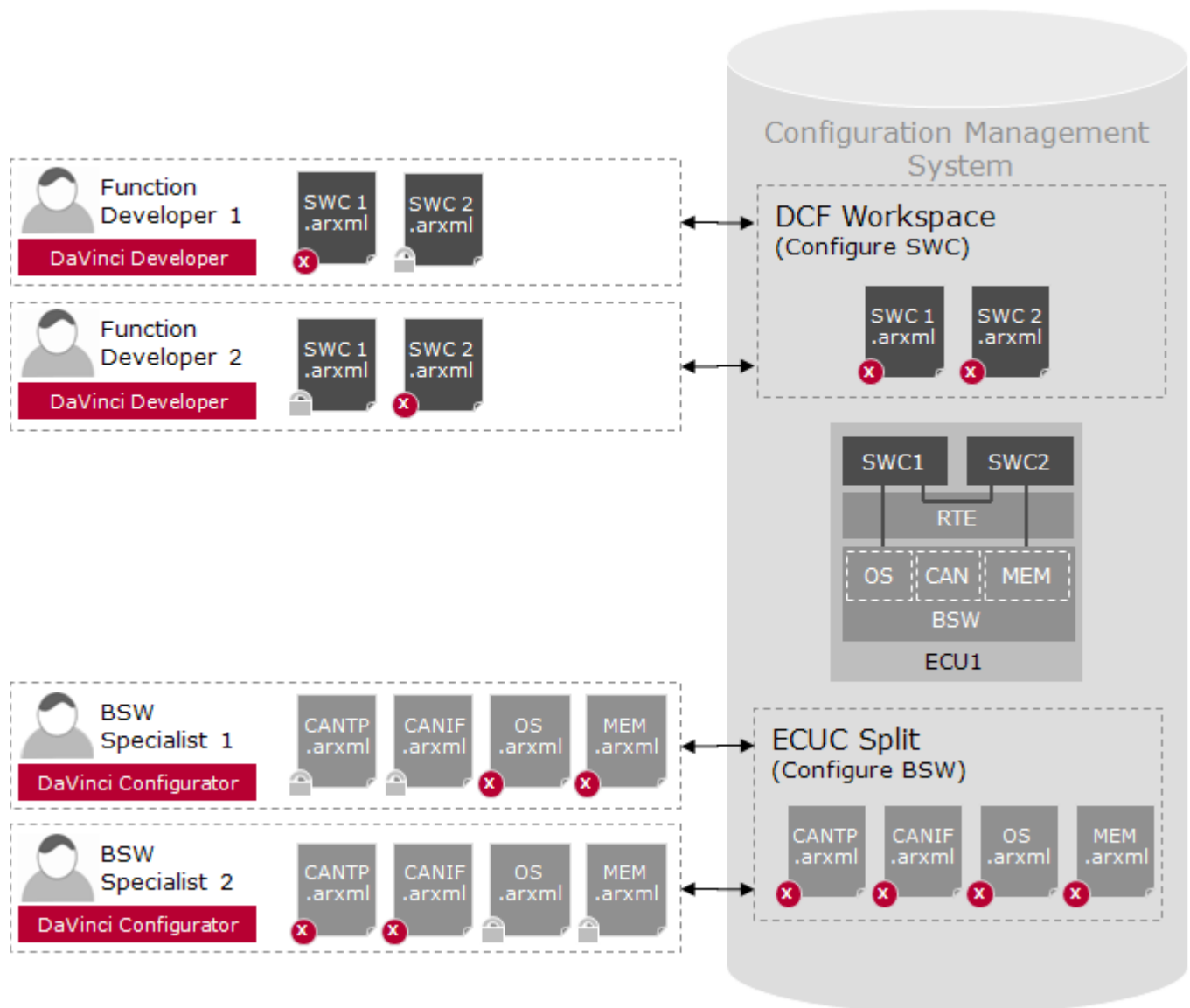
**Support Request via DaVinci Configurator**

Folder/File	Description
	The Project folder includes all defined project files
 SupportRequest.html	The file includes all necessary project and system information (e.g. version of tools)
 SupportRequest.xml	The file includes all necessary project and system information (e.g. version of tools)

Send **<Target Path>.zip** via e-mail to the Vector Support Address embeddedSupport@de.vector.com.

4 Multiple User Concept

This concept helps you to realize the configuration management of the design and configuration data on a fine-grained level in combination with a configuration management (CM) system. It is available for software components as well as for BSW configurations. Split files also help you in multi-user projects: you can control the read/write access on fine-grained level. This helps you to prevent the users from making unwanted changes and therefore reduce the necessary diffs and merges.



4.1 General

The decision whether to use a single configuration file or split configuration files is done in the Project Assistant settings. The resulting split files are ideally managed through a central CM system. Each user needs the full set of split files to work with the project.

The user has to obtain a writeable working copy to his local file system.



The user has to obtain a read-only working copy to his local file system.



Caution!

After a SIP update, all configuration files may have to be updated. This requires all files to be writeable while loading the configuration with a new SIP for the first time.

It is necessary to lock all files which will be edited during the configuration session. The DaVinci tools load all working copy files. The locked files are writeable; all other files are write-protected.



Note

If you prevent parallel editing e.g. by according settings in the CM system you can ensure that only one user can obtain a writeable copy at a time. If you allow parallel editing in the CM system, several users can get a writeable copy. When both users make changes, you can use the Project Merge function to merge the modifications.

After finishing configuration, the locked files have to be checked in (committed) to the CM system.

4.2 Split Files for Software Component and ECU Project Configuration

The split of the **DaVinci Developer workspace** is necessary to enable working in parallel for software component configuration.



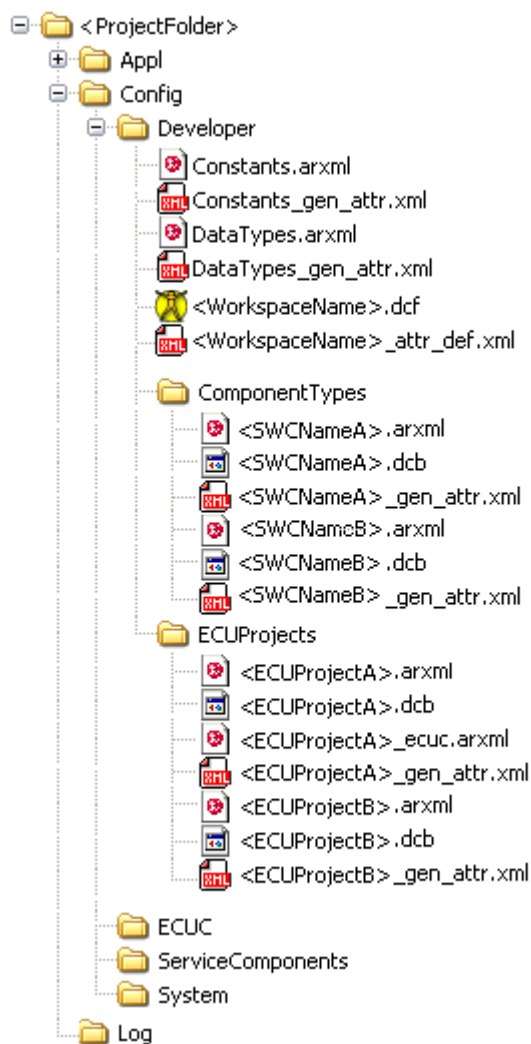
Note

Workspace splitting is only supported by the DCF workspace format. You chose this option during the project setup and you cannot change it afterwards.



Result

Additionally to the workflow file the Project Assistant creates multiple files for each Software Component Type and ECU Project in the Developer folder.



File	Description
<WorkspaceName>.dcf	This file includes references to all single software component and ECU Project files.
<SWCNameA>.arxml	General software component file
<SWCNameA>.dcb	This file includes the binary part of the software component.
<SWCNameA>_gen_attr.xml	This file includes the generic attributes of the software component.
<ECUProjectA>.arxml	General ECU project file
<ECUProjectA>.dcb	This file includes the binary part of the ECU Project.
<ECUProjectA>_ecuc.arxml	Project-specific ECUC file
<ECUProjectA>_gen_attr.xml	This file includes the generic attributes of the ECU Project.



4.3 Configure Software Component Prototype

To configure a software component prototype it is necessary to lock all files, specific for this software component. (**<SWCNameA>.arxml**, **<SWCNameA>.dcb** and **<SWCNameA>_gen_attr.xml**).

4.4 Configure ECU project

To configure an ECU project it is necessary to lock all files, specific for this ECU project. (**<ECUProjectA>.arxml**, **< ECUProjectA >.dcb**, **<ECUProjectA>_ecuc.arxml** and **<ECUProjectA>_gen_attr.xml**).



4.5 Split Files in DaVinci Developer



Note

Parameters of write-protected files are greyed out in the **DaVinci Developer**.

If Parameters of write-protected modules are changed because of module dependencies, the **DaVinci Developer** informs about the write-protection during the synchronization process.

You have the possibility to make the file writeable and repeat the synchronization process.

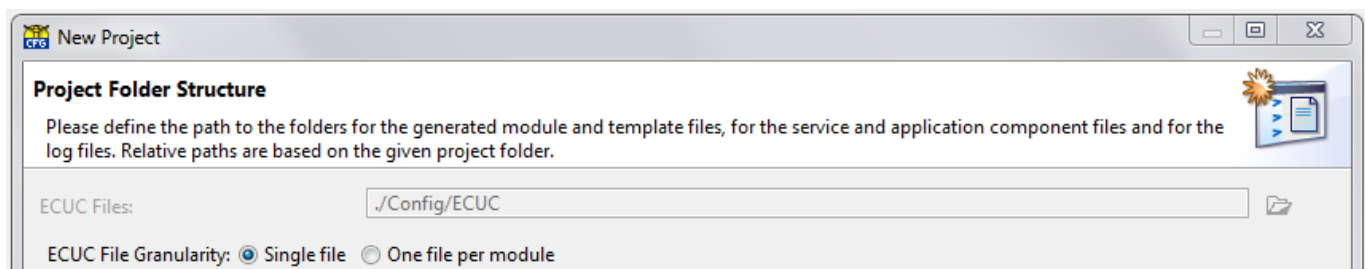
4.6 Split Files for BSW Configuration

The split of the ECUC is necessary to enable working in parallel for BSW configuration (one file per module).



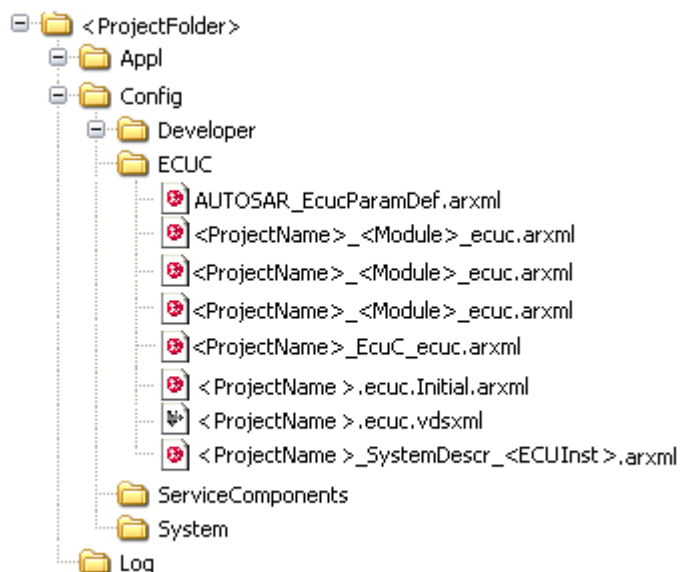
Note

Please note that further .arxml files are created when activating further modules.



Result

Additionally to the standard ECUC files and the AUTOSAR parameter definition file, the Project Assistant creates multiple ECUC files **<ProjectName>_<Module>_ecuc.arxml** to the ECUC folder.



Note

Additionally to the module ECUC files the Project Assistant creates a **<ProjectName>_EcuC.ecuc.arxml** file, containing references to all module ECUC files. This file is used as project file for the **DaVinci Developer** and the **DaVinci Configurator Pro**.



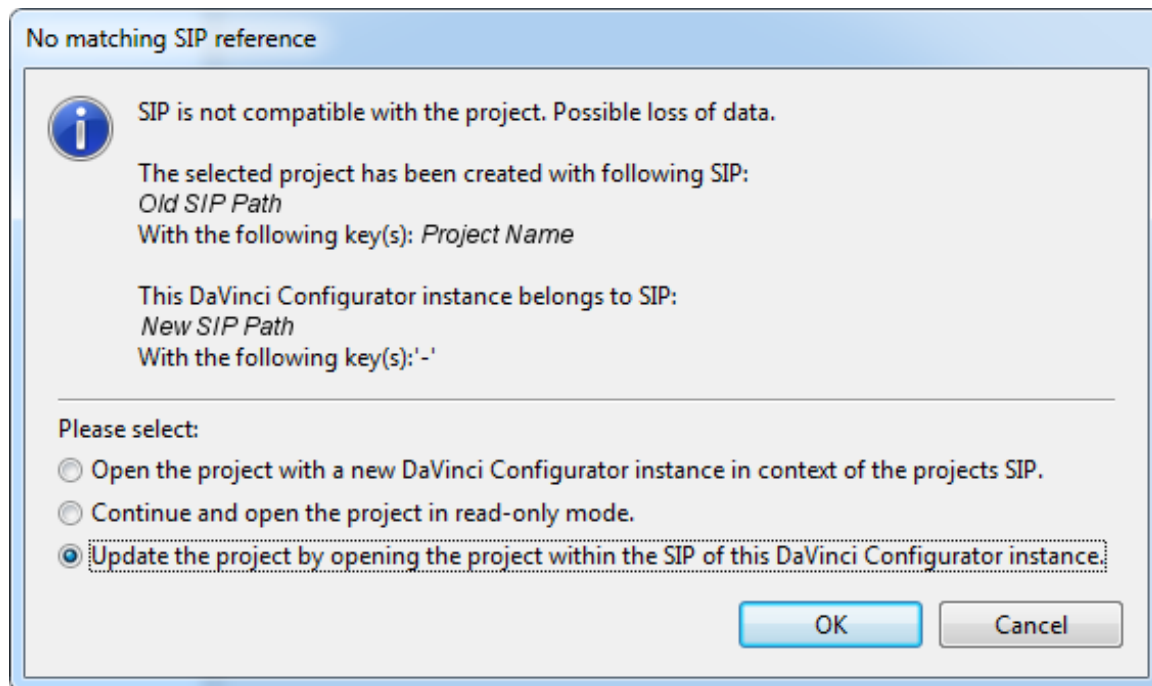
5 Configuration Update

If you get a new delivery with Vector BSW Release x, it is necessary to update some parameter values of your configuration to the new release. There are some steps, that are necessary for most of the updates and there are special steps, that are only necessary for a certain update.

First there are described the necessary steps for any update followed by the special update steps dependent on the release version.

5.1 Release x-1 to Release x (necessary steps for any update)

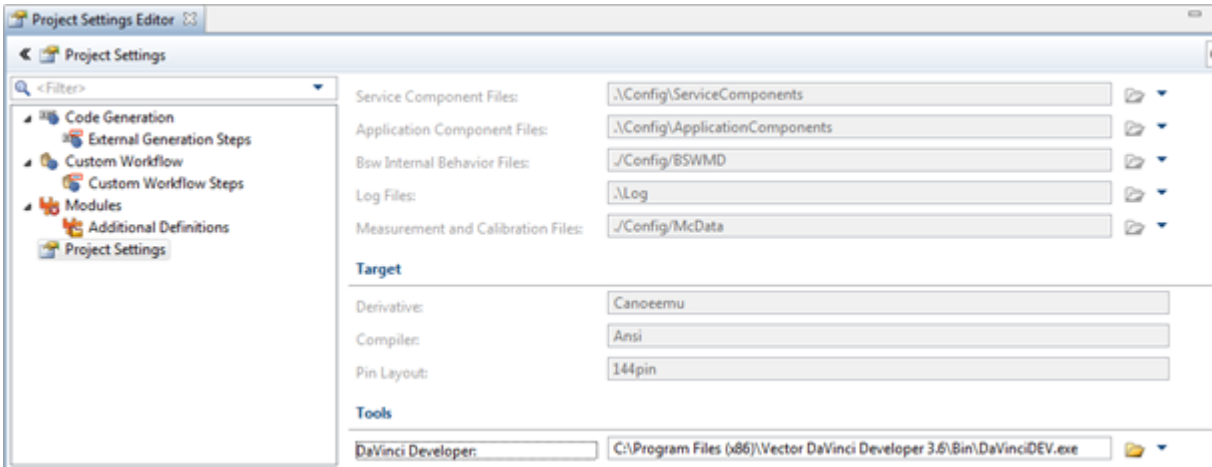
1. Install the latest **DaVinci Developer**
2. Open your configuration with the **DaVinci Configurator** of the new SIP .\DaVinciConfigurator\Core\DaVinciCFG.exe and select the following option:



Note

The path within the *.DPA file will be updated to the new SIP.

3. Change the path to the new **DaVinci Developer** if it was installed to a different location than the former **DaVinci Developer** version.



Note

C:\Program Files (x86)\Vector DaVinci Developer <version>\Bin\DaVinciDEV.exe

4. Handle remaining errors and warning. The following might occur for any update

Errors Invalide/Incorrect Definitions	
> AR-ECUC02008 Invalid multiplicity > AR-ECUC03019 Incorrect definition of configuration element	
Solution	Delete parameter in ECUC file not existing in BSWMD file any more.
Example	Some parameters has been replaced. E.g. /MICROSAR/Dem/DemGeneral/DemEventStorageTrigger has been replaced by /MICROSAR/Dem/DemGeneral/DemEventMemoryEntryStorageTrigger

Errors Inconsistent Connector Prototypes	
> RTE51027 Connector prototype inconsistent. (1 message) > RTE51031 Connector prototype inconsistent. (1 message)	
Solution	Due to changes within the Service Components the ports can be become incompatible Application Software Components. Adapt your Application Software Components with the DaVinci Developer to match the new port definitions. The changes are normally resulting from > Correction (the old definition was wrong) > Changed implementation based on AUTOSAR Bugzilla entries / RfC > Changed implementation due to support of newer AUTOSAR Version (e.g. AUTOSAR 4.0.3 to AUTOSAR 4.1.2)



5. Process all further Errors and Warnings given within the Validation View of the **DaVinci Configurator**
6. Check for updating of the input data base files.
Even though they might not have changed, the deriving of parameters might have improved. Thus an update now would reduce the changes when receiving changed input data base files.



Cross Reference

Additional information about necessary configuration update steps are described within the [Release Notes](#) (**Category: Change**).

The following describes what to do when updating especially from one release to another.

5.2 Release 8 to Release 9

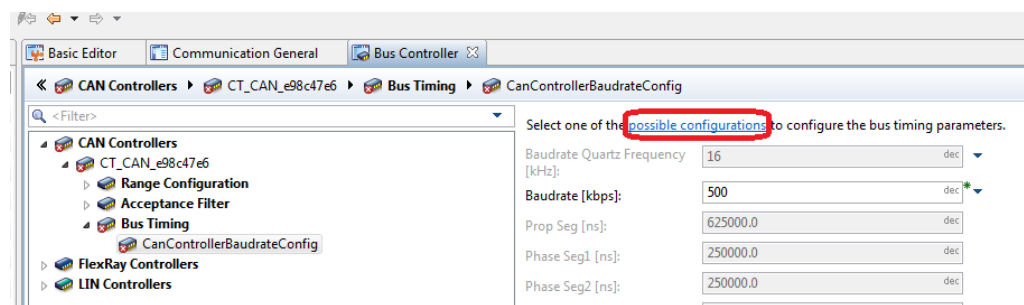
Errors Invalid Baud Rate

- > CAN02012 An invalid baud rate value is configured
- > CAN02012 invalid baudrate settings:
[MANUAL ACTION]: change clock or baudrate.
/ActiveEcuC/Can/CanConfigSet/CT_CAN_e98c47e6/CanControllerBaudrateConfig

Solution

The tool now checks the values of the baud rate configuration, they might have been wrong before.

Check valid settings of McuClockReferencePointFrequency referenced by **/MICROSAR/Can_Cano-eemuCanoe/Can/CanConfigSet/CanController/CanCpuClockRef**
Then select one of the offered baud rate configurations by clicking on 'possible configurations'.



5.3 Release 7 to Release 8

If you get a new delivery with Vector BSW Release 8, it is necessary to update some parameter values of your configuration to the new release.

After you have updated your project with the new SIP the following steps are necessary:



1. Select ECUC parameter which is not defined by a BSWMD Parameter definition.
(Error number AR-ECUC03019)
2. Check whether additional parameters exist.
(Parameters without value definition or with default value)
3. Define value of the new parameter with value from the existing parameter
4. Delete existing parameter

Use the same steps for container updates.



Note

Select all **VectorCommonData** containers, which are marked with an error, use multi-selection in the DaVinci Configurator Basic Editor and delete them via **Delete Container**.

All **Use RTE** switches and **Production Error Detection** switches can be deleted without check.



6 Update DaVinci Tools



Cross Reference

The DaVinci service packs are located at the **Vector Download Center**.
https://vector.com/vi_downloadcenter_en.html

The Update of the DaVinci Tools is described within the installation guide.



Cross Reference

You will find the installation guide document at the Vector Knowledge Base:
<https://vector.com/kbp/entry/640/>

6.1 DaVinci Configurator Pro

If necessary , update **DaVinci Configurator Pro** service packs, therefore you have to download them from the Vector Download Center and install them via DaVinci Configuration Service Pack Installer. Select the installed SIPs which should be updated.

6.2 DaVinci Developer

Service Packs of **DaVinci Developer** are also available in the Vector Download Center. The service packs include an installer to update your **DaVinci Developer** installation.



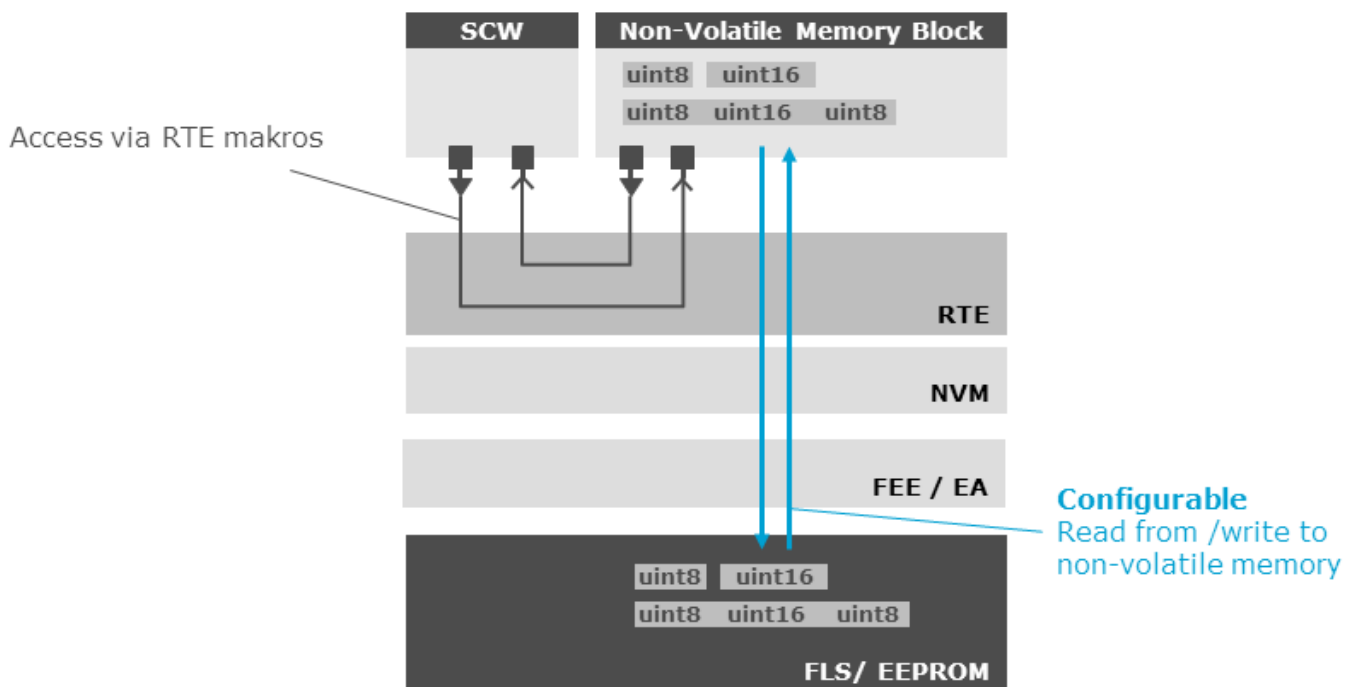
Note

Please note that compatibility is only granted for service packs provided for the same major and minor version.

7 Non-Volatile Memory Block

This is a description how to handle data that has to be stored to non-volatile memory like FLASH or EEPROM.

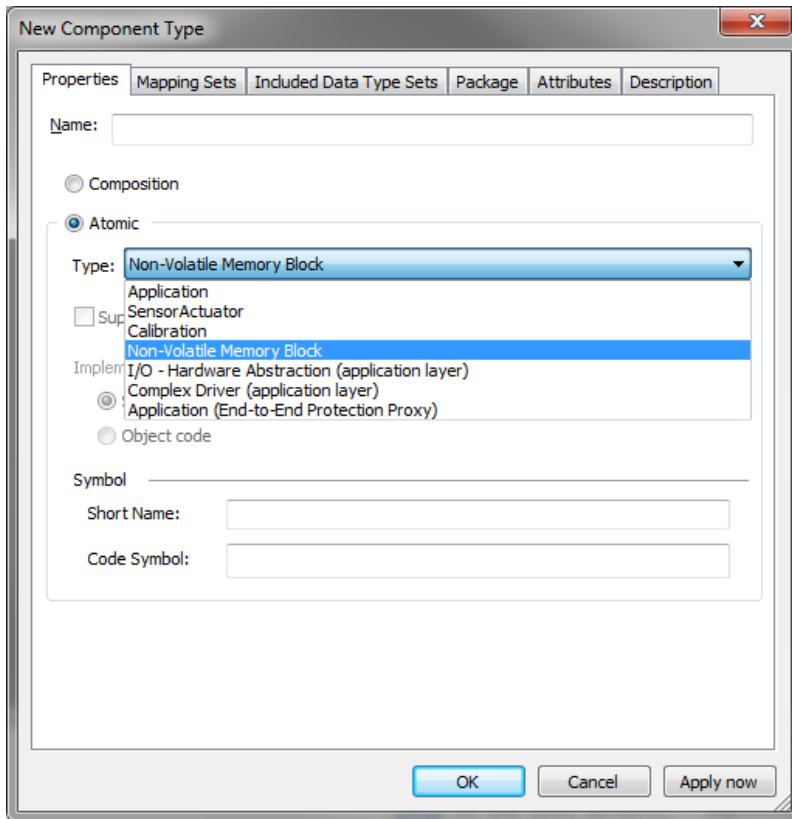
The illustration below shows briefly the concept behind. Data is defined in a Non-Volatile Memory Block. The application SWC has access to this data via the RTE. The data is stored and addressed in RAM. At configuration time you can define which data in which format you need and when and how this data is written to and read from the non-volatile memory. You can also trigger the immediate writing of this data to the FLS or EEPROM.



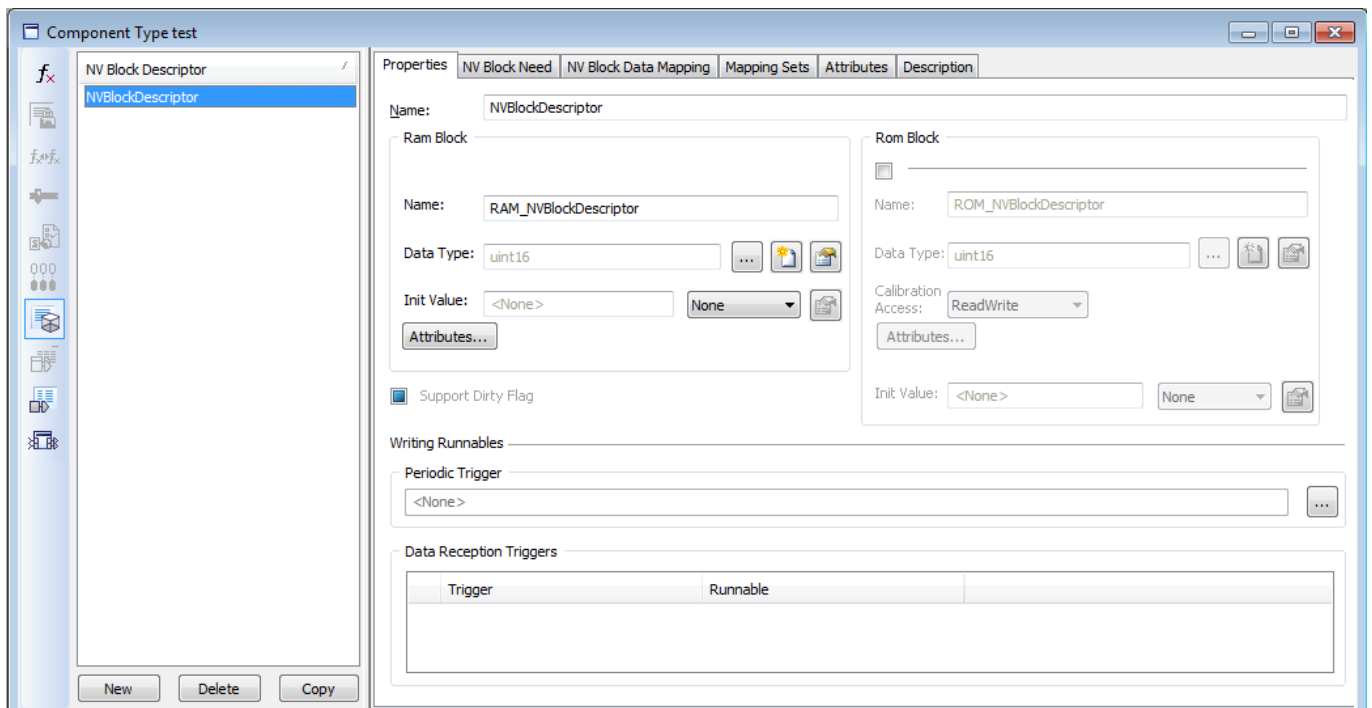
7.1 Configure and use Non-Volatile Memory Block

The following description shows, how the Non-Volatile Memory Block is configured and used.

In the Library of DaVinci Developer, add a **New Application Component Type**, select **Non-Volatile Memory Block** as type and give a **Name** to this new component type. Double-click the new **Non-Volatile Memory Block** in the **Library** and select **NV Block Descriptors** 



Use **[New]** to define a new NV Block Descriptor. On the Properties tab define the data you want to store to non-volatile memory and its type. This can be a single variable like integer or boolean. Also complex data types like records or arrays can be defined.

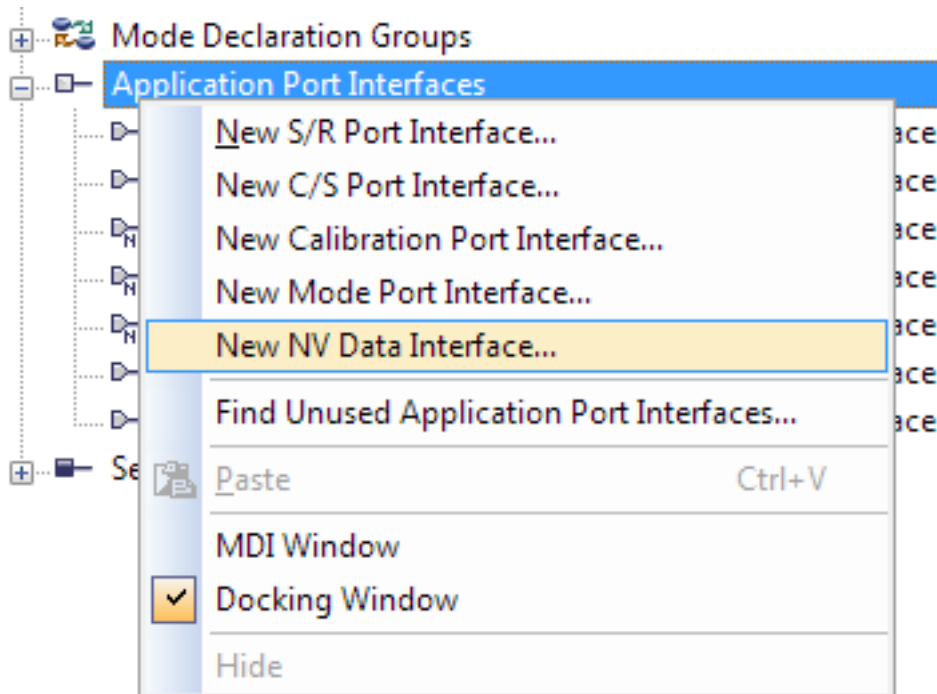




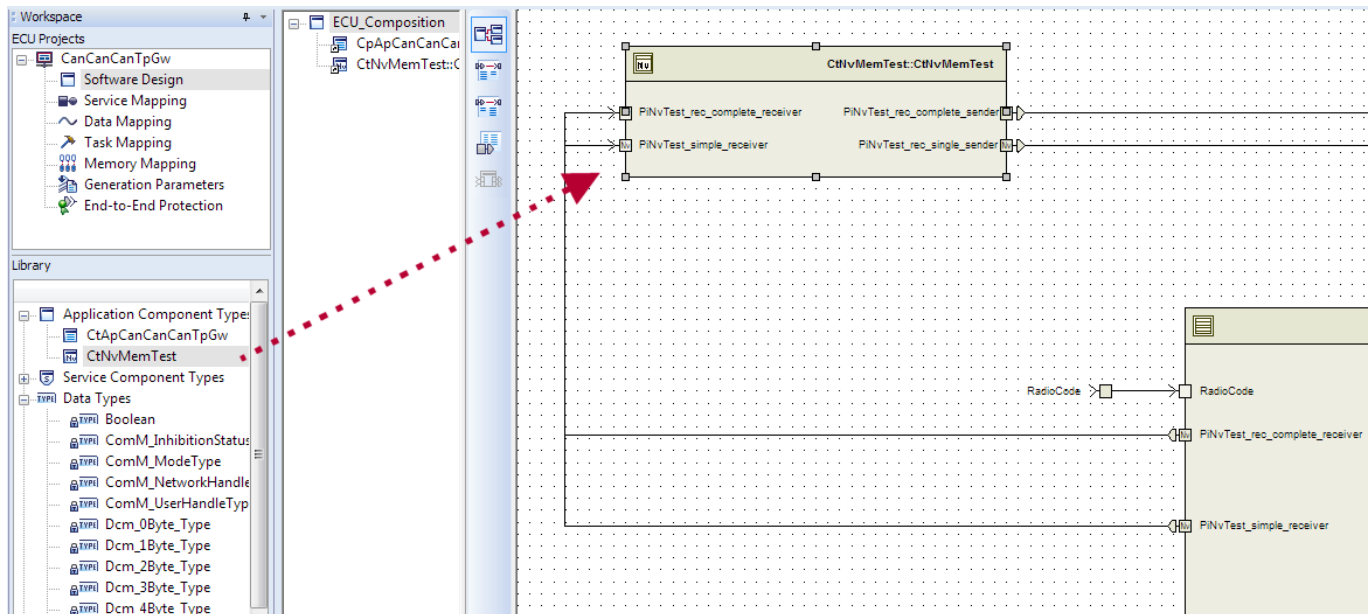
Select the tab NV Block Need and define, when and how your data should be finally written to NV memory (FLS or EEPROM).

Number of Data	0	☐ ReadOnly
Reliability:	Low	☐ Resistant to Changes
Writing Priority:	Low	☐ Restore at Startup
Writing Frequency [write access per year]:	0	☐ Store at Shutdown
Block Size [byte]:	0	☐ Write only once
Ram Block Status Control:	API	

Now add New NV Data Interface... to define a port to access the data defined before. Select suitable Data Element Prototypes.



Add the NV component type to your Software Design (drag'n'drop from Library), assign the NV Data Interface to the NV component prototype and another NV Data Interface to your application SWC. Draw the necessary connectors by hand. Define the Init Values of all ports.



Double-click NV component prototype and perform the internal mapping on the NV Block Data Mapping tab (right-click to open context menu).

Properties	NV Block Need	NV Block Data Mapping	Attributes	Description
Ram Block Element		Read NV Data: Port		
RAM_NVBlockDescriptor_1.RtNvTest		PiNvTest_rec_complete_sender		

7.2 Port Access of your Runnables

What is left now – the access of your application runnable to the **NV Data Interface(s)**. Open your application software component, select a runnable and then the tab **Port Access**. Use the **[New]** button to add a read or write data access to the **Non-Volatile Memory Block**.

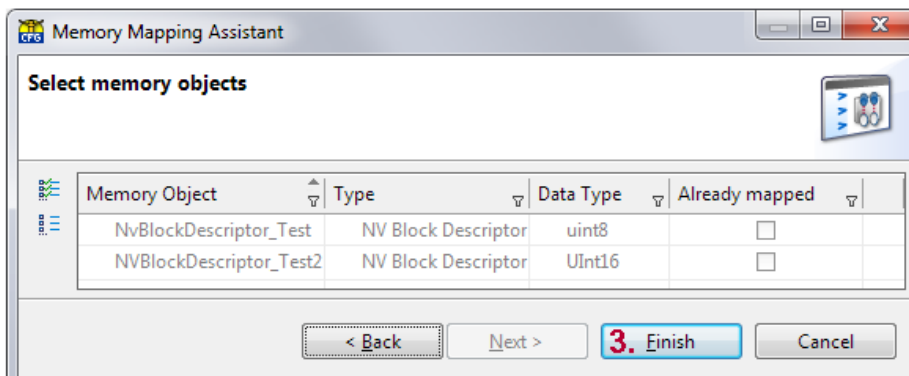
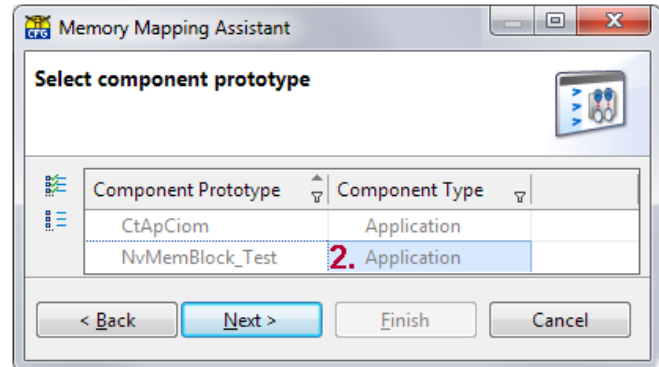
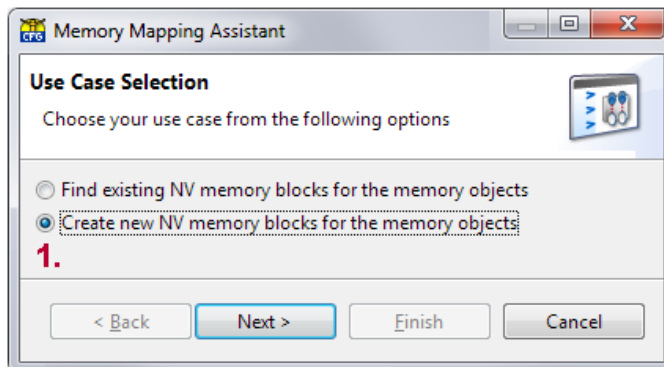
Save your project and switch to **DaVinci Configurator Pro**.

7.3 Memory Mapping in DaVinci Configurator Pro

Synchronize the system using the synchronization message in the **Validation** area.

Open **Runtime System**. Click **Add Memory Mapping**.

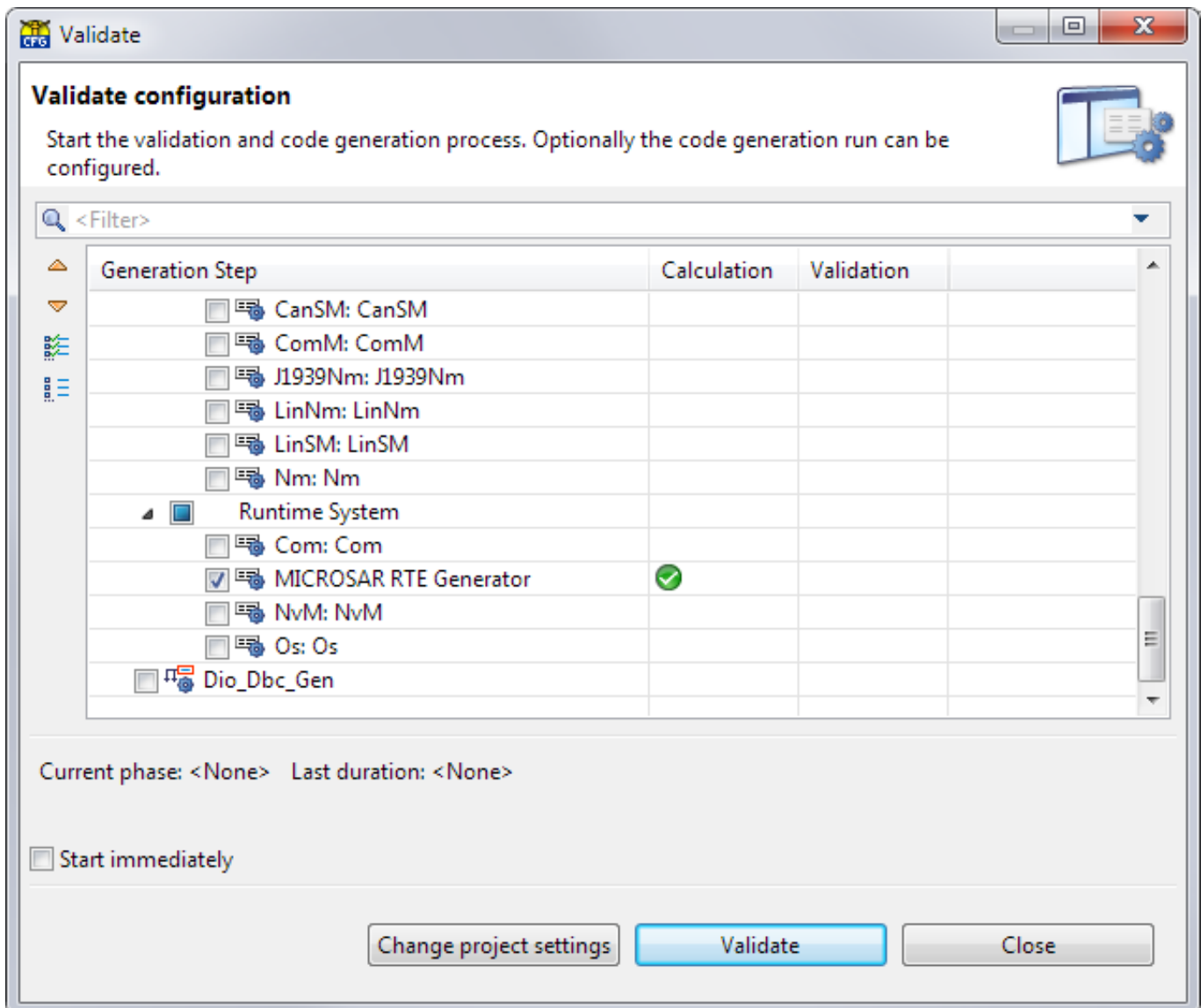
1. Select **Create new NV memory blocks for the memory objects**.
2. Select the block created before in the DaVinci Developer
3. Get information about unmapped blocks and click **[Finish]**.





7.4 Validate the RTE

1. Open the validation window via 
2. Deactivate all boxes via 
3. Select RTE and click **[Validate]**.



When implementing you runnable code, you can now access the defined variables by using RTE read and write macros.



8 Basic Software Modules

This chapter deals with BSW modules, Basic Software Modules. First they are introduced theoretically then configured with the **DaVinci Configurator Pro**.

8.1 Generic BSW Modules

Before you start to configure the BSW module with **DaVinci Configurator Pro** you have to know something about BSW modules.

This chapter introduces to you a generic BSW module and wants you to get a basic understanding of it:

- > How it looks like
- > Of what it is formed
- > How it is configured
- > How it works
- > What is fix
- > What is generated and

some special topics like

- > Memory sections and
- > Exclusive areas



Cross Reference

Find the detailed description of the BSW module in the folder **Doc|TechnicalReferences** of your delivery.

8.2 What is a BSW Module?

A Basis Software Module (BSW module) consists of:

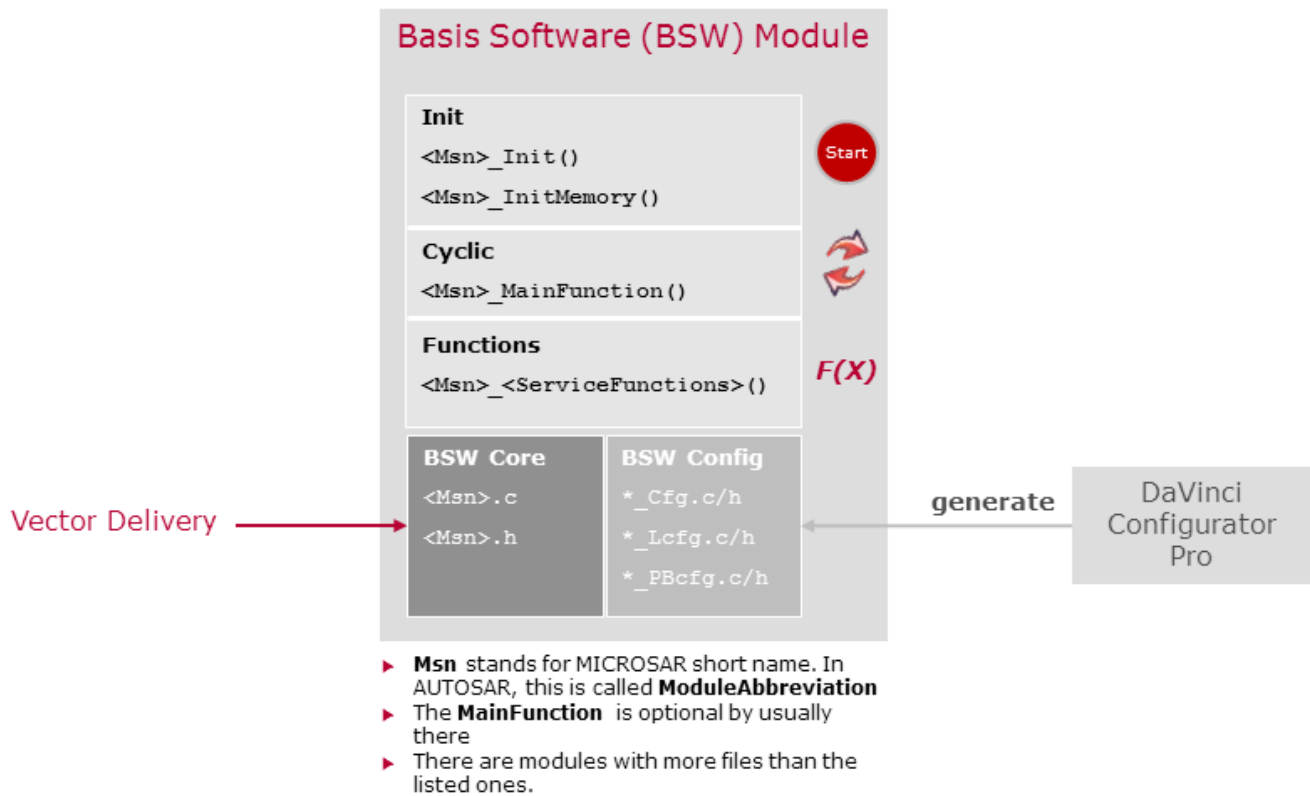
- > Static BSW files (algorithms and functions)
- > Generated files (data and sometime algorithms and functions)

To get it to work it has to be

- > Included
- > Initialized
- > Its main function has to be called cyclically (in most of the cases)

The mapping of its code and variables can be configured as well as their exclusive areas, to protect the module's internal data.

- > Memory Mapping
- > Exclusive Areas



Example

Examples for the short names: Dem, Dcm, EcuM, etc.

8.3 BSW Module Configuration

The amount and properties of configurable parameters is defined within the `<Msn>.bswmd` file. BSWMD stands for Basic Software Module Description file. This file is read by the BSW configuration tool (**DaVinci Configurator Pro**) and used to create the Basic Editor. The Basic Editor is the tool expression of all configurable parameters of the BSW module, displayed in a generic way.



Note

In addition to the Basic Editor, the Vector BSW configuration tool **DaVinci Configurator Pro** provides comfortable views that make configuration of BSW modules very comfortable.

The DaVinci Configurator Pro generates the configuration files for every used BSW module. These are files like



- > <Msn>_cfg.c/h,
- > <Msn>_Lcfg.c/h,
- > <Msn>_PBcfg.c/h

8.4 BSW Initialization

8.4.1 <Msn>_InitMemory()

Most BSW module needs variables that have to be initialized before the call of `<Msn>_Init()`. This can be global or static variables. According AUTSOAR concept, all necessary variables will be initialized by the start-up code. Where this is not done or not possible the function `<MSN>_InitMemory` has to be called as an alternative.

8.4.2 <Msn>_Init()

Every BSW module has to be initialized at start-up. This is done with the start-up of the ECUM. The function to initialize a BSW module is called: `<Msn>_Init()`



Note

Our modules and functions are protected against multiple initializations.

8.5 BSW Module Version (xxx_GetVersionInfo)

Each BSW module provides an API `<MSN>_GetVersionInfo`. This API returns the BSW published information

- > BSW version
- > Module ID
- > Vendor ID

The BSW versions are BCD-coded. The availability of this API can be individually configured for each module.

8.6 Cyclic Calls

8.6.1 <Msn>_MainFunction()

To run and to work, most of the BSW modules (except for those, that are triggered by interrupt), have to be called cyclically with a configured call cycle. The module derives its time base from those cyclic calls. This cyclically called function is called main function and its name is formed like: `<MSN>_MainFunction()`



Note

In most of the cases it is mapped to a cyclic task that is called cyclically triggered by an Alarm of the Operating System.



8.7 Service Functions



Cross Reference

For information about **Client Server Theory**, refer to Data Mapping (section Service Mapping).

8.8 Critical Sections - Exclusive Areas

BSW modules use so-called exclusive areas to protect their resources (module internal data) from concurrency. Each module defines its own exclusive areas, up to 5 can be defined per AUTOSAR definition.

Whether a critical section needs to be handled or not depends on the possibility of a concurrent access onto a given resource of the BSW. It therefore depends on e.g. OS or hardware configuration constraints. If a critical section needs to be handled, different measurements are possible, ranging from a global interrupt lock to the usage of OS semaphores.



Example

As an example, a definition of an exclusive area could look like:

```
SchM_Enter_Com_COM_EXCLUSIVE_AREA_2();  
  
com_TxModeHdlr_DelayTimeCnt[i]--; /* Protected object */  
  
SchM_Exit_Com_COM_EXCLUSIVE_AREA_2();
```

There are different locks available i.e. different interrupt sources could be locked. Global interrupt, peripheral interrupt like CAN or no lock at all.



Cross Reference

Refer to the technical reference document of the BSW module to get information how exclusive areas are defined and used.

8.8.1 Memory Section

Every generated code and variable of the <MSN> modules is enclosed by #defines. Via the template file `compiler_memMap.h` you can define the mapping of each single section of a <MSN> module.



Cross Reference

See more about the memory mapping via `memMap.h` in the technical references of each <MSN> module.

8.8.2 Switch <MSN> Modules Off

Most of the modules can be switched off via: `<Msn>_Shutdown`

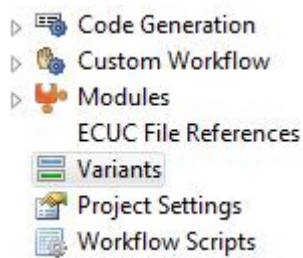


9 Variant Handling

9.1 Define Criterion and Variants

If you use variant handling within your project you have to define your variants first.

After you setup your project, open the **Project Settings Editor** via **settings icon**  and select **Variants**.



Select the **Edit Variance icon**  to add criterion, define criterion values and map criterion values to variants.

Edit Variance

Edit Criteria

Please edit the postbuild variance criteria.

1. **+** **Communication**

Name:

Communication Criterion: ☒

Diagnostic Criterion: ☒

2. **+**

Name	Value
Left	0
Right	1

< Back 3. **Next >** Finish Cancel

1. Add **[+]** new Criterion.

**Note**

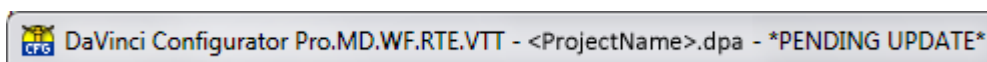
The DaVinci Configurator Pro is currently limited to a single criterion. This criterion can be used for **diagnostic** and **communication** variance.

2. Add **[+]** one criterion value for each variant.
3. Go on with **[Next >]**

[illegible]

4. Add **[+]** your Variants
5. Define Criteria Value for your Variants
6. Go on with **[Next>]** to see a summary of your definitions or **[Finish]** to come back to Variants view.

After finishing variant definition, the tool enters to **PENDING UPDATE** project state.



Note

The DaVinci Configurator Pro is currently limited to a single criterion. This criterion can be used for **diagnostic** and **communication** variance.

9.2 Add and Assign Input Files to Variants

Open Input Files editor via **Project | Input Files** or use the **input file button**  of DaVinci Configurator Pro toolbar.



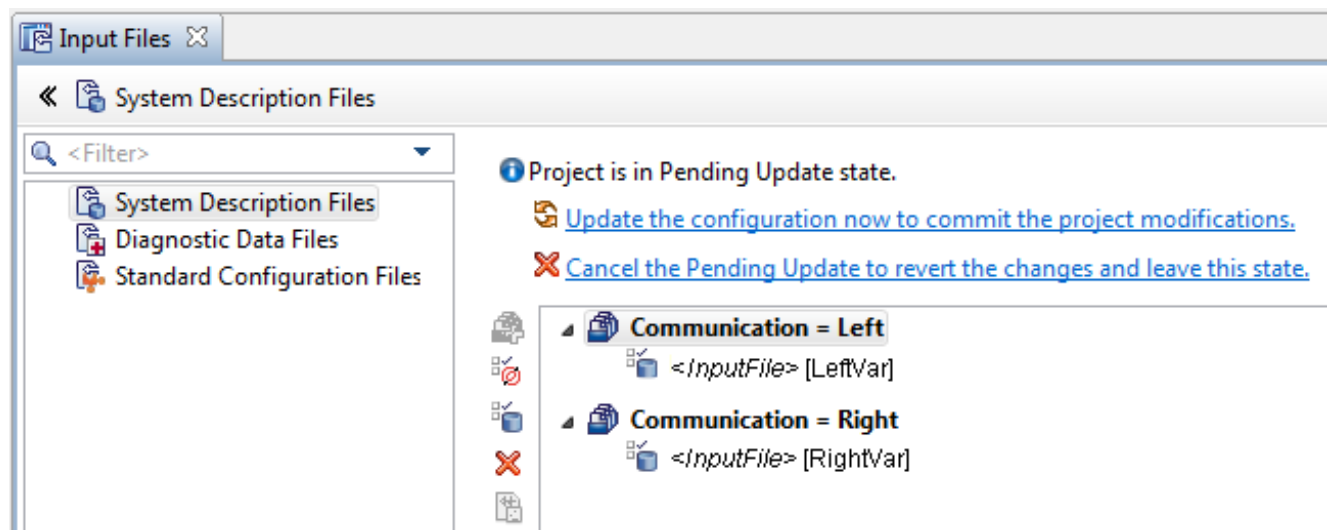
Select **System Description Files** and add your Input Files via **Add File Sets icon** . The Add File Set Assistant opens.



Note

The following step is necessary for each variant.

Select the **Post-Build Criterion Value** (Variant) your input file will be valid and go on with **[Next >]**. Add **[+]** your input file(s), select your ECU Instance and confirm with **[Finish]**.



After adding and assigning your input files our project is still in PENDING UPDATE state.

Project is in Pending Update state.

[Update the configuration now to commit the project modifications.](#)

[Cancel the Pending Update to revert the changes and leave this state.](#)

Select **Update the configuration now to commit the project modifications** to start update process.



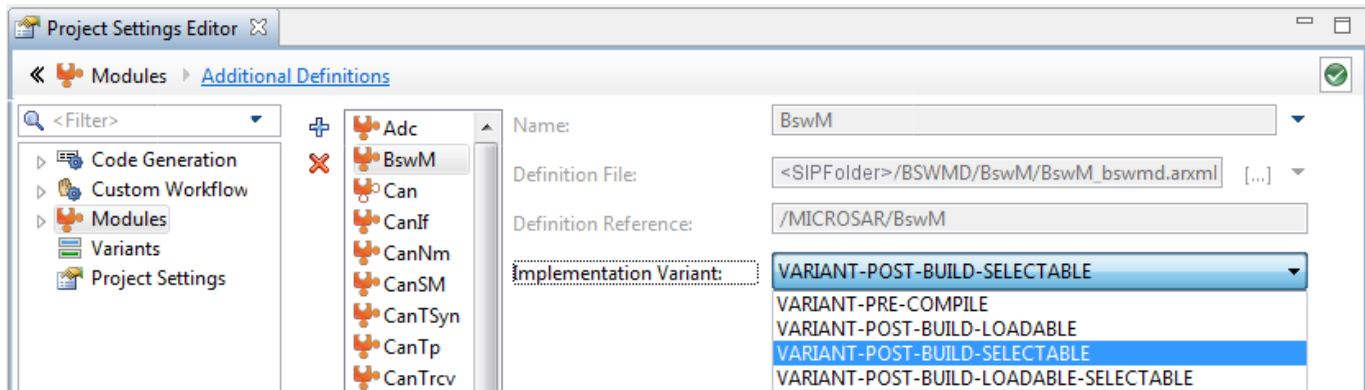
Cross Reference

What happens while Update Process? See section STEP2 Define Project Settings on page 29.

9.3 Define Variance for BSW Modules

Open Project Settings Editor via **Project | Project Settings** or use **project settings icon**  at **DaVinci Configurator Pro** toolbar.

Select **Modules** to see all modules currently activated for your project.



For each BSW module define if it supports variance or not. Therefore choose **Implementation Variant**.

> **VARIANT-POST-BUILD-SELECTABLE**

No post-build loadable update of the configuration at post-build time. All variants are configured at pre-compile time.

> **VARIANT-POST-BUILD-LOADABLE-SELECTABLE**

post-build loadable update of the configuration data is supported using MICROSAR Post-Build Loadable. This feature required dedicated licensing.



Note

Multi selection is possible within module view of the DaVinci Configurator Pro, this enables you to set variance for several BSW modules at the same time.

9.4 Configure and Validate BSW

The **DaVinci Configurator Pro** highlights variant parameter and container using a brown [V]. When changing the configuration it has to be considered whether the change shall be applied to all variants or if the change shall be done only for one variant.

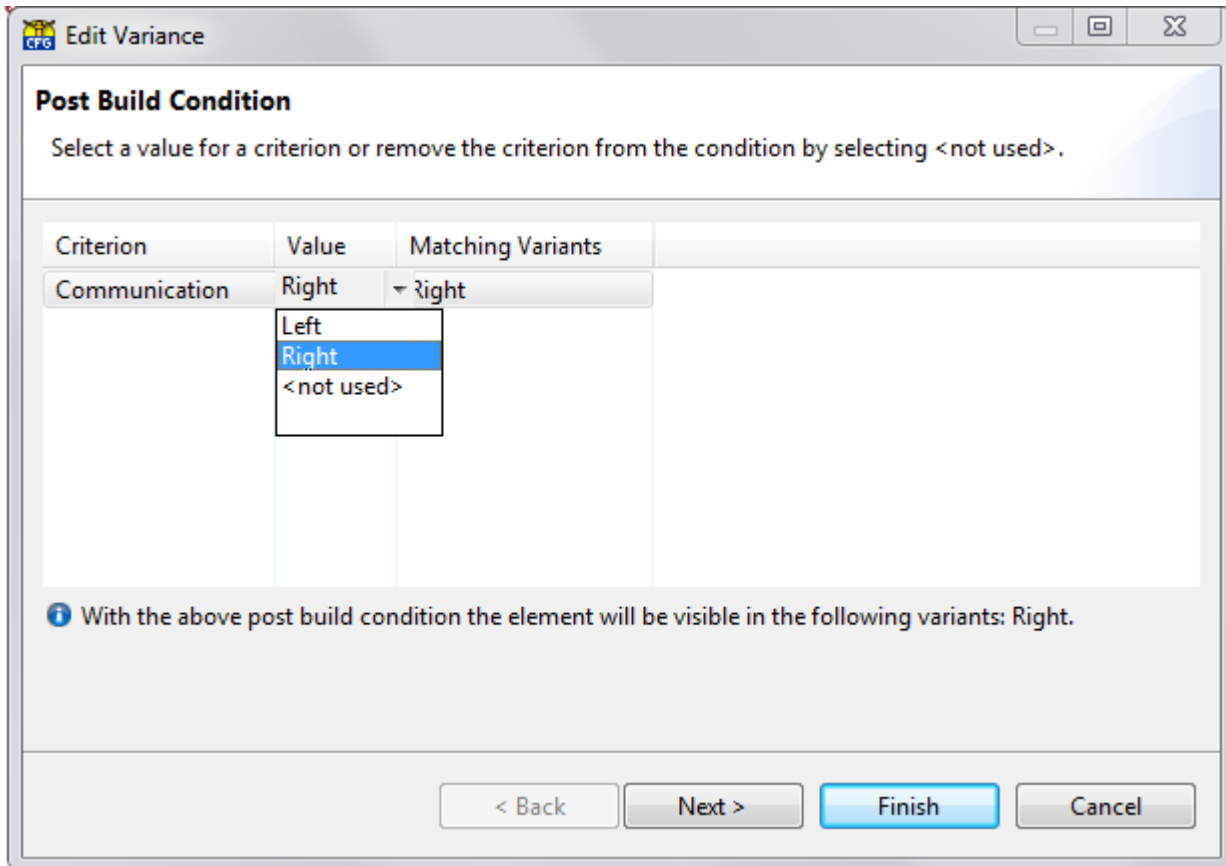


Note

You can select your **Active Variant** at the **DaVinci Configurator** toolbar.

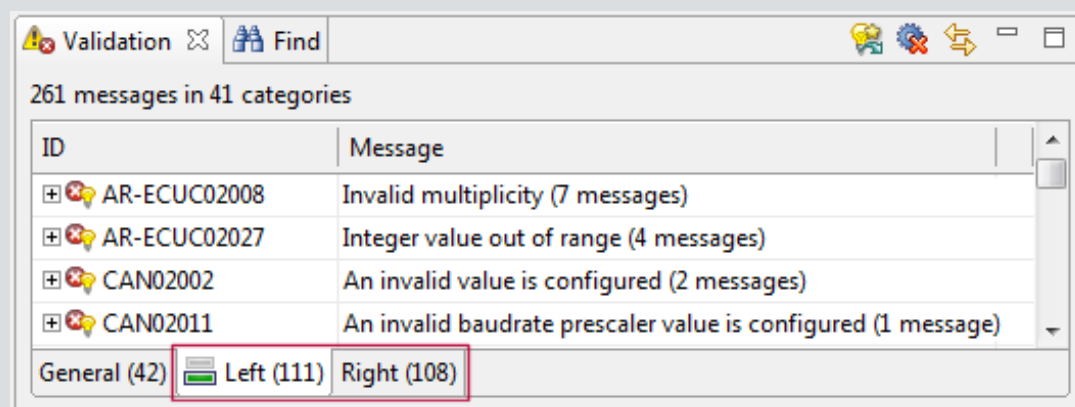


To define a value for a single variant only, right-click on parameter, select **Edit variance** and define variant (**Value**).



Note

The validation view includes views for each variant.



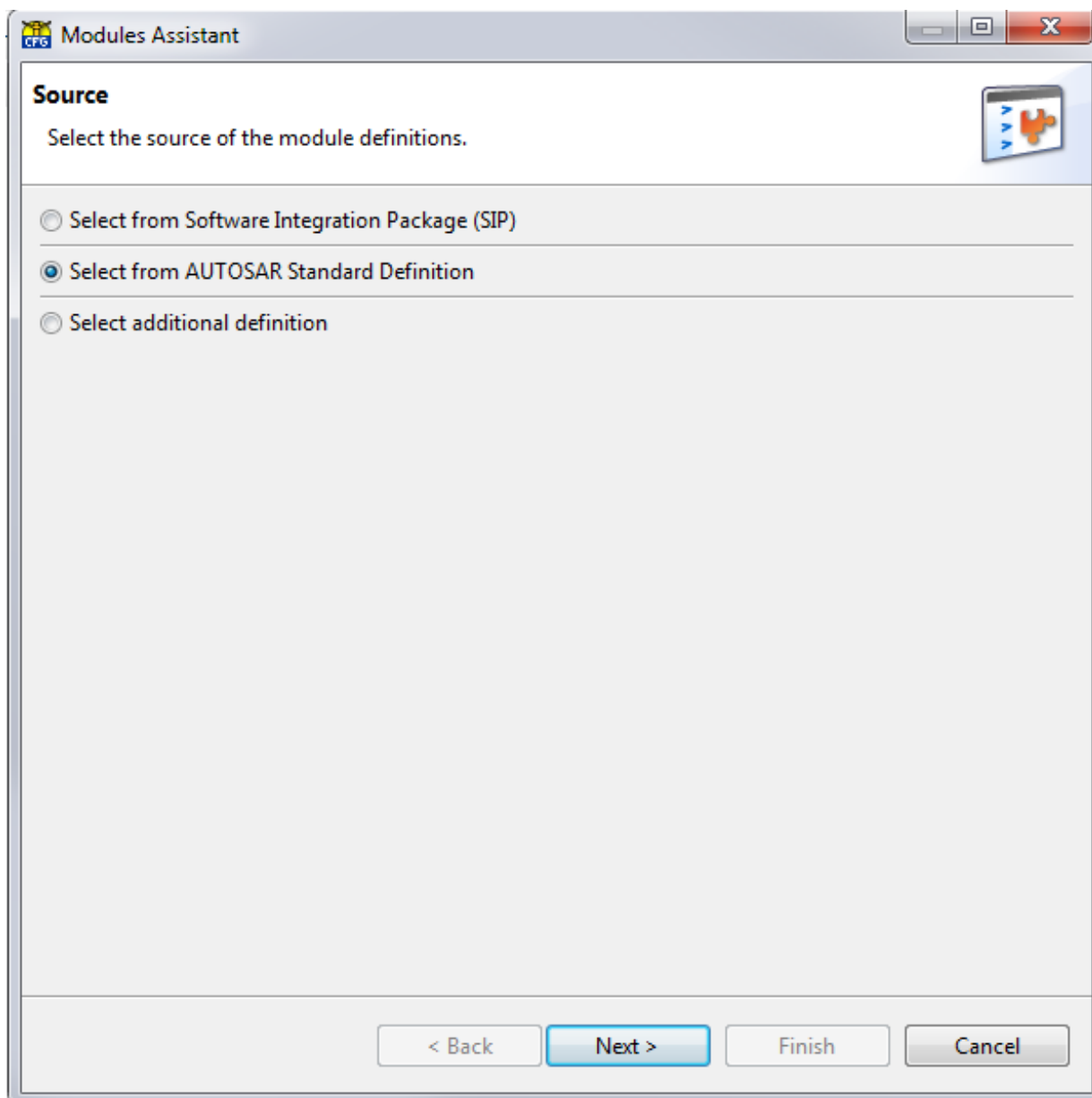


10 Add Module Stubs From AUTOSAR Definition

In some cases it is necessary to add a module, which is not delivered within your SIP, e.g. a delivered module (DCM) needs references and interfaces to a not delivered module (NVM), otherwise the validation fails.

To avoid validation and generation errors you have to add the required module according to AUTOSAR standard definition. Therefore open **Project Settings Editor**, select **Modules** and open Modules Assistant via add [+].

Choose **Select from AUTOSAR Standard Definition** and go on with **[Next]**, select the required module and confirm with **[Finish]**.



**Add Module Stubs from AUTOSAR definition**

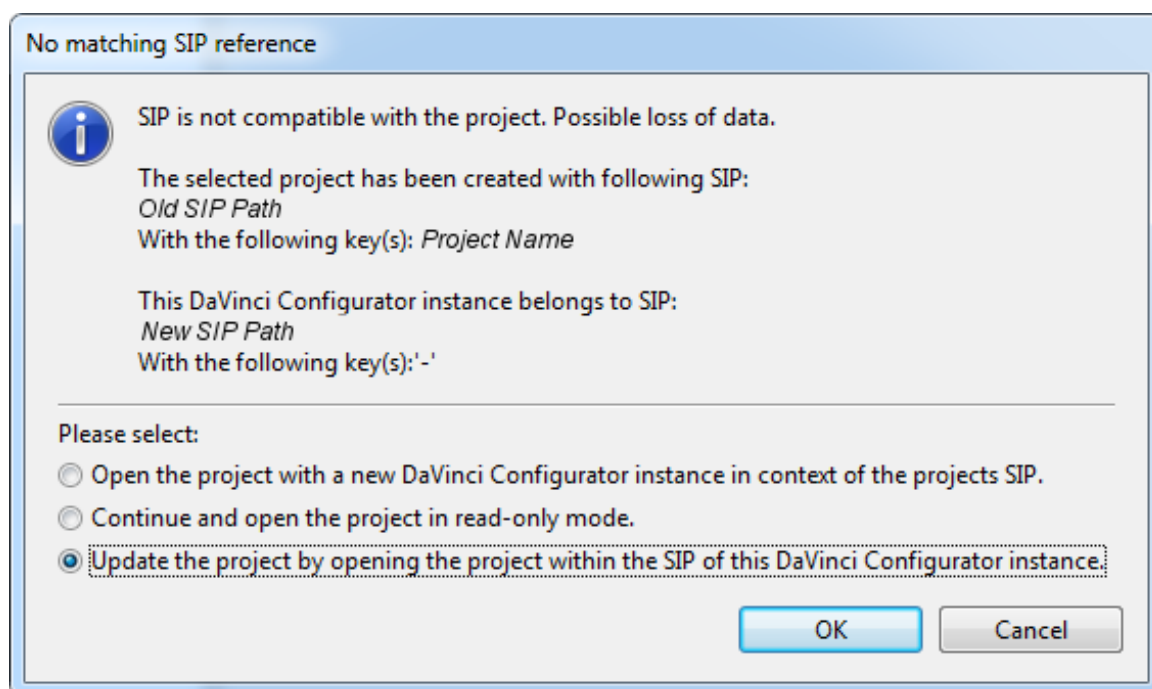
Now the module is available at the **Basic Editor**. The module based on AUTOSAR Standard definition needs to be configured basically with all necessary definitions required by the MICROSAR modules within your project.



11 SIP Update

You received a newer version of your SIP. This requires an update of your current project. Therefore open the project with the new version of the **DaVinci Configurator Pro**. The **DaVinciCFG.exe** is located within the folder `<UpdateSIP>\DaVinciConfigurator\Core`.

The **DaVinci Configurator Pro** identifies that the project was created with an older SIP and displays the following message. Select **Update the project by opening the project within the SIP of this DaVinci Configurator instance** and click **OK**.



The update will performed. Wait until the **DaVinci Developer** project opens. Click **[Accept]** when the warning message appears. Click **[Import]** in the next dialog.

The Signal Import Mode dialog opens, select **Use import mode for all remaining objects** and click **[OK]**. The dialog proposing to **Save and close now** opens, confirm with **[Yes]**.

The **DaVinci Developer** project is closed and the update of the **DaVinci Configurator Pro** project continues. Once the update finishes click **[Ok]**.

The project is now migrated to the new SIP.



12 Project Migration

12.1 Migration Steps from Release 17 SIP to Release 18

Open your existing configuration with the new SIP environment, i.e. Configurator 5.15.xx and execute **[SolveAll]** . After that, solve the remaining validation messages according to your knowledge and make sure all unmapped schedulable entities are mapped (RTE01055).

12.2 Migration Steps from Release 16 SIP to Release 17

Open your existing configuration with the new SIP environment, i.e. Configurator 5.14.xx and execute **[SolveAll]** . After that, solve the remaining validation messages according to your knowledge.



IV Appendix

- > [FAQ](#)
- > [Release Notes](#)
- > [Whats new, whats changed](#)
- > [Glossary](#)
- > [Index](#)



1 Frequently Asked Questions



You have a certain question? You just want to know how to do e.g. a certain setting without reading the whole document again? Then go on reading the following list and use the links to get at the place in the document where your question will be answered. This chapter will be extended continuously.

1.1 Problems with using two different DaVinci Configurator Versions on the same PC

You get problems with using DaVinci Configurator versions 5.10.xx and DaVinci Configurator version 5.11.xx on the same PC?

Start your project with command line option **-clean** for solving Eclipse caching problems when DaVinci Configurator versions 5.10.xx and older are used on the same PC with DaVinci Configurator version 5.11.xx and younger.

1.2 Annotations for any parameter

How to append an annotation to a parameter

For almost any parameter you can add individual annotations regardless whether you are in the Basic Editor or in the Configuration Editor. Open the context menu of a parameter by clicking the little right-pointing arrow and click **Add annotation**.

The screenshot shows the DaVinci Configurator interface with a context menu open for a parameter. The parameter is 'Length [Byte]' with a value of '8' and a unit of 'dec'. The context menu includes the following options:

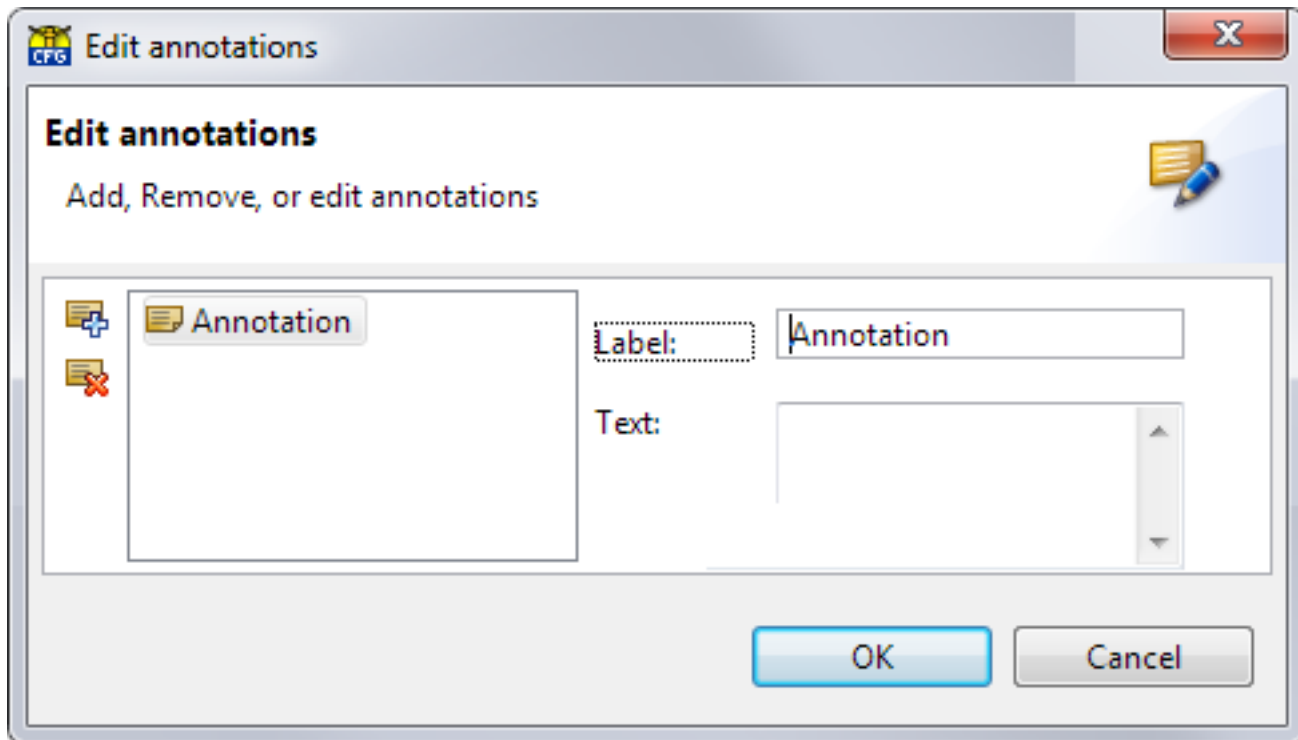
- Set to default
- Set user defined
- Remove user defined
- Create parameter
- Delete parameter
- Add annotation
- Edit annotations
- Copy Path
- Physical Units
- Show in
- Show properties

The interface also shows other parameters like 'Callout', 'Signal Processing' (set to DEFERRED), 'Type' (set to NORMAL), 'Tx Behavior', 'Minimum Delay Time [ms]' (set to 0), 'Unused Area Default' (set to 0x0), 'Clear Update Bit', 'Tx Mode True', and 'Mode' (set to PERIODIC).

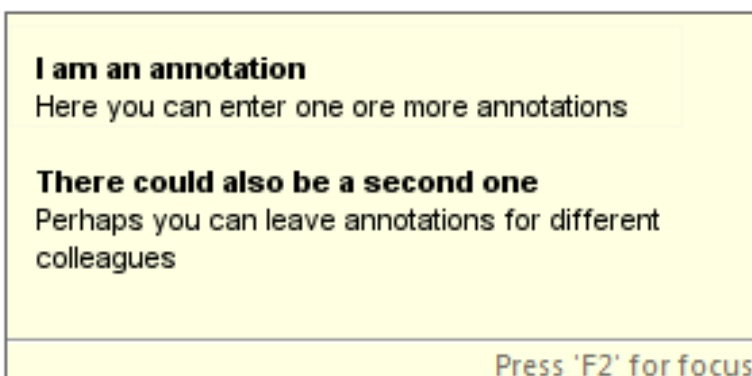


The annotation editor will open. You can enter one or more annotations each with a headline and floating text.

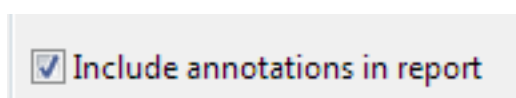
Via **Edit annotations** you can open and edit an already written annotation.



To read your annotations you can either open it via the editor or just by touching the little letter icon with the mouse, its content will be displayed like shown below as an example.



Via a report (**Project|Report...**) it is possible to see all your annotations. Make sure to activate the checkbox **Include annotations in report**





1.3 Find Reference Container

1.3.1 How to find references using the [Find] dialog

Open the **Find** dialog, type in **value==""** and give a container name between the quotes.



2 Release Notes

This section gives an overview of the changes which have been made on the MICROSAR Basic Software Modules (BSW) within the last releases.

Listed changes are new features or the extension of existing functionality. Please note that this document does not provide a list of fixed issues. Open issues are documented in the issue report document IssueReport_<CBD-Number>.pdf, which is part of the deliveries documentation.

This section refers to Vector internal tracking numbers (ESCAN, DAVID tickets and Feature List numbers) which uniquely identify the changes.

Please refer also to the Technical References for more information on the changes, modifications or extensions.

Changes to **DaVinci Configurator Pro** and **DaVinci Developer** are documented in Release Notes provided along with the tools.

2.1 Release 18

Breaking Changes

Modification of existing functionality or APIs that can have an impact on existing applications (such as SWCs, CDDs ...).

Change ID	Affected Modules	Description
FEAT-1824	AllBswModules	The checks required for SafeBSW use-cases are now configured in each component individually using the switch <Ma>SafeBswChecks. Impact to existing projects: > SafeBSW projects: Ensure that this switch is enabled for all BSW modules that are ASIL relevant > Non- SafeBSW projects: This switch should be disabled by default. No changes are expected.
FEAT-1861	CRY CRYIF CSM SECOC	Introduction of the AR4.3 compliant crypto stack including CSM, CRYIF, SECOC and selected CRYPTO drivers. Please note that OEM and project dependent either the new (AR4.3 based) or the old (AR4.2 based) crypto stack is delivered. Impact on existing projects:



Change ID	Affected Modules	Description
		> If the old crypto stack is included (no CryIf) no change > If the new crypto stack is delivered for the first time the crypto stack integration must be redone due to changed APIs and architecture Beta Implementation. This feature shall not be used for production.
FEAT-1939	DET	The DET implementation has been realized according to AR4.3 specification. Impact to existing projects: > If the DET S-SWC interface is used the DET integration has to be reworked due to changed SWC APIs. > Please refer to the DET technical reference for migration and compatibility mode hints.
FEAT-2231	COM	The semantics of the attribute ComTxRepetitionCnt has been changed in AR4.2.2. It now defines the number of repetitions excluding the initial send request. In previous specification versions the initial transmission was included. Impact to existing projects: Number of transmissions may change. An update of the ECU configuration or the upstream input files may be required.
FEAT-2331	FLS (Stub) WDG (Stub)	The generic WDG and FLS drivers now use the API Infix to allow better coexistence with other drivers. Impact to existing projects: - Due to the API rename the integration of these modules may be reworked (replace API calls - no change in API semantics).
FEAT-2364	WDGM	The existing WDGM that was used in non-SafeBSW projects is replaced by the WDGM that was only used for SafeBSW and multi-core projects in the past.



Change ID	Affected Modules	Description
		> Thus there is a single WDGM implementation for all use-cases > The WDGM provides options for flow control monitoring, SafeBSW and multi-core > The WDGM also supports mode ports > The WDGM initialization is configured automatically for single core projects. Multi-core initialization still requires a manual initialization sequence. Impact on existing projects Affected are only projects that did not use SafeWDG Stack in the past. > The configuration of the old WDG stack is copied to the new WDG implementation. Some additional parameters may have to be configured. > Verify the configuration after the SIP update and fill in missing parameters. > Configure the WdgM Service Connector Mapping as it cannot be taken over > No changes to the embedded integration required if the same feature set are used

Extension

New APIs have been introduced or functionality has been added. There is no influence on existing implementations.

Change ID	Affected Modules	Description
FEAT-1520	COMXF DaVinci Configurator Pro 5 DaVinci Developer E2EXF RTE SOMEIPXF	Transformer error handling for explicit S/R communication is now supported.
FEAT-1989	vMIRROR	Mirroring between two CAN channels (CAN to CAN) is supported. Beta Implementation. This feature shall not be used for production.



Change ID	Affected Modules	Description
FEAT-2072	IPDUM	IPDUM now provides a distinct Rx and Tx pathway main function. By appropriate scheduling this allows reducing latency of e.g. gateway use-cases.
FEAT-2127	RTE	The RTE is now able to run SWCs with transformers in one partition and the BSW (esp. COM-Stack) in a separate partition.
FEAT-2151	ETH ETHIF ETHTRCV TCPIP	Provision of Ethernet bus diagnostics information: > Discarded ARP Entries > Duplicate DHCP Addresses > IP Configuration > DHCP Status > Rx & Tx Statistics
FEAT-2161	LIN (S32) LINIF	LIN and LINIF now support microcontrollers with multiple LIN drivers for different LIN peripheral interfaces such as UART and a dedicated LIN controller or external LIN controller.
FEAT-2194	WDGM	WDGM now supports the usage of OS counters for deadline monitoring.
FEAT-2229	SECOC	A dedicated Rx and Tx path MainFunction is now provided by the SecOC module.
FEAT-2234	ETH ETHTRCV	Link Quality Monitoring of Automotive Ethernet (100BASE-T1) is now supported by specific Ethernet transceiver drivers: > API to retrieve the monitored values > API to reset the stored values
FEAT-2252	COMXF CRC DaVinci Developer E2E Library E2EXF RTE SOMEIPXF	Support of AUTOSAR E2E profile 7 for large data using transformers.
FEAT-2263	LINIF	Support fast schedule table changes not aligned to schedule table slot definitions by allowing



Change ID	Affected Modules	Description
		schedule tables to be changed directly after any frame transmission.
FEAT-2289	FR	Using the optional callout ApplFr_WriteCcReg() it becomes possible for the user to modify the PLatestTx settings of the FlexRay controller. Use-case: Number of the last minislot in which a frame transmission can start in the dynamic segment shall be configurable dynamically during startup or runtime. The feature can be enabled by defining the switch FR_OVERWRITE_REGISTER manually in the build environment or the use-configuration file. Please Note: This is a hardware specific feature that may not available with all FR implementations.
FEAT-2307	DCM vDIAGXF	I/O control is able to use S/R communication for struct data types containing atomic and byte aligned data types. Beta Implementation. This feature shall not be used for production.
FEAT-2333	COM	Tx Timeout for cyclic messages is now supported.
FEAT-2338	Diagnostic Data Model	Extended the Autosar Diagnostic Extract based workflow to support boolean data types. These data types are marked with the CANdelaStudio attribute "ASREncoding" set to "boolean".
FEAT-2340	TCPIP	Transmission of gratuitous ARP packages are now supported to accelerate start up of Internet Protocol based communication (i.e. TCP or UDP).
FEAT-2373	WDGIF	The APIs towards WDG are now configurable to support hook and wrapper use-cases.
FEAT-2724	DaVinci Configurator Pro 5 DaVinci Developer FR FRIF FRNM FRSM	The FlexRay stack now supports configurations with multiple FlexRay controllers that operate each on a distinct cluster. Beta Implementation. This feature shall not be used for production.



Change ID	Affected Modules	Description
FEAT-2764	MEMIF	It is now possible to use driver specific APIs using the optional container MemIfMemHwA/MemIfMemHwAAPI. If the container is present each FEE/EA API and the required header can be defined individually. The feature shall simplify the integration of 3rd party components that do not follow the standards naming convention.
FEAT-2778	DOIP SOAD	Enhance data blocks to up to 128k for diagnostics and flashing over Ethernet (previous limit was 64k). Thus the data type of DoIPTargetMaxPduSize and DoIPTargetMaxBufSize was enlarged to 16 bits.
FEAT-936	DCM Diagnostic Data Model VSG Complex Driver	DCM now supports variance as defined by the Vehicle System Group (VSG) mechanism of Ford CDD files.

Information

The internal behavior of the BSW has been improved without any change in the API. Only for information purpose.

Change ID	Affected Modules	Description
FEAT-1918	ADC (VTT) CAN (VTT) DaVinci Configurator Pro 5 EEP (VTT) FLS (VTT) GPT (VTT) PORT (VTT)	The configuration process of the "VTT-only" use-case was improved. Apart from the very few VTT-specific parameters, parameters of VTT modules and corresponding hardware stub modules are now synchronized. Affected VTT modules: Can, Port, Gpt, Fls, Eep, Adc
FEAT-2336	J1939TP	Production release of fast-packet protocol for the transport of GPS data according to NMEA 2000.
FEAT-2346	Legacy DB Converter	TP connections for diagnostics are now configured explicitly as half-duplex as defined by Ford GGDS 004.
FEAT-2351	OS	The MICROSAR guest OS allows realizing a classic AUTOSAR Stack on top of a POSIX compatible OS by providing AUTOSAR compliant OS APIs (SC1) and configuration elements.



Change ID	Affected Modules	Description
		Due to the usage of POSIX APIs the guest OS is hardware independent.
FEAT-2355	General	Several files have been removed from the SIP as the old DaVinci Configurator 4 framework has been removed: > Major parts of the .\Generators\Components Folder have been removed. BSWMD files are now only located in the .\BSWMD folder. > .\Generators\MicrosarGen removed
FEAT-2737	ETHIF vETHFW (Firewall ETH)	A static firewall for Ethernet communication is now available as prototype. Beta Implementation. This feature shall not be used for production.

2.1.1 Required AUTOSAR Tools

This MICROSAR release requires the usage of the following tools:

Tool	Version
DaVinci Developer	3.13 (SP3) Please use the latest service pack available from the Vector download center
DaVinci Configurator Pro	The latest service pack version is delivered as part of the SIP. Upcoming service packs can be retrieved from the Vector download center

2.2 Release 17

Breaking Changes

Modification of existing functionality or APIs that can have an impact on existing applications (such as SWCs, CDDs ...).

Change ID	Affected Modules	Description
FEAT-2097	SPI (VTT)	Spi_GetHWUnitStatus() now returns SPI_BUSY when a transmission is pending. In the past SPI_IDLE was returned. Impact to existing projects: Evaluation of the return value may have to be updated.
FEAT-784	DEM	DEM now uses available data alignment and size



Change ID	Affected Modules	Description
	ECUC	information configured in the ECUC module (EcuC/EcucGeneral) to estimate the NVM block size. Impact on existing projects: The alignment attributes (EcuC/EcucGeneral) shall match the linker settings. > For missing attributes the DEM substitutes default values. > DEM reports the result of its estimation to the user, as well as substitutions for missing attributes. > Due to missing values and substitutions, the estimated buffer size might differ from the actual size, so estimation is just reported and the buffer size value in NVM is not changed automatically. When the user knows the exact size (e.g. if the block size is extracted from the MAP file), DEM's recommendation can safely be ignored.

Extension

New APIs have been introduced or functionality has been added. There is no influence on existing implementations.

Change ID	Affected Modules	Description
FEAT-1350	J1939TP	The ISOBUS and NMEA Fast Packet Transport Protocol (FPTP) has been implemented. Beta Implementation. This feature shall not be used for production.
FEAT-1445	DCM DEM Diagnostic Data Model	WWH-OBD has been released and extended: > NODI removal added
FEAT-1454	BSWM COMM ECUM ETHIF ETHSM	Support Partial Network Cluster specific Ethernet switch port handling. Beta Implementation. This feature shall not be used for production.



Change ID	Affected Modules	Description
	UDPNM	
FEAT-1509	TCPIP	Support of TCP Keep Alive mechanism according to IETF RFC 1122.
FEAT-1611	BSWM PDUR	Additional OVTP gateway features available: > Extended PduR switching support
FEAT-1666	DOIP	Now, there is only one DoIPChannel required for each target ECU even if the target ECUs' DoIP logical address differs between request and response (simplified configuration).
FEAT-1706	DOIP	Support of OEM-specific DoIP payload type handling.
FEAT-1783	RTM	A script was developed to automatically determine runnable names, insert their Vfb Hook functions, add measurement points and generate a template implementation for the Vfb Hook functions. The script uses the Automation Interface of DaVinci Configurator and can be executed by using the context menu (right-click) of the RTM module in the Basic Editor. Beta Implementation. This feature shall not be used for production.
FEAT-1816	DOIP SOAD TCPIP	The MainFunctions of TCPIP and SOAD can now be split into an Rx and a Tx MainFunction. This is in order to reduce latency of gateway routings. To do so, use the following sequence: > Rx MainFunctions of communication stack > Gateway MainFunctions (if required) > Tx MainFunctions of communication stack
FEAT-1841	DCM	Support of Routing Info Byte (SID 0x31 "RoutineInfo") in Dcm.
FEAT-1842	EEP (VTT) FLS (VTT) WDG (VTT)	VTT is now able to also simulate drivers of external devices for > FLS > EEP > WDG Precondition is that these devices are configured as AUTOSAR modules in DaVinci Configurator



Change ID	Affected Modules	Description
		Pro.
FEAT-1862	Crypto Algorithm Library	The following algorithms have been added: Symmetric algorithms: > ChaCha2020 > ChaCha2012 > Cipher Modes: CTR, GCM, OFB, CFB, XTS Asymmetric algorithms: > RSA with 3072 bit key length > Elliptic Curve 25519 Hash functions: > SHA3 Symmetric signatures: > GMAC Beta Implementation. This feature shall not be used for production.
FEAT-1890	COM	The COM description based routing for signals allows a more efficient way to route signals between messages with similar layout compared to an ordinary signal routing. The description routing is now able to also handle > signals with update bits > signals that are placed at differing locations in the destination PDU.
FEAT-1900	DaVinci Configurator Pro 5 DEM Diagnostic Data Model	Supporting S/R communication in the Dem application SWC interface.
FEAT-1901	DCM	Support of I/O control using S/R communication as defined by AUTOSAR RfC68881. Beta Implementation. This feature shall not be used for production.



Change ID	Affected Modules	Description
FEAT-1924	DCM	Memory ranges will be exported for diagnostic services ReadMemoryByAddress (0x23) and WriteMemoryByAddress (0x3D). The memory range must be defined in CANdelaStudio as a service 0x23 or 0x3D using one of the following protocols: > 0x23 ALFID Memory-Start-Address > 0x3D ALFID Memory-Start-Address Please consider: > The Address-and-Length-Format-Identifier (ALFID) must be of constant value, e.g. 0x24. > The memory-start-address must be a variable parameter. > The memory-size must be defined by a service attribute of the name "Memory Area Size", e.g. "0x1000".
FEAT-1941	COM	It is now possible to route signals between messages that are transmitted or received by TP PDUs (such as J1939 messages).
FEAT-1953	SCC SOAD TCPIP	TCPIP now supports transmission requests of PDUs with a data length unknown at the point of the transmission request. The final length is provided by each subsequent call of CopyTxData.
FEAT-1955	PDUR	The TP buffer handling became more flexible by allowing buffers to be assigned to routing relations. > TP routing relations can now refer to dedicated buffer(s) using the PduRDestTxBuffer-Ref. > The buffer pool is now configured in the TxBufferTable list and no longer in the TpBufferTable. > Buffers that are referred by routing relations are used by these routing relations in a preferred way. > If all assigned buffers are used (or none assigned) globally available buffers are used (i.e. these are buffers that are not referenced by a routing relation).



Change ID	Affected Modules	Description
		> Tx Buffers can be shared between different routings as they can be referred to by multiple routing relations. Impact to existing configurations: A migration script will automatically transfer the buffers from TpBufferTable to TxBufferTable.
FEAT-1980	XCP XCP On CAN XCP On TCPIP	Multi- Connection support has been added. It is now possible that multiple client tools connect to the ECU on the same or over different networks at the same point in time.
FEAT-1995	CRY CSM	It is now possible to use hardware and software based CRYs in the same project.
FEAT-1996	TLS	TLS Heartbeat Extension (as defined by IETF RFC 6520) Beta Implementation. This feature shall not be used for production.
FEAT-1998	ETH Switch Driver (Bcm8953x) ETHIF ETHSW TSYNC ETH	Hardware time stamping for switches is now supported. Note: In a first step this feature is only supported for BCM8953x aka Leo.
FEAT-2001	SECOC	AR4.3 SecOC RfCs have been implemented: > RfC 71785 - [SecOC] - Send authentication data in a separate message > RfC 71787 - [SecOC] - Linking of split authentication data / authentic PDU Beta Implementation. This feature shall not be used for production.
FEAT-2002	Base EcuC Converter COM Common Files COMXF DaVinci Configurator Pro 5 DaVinci Developer	Support 64 Bit Signal Types according to AR 4.2.2



Change ID	Affected Modules	Description
	IPDUM RTE	
FEAT-2005	TLS	Support of cipher suite TLS_RSA_WITH_AES_128_CBC_SHA256
FEAT-2024	CANIF	By a CANIF extension it is now possible to route a CAN PDU using the PDU Interface Layer Gateway and the TP Layer Gateway at once. Use-case if a routing from CAN to CAN (low level) and DOIP (high level) routing. Note: This feature requires dedicated licensing.
FEAT-2039	BSWM COMM	If the application triggers a ModeLimitation to NoCom it is now possible to reset the ECU. This is realized by a optional notification to the BSWM that can trigger any action based on this mode indication.
FEAT-2051	CRY	VTT now provides a VTT MCAL that simulates a hardware based CRY driver.
FEAT-2055	RTM	RTM can be disabled at runtime in a safe way. This allows the module to be shipped as part of safety-related ECUs. Once RTM is enabled at runtime the module must be considered as QM software. Note: This feature requires dedicated licensing of RTM.
FEAT-2058	SECOC	SecOC now supports post-build selectable for variant handling. Note. This feature requires the licensing of MICROSAR IDM (Identity Manager).
FEAT-2068	AVTP (IEEE1722) SysService_Srp8021Q	Finalization and release of SRP Client.
FEAT-2076	CANIF	A passive wakeup of ECUs is now sped up (i.e. application messages are send earlier) as the PnTxfilter is no longer considered in such scenarios. The PnTxFilter remains relevant for NM PDU filtering during a wake-up that was triggered from the network.



Change ID	Affected Modules	Description
FEAT-2082	PDUR	PduR is now able to call a Tx queue specific over-run notification to signal loss of data to the application.
FEAT-2086	UDPNM	UdpNM now supports the immediate transmission feature (as known from CanNm). See also AUTOSAR 4.3 RFC 70837.
FEAT-2087	CANNM UDPNM	Implementation of AUTOSAR 4.3 RfC72701 Coordinator should send NM message when Sleep Ready Bit is set.
FEAT-2088	XCP	Using the new API Xcp_ModifyProtectionStatus it is now possible to disable (or enable) the seed&key protection at runtime. By default the protection is enabled (as in past versions).
FEAT-2136	TCPIP	Storage of IPv4 link-local (AutoIP) addresses in NvM is now possible.
FEAT-972	SOAD	Support of SoAd PDU fan-out with a reduced number of buffers (memory and runtime optimization).

Information

The internal behavior of the BSW has been improved without any change in the API. Only for information purpose.

Change ID	Affected Modules	Description
FEAT-1340	IPDUM	The container PDU feature has been released. Hint: This feature is typically used in combination with CAN-FD.
FEAT-1583	CRYPTO	Support of concurrent TLS and CSM usage.
FEAT-1656	Bus Mirroring Complex Driver CAN SOAD	MICROSAR offers now a mirroring module that allows mapping the complete network communication of a selected network to another network. The following mirroring scenarios are currently supported: > CAN->ETH Beta Implementation. This feature shall not be used for production.
FEAT-1779	SECOC	Release of SECOC module.
FEAT-1784	DCM	Additional validations were implemented that



Change ID	Affected Modules	Description
		check the configured DCM buffer size against the minimum required size for the particular configuration. In this way most of the major runtime problems caused by a too small buffer (such as disrupted request reception, permanent negative responses etc.) are avoided.
FEAT-1879	RTP	Real-Time Transport Protocol (RTP, https://tools.ietf.org/html/rfc3550) is now available as BSW module. Note: This module requires dedicated licensing. Beta Implementation. This feature shall not be used for production.
FEAT-1963	DCM	Release of variant diagnostic data handling of DCM using post-build selectable. Note: This feature requires MICROSAR IDM to be licensed.
FEAT-2004	CAN CANIF	Release of FEAT-912: Support of several CAN Drivers in one ECU configuration.
FEAT-2239	CRY (Renesas RH850)	Hardware CRY for Renesas RH850 (IcuS) available. Beta Implementation. This feature shall not be used for production.

2.2.1 Required AUTOSAR Tools

This MICROSAR release requires the usage of the following tools:

Tool	Version
DaVinci Developer	3.13 (SP2) Please use the latest service pack available from the Vector download center
DaVinci Configurator Pro	The latest service pack version is delivered as part of the SIP. Upcoming service packs can be retrieved from the Vector download center

2.3 Release 16

Breaking Changes

Modification of existing functionality or APIs that can have an impact on existing applications (such as SWCs, CDDs ...).



Change ID	Affected Modules	Description
FEAT-1915	WDGM	A new feature for checkpoint generation has been added in this release of the WDGM. With the switch "WdgMGenerateCPIIdAsPortDefinedArgument" it can be influenced if the checkpoint ID shall be generated as a port-defined argument or a single port per supervised entity is generated. Please note: To be compatible to older versions of the WDGM you need to enable this option. However, an AUTOSAR compliant behavior is achieved by disabling this functionality. By default this feature is disabled.
FEAT-1964	DEM	Removed post-build selectable from EnableConditionGroup and StorageConditionGroup. Variable groups of those types need to get duplicated and referenced by the events of a variant.

Extension

New APIs have been introduced or functionality has been added. There is no influence on existing implementations.

Change ID	Affected Modules	Description
FEAT-1327	J1939DCM	The J1939DCM DM1 transmission is now ISOBUS compliant (cyclically only if active instead of cyclic)
FEAT-1413	SOAD	Service oriented communication for Linux (BSD socket API enhancements for SOME/IP and Service Discovery).
FEAT-1498	DaVinci Configurator Pro 5 TCPIP	It is now possible to mark TcpIp (IPv6) address prefixes as on-link during configuration time.
FEAT-1516	TCPIP	It is now possible to assign a static IPv4 address in addition to an address assigned by DHCP (priority based). Limitations: > one assignment method of each type per address > one IPv4 address per logical controller (VLAN, physical controller) Beta Implementation. This feature shall not be used for production.



Change ID	Affected Modules	Description
FEAT-1574	CAL CPL CRY SECOC	New SECOC functionality: > Generic Freshness value interface > Provision of SecOC_VerifyStatusOverride API > SecOCAuthInfoTxLength and SecOCFreshnessValueTxLength can have an arbitrary size but must result in a full byte length
FEAT-1641	SD	The following AUTOSAR 4.3 features are now supported: > [Issue 68746] Impact of configuration options on service matching algorithm is unspecified > [Issue 68823] Switching of EventHandler from Unicast to Multicast
FEAT-1678	FLS (VTT) VTT Control Layer	VTT memory stack now supports the AURIX Fee in case of dual target projects.
FEAT-1723	DEM	Support for OBD major monitors. Please note: This feature need to be licensed separately. OBD major monitors require a project specific analysis of the requirements to the basic software. Beta Implementation. This feature shall not be used for production.
FEAT-1724	DCM DEM	Support for DTR (Diag Test Results) for OBD projects. Beta Implementation. This feature shall not be used for production.
FEAT-1726	DaVinci Configurator Pro 5 DaVinci Developer RTE	RTE Nv ports can now be connected to S/R ports. Beta Implementation. This feature shall not be used for production.
FEAT-1738	ETHIF	Allow concatenated receive buffers that can be smaller than max frame size.
FEAT-1770	DEM	Up to 255 Enable- and Storage-Conditions are now supported.
FEAT-1791	RTE SOMEIPXF	Support autonomous error responses from transformers based on AUTOSAR 4.2.2 for SOMEIPXF.



Change ID	Affected Modules	Description
FEAT-1807	CFG5 (DCM ARXML Converter) CFG5 (DEM ARXML Converter) DCM Diagnostic Data Model	DCM diagnostic data now can be variant in configurations that use the MICROSAR Identity Manager. Beta Implementation. This feature shall not be used for production.
FEAT-1818	COM	Support the configuration switch minimum delay time for cyclic transmission (Parameter "ComEnableMDTForCyclicTransmission").
FEAT-1820	Base EcuC Converter DOIP	Improved Vehicle Announcement Handling for DoIP based on AUTOSAR [Issue 67021].
FEAT-1846	RTM	> RTM measurements can now be controlled via CAPL without using the test feature set UI. This allows integrating RTM into complex test environments. > Net runtime measurement support on multi-core ECUs (Beta) > Response time measurement on multi core ECUs (Beta)
FEAT-1864	DRM	Release of DRM (Diagnostic Request Manager) (CDDDRM)
FEAT-1889	CAN (VTT) LIN (VTT) VTT Control Layer VTT TechnicalReference	VTT now supports the MICROSAR Identity Manager and PB-Loadable for CAN and LIN. The PB-Loadable update process is not supported in VTT. Note: FlexRay is also supported (since Release 15). Note: This feature requires an IDM / PB-Loadable license.
FEAT-1896	IPDUM	The nPduToFrame mapping as used to pack several COM I-PDUs into one CAN-FD frame has been extended: > Last is best semantics for Tx PDUs > Runtime optimization: Rx pathway only handles PDUs that are actually required by the ECU Beta Implementation. This feature shall not be used for production.



Change ID	Affected Modules	Description
FEAT-1899	DaVinci Configurator Pro 5 DaVinci Developer DCM DIAGXF RTE	Introduction of DIAGXF module to support S/R interaction with DCM. This feature mandates a MICROSAR RTE. Beta Implementation. This feature shall not be used for production.
FEAT-1908	All-MCALs BSWM CAN FR LIN WDG	Access to interrupt controller register has been updated to better support safety use-cases. Per driver it is now possible to configure two options via the parameter UseOsInterruptControl > OS is in charge of the interrupt control register. This new functionality mandates the latest MICROSAR OS. The module no longer initialized the interrupt control register by itself but informs the OS. Register values are derived through OS APIs. This setting is recommended for SafeBSW projects. > Same behavior as today: Depending on the driver implementation (see its technical reference) a driver may perform the initialization of the interrupt control register and accesses the relevant registers directly.
FEAT-1912	ETH Switch Driver (Bcm8953x)	The switch driver has been extended to support the AUTOSAR 4.2.1 frame mirroring and forwarding extensions. > CFP Rules for VLAN-filtering (mirroring) > Reserved MAC multicast packets shall be forwarded by the switch > Mirroring VLAN double-tagging > Enabling/disabling mirroring during runtime > Mirroring filters Please note that these features may not be available for each switch hardware.
FEAT-1922	DCM	Support for OBD2 Mode 0x09 with variable length as required by primary OBD nodes with multiple secondary nodes.
FEAT-1938	CRY	Software CRY now supports the CMAC AES-128



Change ID	Affected Modules	Description
		algorithm.
FEAT-1987	TSYNC ETH	The IEEE 802.AS Pdelay message configuration is more flexible and allows the following options: > Dynamic Pdelay protocol for time master and slave > Optimized Pdelay configuration by time slave Pdelay_Req. Time master answers with Pdelay_Res and Pdelay_res_Follow_Up. > Optionally a static Pdelay configuration can cause Pdelay requests to be ignored.
FEAT-451	DaVinci Configurator Pro 5 DaVinci Developer RTE	The RTE implements automatic scaling (offset and factor) between a FixedPoint and Float. Use-Case: network representation is FixedPoint (with factor and offset) while the internal processing is Float based.

Information

The internal behavior of the BSW has been improved without any change in the API. Only for information purpose.

Change ID	Affected Modules	Description
FEAT-1506	ETH Switch Driver (Bcm8953x)	The Ethernet switch configuration has been improved. Thus only used ports have to be configured and pre-configured ports can be mapped to any used port. Beta Implementation. This feature shall not be used for production.
FEAT-1513	CAN CANIF XCP On CAN	Release of CAN-FD functionality.
FEAT-1640	COM	Runtime improvement in the COM Rx signal notification handling: > COM Rx signal notifications are now always called with the interrupt not being locked by the COM module > The Rx signal notifications are called after a configured amount of notifications has to be



Change ID	Affected Modules	Description
		called. > An ISR lock threshold is configurable per timing domain.
FEAT-1701	FEE	The API ForceSectorSwitch now always causes the sector to be switched. This shall result in a more robust FEE in case of errors.
FEAT-1779	SECOC	Release of SECOC module.
FEAT-1839	FEE (Small Sector Flash)	An alternative FEE implementation is now available that is specifically designed to support devices with very small flash sectors. On such devices (currently RH850) this implementation is more robust and has less overhead.
FEAT-1886	IPDUM	Feature Release: 16bit IPDUM selector field values are now supported for production projects
FEAT-1888	NVM	The problem that RTE invokes the DET during NvM ReadAll (ESCAN00087644) is now solved.
FEAT-1910	SOAD	Release of BSD-Socket API for series production
FEAT-1948	OSEKNM	Release of OSEK-NM implementation.

2.3.1 Required AUTOSAR Tools

This MICROSAR release requires the usage of the following tools:

Tool	Version
DaVinci Developer	3.13.0 Please use the latest service pack available from the Vector download center
DaVinci Configurator Pro	The latest service pack version is delivered as part of the SIP. Upcoming service packs can be retrieved from the Vector download center



3 What's New, What's Changed

What's new and what's changed?

This section explains the changes within this document from the previous version to the one mentioned in the headline.

Version 9.x.x

Author	Date
Monika Sturm	2017-03-28

What's new?

- > [Migration steps](#) from Release 17 to Release 18

What's changed?

- > AUTOSAR Layer models updated
- > Minor changes, screenshots, typos, etc.
- > Topic Command Line Parameters of the DaVinci Configurator removed (please refer to the [DaVinci Configurator](#) help)

Version 8.x.x

Author	Date
Klaus Emmert Monika Sturm	2016-11-22

What's new?

- > [Migration steps](#) from Release 16 to Release 17

What's changed?

- > Minor changed, screenshots, typos, etc.

Version 7.x.x

Author	Date
Klaus Emmert	2016-07-27

What's new?

- > BswM: activation of peripheral interrupt devices via callout [04_ConfigureBSW.htm](#).
- > New view for comfortable task mapping [06_Mapping.htm](#)
- > [Migration steps](#) from Release 15 to Release 16



What's changed?

- > New symbols in the chapter [00_About This User Manual.htm](#)
- > Topics concerning older releases deleted
- > Minor changed, screenshots, typos, etc.
- > Workflow illustration now visible



4 Glossary

Adc

(Analog Digital Converter Driver) The ADC driver abstracts hardware access to the analog-digital converter. For every input, the conversion parameters are configured (e.g. resolution, trigger source and trigger conditions).

AVTP

(Audio/Video Transport Protocol) The Audio/Video Transport Protocol is specified in IEEE 1722/1722a. In AVB networks it is responsible for the transport of audio/video data, including the Presentation Time.

Bfx

(Bitfield functions for fixed point) Library for bit handling in fixed point arithmetic functions

BMCA

(Best Master Clock Algorithm) The Best Master Clock Algorithm is specified in IEEE 802.1AS and is used to detect the device with the most precise time unit in an AVB network. After finding the device with the most precise time unit, it is used as the time base for the entire system.

BswM

(BSW Mode Manager) The BswM module contains vehicle mode management and application mode management. It processes mode requests from SWCs or other BSW modules, and it performs actions based on the arbitration, such as control of deadline monitoring, switching schedule tables, and handling of IPDU groups. In conjunction with the EcuM, the BswM module is responsible for starting up and shutting down the ECU. The BswM also coordinates the multi-core partitions.

Cal (Cpl)

(Crypto Abstraction Library) The Cal library offers SWCs and other BSW modules access to basic cryptographic functions. The individual cryptographic functions are implemented in software via the Cpl module.

Can

(CAN Driver) The CAN Driver abstracts access to the CAN hardware for sending and receiving messages and for switching between controller states (sleep, stop, etc.)

CanIf

(CAN Interface) The CAN Interface offers abstracted (PDU-based) access to the CAN Driver. It controls the CAN Driver ((Can) as well as the transceiver driver ((CanTrcv).

CanNm

(CAN Network Management) Within a CAN network, CAN Network Management is responsible for coordinated transitions between the wake up and sleep state.

CanSM

(CAN State Manager) The CAN State Manager is responsible for the bus-specific error handling.



CanTp

(CAN Transport Layer) The CanTp module conforms to ISO standard 15765-2. As the transport protocol for CAN, it is responsible for segmenting the data in the Tx direction, collecting data in the Rx direction and monitoring the data stream.

CanTrcv

(CAN Transceiver Driver) This driver is responsible for controlling the operating states of an external CAN transceiver. It contains control of wake up and sleep functions.

CanTSyn

(Time Sync Over CAN) This module realizes the CAN specific time synchronization protocol. An access to the synchronized time base by the SWCs requires the Synchronized Time-Base Manager ((StbM).

CanXcp / XCPonCan

(CAN XCP-Module) The CanXcp module contains CAN-specific contents of the XCP module ((Xcp).

CDD

(Complex Drivers) The Complex Drivers are software modules that are not standardized by AUTOSAR. They have access to other BSW modules, the RTE and direct hardware access. For example, these modules are not standardized communication drivers for SPI or legacy software.

Com

(Communication) The Com module provides a signal-based data interface for the RTE. It places signals in messages and sends them according to the defined send type. The module contains various notification mechanisms for receiving signals. It also manages initial values, update bits and timeouts on the signal level. In multi-channel ECUs, the integrated signal gateway offers the ability to route signals between the communication buses.

ComM

(Communication Manager) The ComM module coordinates the communication channels and Partial Network Cluster with the communication requirements of the application.

ComXf

(COM Based Transformer) The COM based transformer optimizes for signal groups the interaction between RTE and the COM module, because you can avoid the shadow buffer in the COM module. It de-/serializes the data identical to the COM module.

CorTst

(Core Test Driver) The CorTst module contains configuration and control of the test capabilities included in the microcontroller core. In addition, it offers a framework for extending these test capabilities. Specifically for safety-related software, the CorTst module tests critical units such as the ALU and the registers.

Crc

(CRC Routines) The Cyclic Redundancy Check library calculates CRC checksums.



Csm (Cry)

(Crypto Service Manager) The Csm module offers SWCs access to basic cryptographic functions. The individual cryptographic functions are implemented by the Cry module in software or hardware (access via (She).

Dbg

(Debugging) The debugging module enables external access to internal information of the basic software. It is also possible to modify memory data.

Dcm

(Diagnostic Communication Manager) The Dcm module implements diagnostic communication according to ISO 14229-1:2006 (UDS). Some diagnostic requests are processed directly in the Dcm (management of diagnostic sessions, reading of error codes, EcuReset, etc.) and some are routed to the SWCs via port interfaces (reading, writing and controlling of data elements within a data identifier, execution of routines, etc.). Legal requirements of OBDII / SAE J1979 are also supported.

Dem

(Diagnostic Event Manager) The Dem module implements a fault memory. The standardized interface for "DiagnosticMonitors" enables uniform development of manufacturer-independent SWCs. The Dem module is responsible for administering the DiagnosticTroubleCode states, environmental data, and for storing the data in NVRAM. The legal requirements of OBDII / SAE J1979 are also supported.

Det

(Development Error Tracer) The Det module supports error debugging during software development. It provides an interface for error notification, which is called by the individual BSW modules in case of error.

Dio

(Digital Input Output Driver) The Digital Input Output Driver provides read and write services for the DIO channels (pins), DIO ports and DIO channel groups.

Dlt

(Diagnostic Log and Trace) The Dlt module provides generic "Logging and Tracing" functionality for SWCs and for the BSW modules (Rte, (Det and (Dem.

DNS

(Domain Name System) The DNS module contains a DNS resolver. It is responsible for resolving a domain, e.g. vector.com, into a valid IP address.

DoIP

(Diagnostics over IP) Since AR 4.1.1 the DoIP module contains diagnostic functionality according to ISO 13400-2 like vehicle discovery. Up to and including AR 4.0.3, this functionality is part of the Socket Adaptor ((SoAd).

DrvExt

(External Driver) Upon request, you can obtain the implementation of drivers for externally connected components. They are already available for plenty of external devices, e.g. for driv-



ing certain EEPROMs (EEPEXT), flash chips (FLSEXT), watchdogs (WDGEXT), analog-digital converters (ADCEXT) and communication controllers (CANEXT, LINEXT, ETHSWTEXT).

E2E

(End-to-End Communication Protection Library) Library for secure data exchange according to ISO 26262 for safety-related ECUs. It is responsible for calculating the checksum and providing the message counter.

E2EPW

(End-to-End Protection Wrapper) The E2E Protection Wrapper extends the RTE by adding verification of safety-related signals.

E2EXf

(E2E Transformer) With the E2E Transformer, you integrate the protection of safety-relevant signals according to ISO 26262 in the RTE interface. In contrast to the E2E protection wrapper, the standard interfaces of the RTE are used.

Ea

(EEPROM Abstraction) The Ea module offers a hardware-independent interface for accessing EEPROM data and uses an EEPROM driver ((Eep) for this. In addition to reading, writing and clearing data, the Ea module also distributes write accesses to different areas of the EEPROM, so that all EEPROM cells are uniformly stressed, which increases their life-time.

EcuM

(ECU State Manager) The ECU State Manager is responsible for starting up and shutting down the ECU as well as waking it up. It is available in two variants: flexible and fixed. The EcuM Fix manages a number of defined, fixed operating states. Using the EcuM Flex, the user defines all operating states flexibly in the (BswM module. This lets you implement special energy-saving states and various types of startup. In a multi-core system, the EcuM coordinates the various cores.

Eep

(EEPROM Driver) The EEPROM driver enables hardware-independent access to EEPROM memory. It provides services for reading, writing and comparing data.

Efx

(Extended Fixed Point Routines) Library with extended mathematical functions for fixed-point values

Eth

(Ethernet Driver) The Ethernet Driver abstracts access to the Ethernet hardware for sending and receiving data and for switching between controller states.

EthIf

(Ethernet Interface) The Ethernet Interface enables bus-independent control of the Ethernet driver ((Eth) and Ethernet transceiver driver ((EthTrcv). Since AR 4.1, this module is also responsible for VLAN handling.



EthSM

(Ethernet State Manager) The Ethernet State Manager provides the Communication Manager ((ComM) with an abstract interface for starting up or shutting down communications in Ethernet clusters. EthSM accesses the Ethernet hardware via (EthIf.

EthSwt

(Ethernet Switch Driver) The module EthSwt provides a uniform and hardware independent interface for controlling and configuring Ethernet switches. It also coordinates the MAC learning when using multiple identical ECUs like cameras for the surround view.

EthTrcv

(Ethernet Transceiver Driver) EthTrcv offers a uniform and hardware-independent interface for driving multiple transceivers of the same kind. Configuration of EthTrcv is transceiver-specific and considers the properties of the physical network that is used.

EthTSyn

(Time Sync Over Ethernet) This module realizes the Ethernet specific time synchronization protocol and references to IEEE Standard 802.1AS ((PTP). An access to the synchronized time base by the SWCs requires the Synchronized Time-Base Manager ((StbM).

EthXcp/ XCPonEth

(Ethernet XCP-Module) The EthXcp module contains Ethernet-specific contents of the XCP module ((Xcp).

EXI

(Efficient XML Interchange) The EXI module is used to interpret XML document and to convert them to a binary format. This makes it more efficient to process the files and to transmit them, in order to economize on communication bandwidth. It is used in the Vehicle2Grid environment.

Fee

(Flash EEPROM Emulation) The Fee module offers a hardware-independent interface for accessing flash data and uses a flash driver ((Fls) for this. In addition to reading, writing and clearing data, the Fee module also distributes write accesses to different areas of flash memory, so that all flash cells are uniformly stressed, which increases their life-time.

FiM

(Function Inhibition Manager) Based on the active errors managed by the (Dem module, the FiM offers the ability to prevent execution of functionalities in SWCs.

Fls

(Flash Driver) The flash driver enables hardware-independent and uniform access to flash memory. It provides services for reading, writing and comparing data, and for deleting blocks (sectors).

FlsTst

(Flash Test) The Flash Test module offers algorithms for testing nonvolatile memory such as data or program flash, SRAM and protected cache.



Fr

(FlexRay Driver) The FlexRay Driver abstracts access to the FlexRay hardware for sending and receiving data and for switching between controller states.

FrArTp

(FlexRay AUTOSAR Transport Layer) FrArTp is a FlexRay transport protocol. Based on ISO 15765-2 ((CanTp), it contains a frame compatibility with the CAN bus.

FrIf

(FlexRay Interface) The FlexRay Interface offers abstracted (PDU-based) access to the FlexRay hardware. In addition, it offers support for synchronization with the global FlexRay time.

FrNm

(FlexRay Network Management) This module is responsible for network management in FlexRay. It synchronizes the transition to the bus sleep state.

FrSM

(FlexRay State Manager) The FlexRay State Manager controls and monitors the wake up and startup of nodes in the FlexRay cluster.

FrTp

(FlexRay ISO Transport Layer) FrTp is a FlexRay transport protocol and is based on the ISO 10681-2 standard.

FrTrcv

(FlexRay Transceiver Driver) The driver for an external FlexRay transceiver is responsible for switching the transceiver on and off.

FrTSyn

(Time Sync Over FlexRay) This module realizes the FlexRay specific time synchronization protocol. An access to the synchronized time base by the SWCs requires the Synchronized Time-Base Manager ((StbM).

FrXcp/ XCPonFr

(FlexRay XCP-Module) The FrXcp module contains FlexRay-specific contents of the XCP module ((Xcp).

Gpt

(General Purpose Timer Driver) The General Purpose Timer Driver provides an interface for accessing internal timers of the microcontroller. It is used for controlling periodic and singular events, for example.

HTTP

(Hypertext Transfer Protocol) The Hypertext Transfer Protocol, for instance, routes browser requests to a server. The module contains a HTTP client. It is used in the Vehicle2Grid environment.



Icu

(Input Capture Unit Driver) The Icu driver provides services for edge detection, measurement of periodic signals, assignment of edge time stamps and control of wake up interrupts.

IfI

(Interpolation Floating Point) Library with interpolation functions for floating point values

Ifx

(Interpolation Fixed Point) Library with interpolation functions for fixed point values

IicDrv

(I²C Driver) The I²C Driver provides services for communication with external I2C chips.

IoHwAb

(I/O Hardware Abstraction) The I/O Hardware Abstraction represents the connection between the RTE and the ECU's I/O channels. It encapsulates access to the I/O drivers such as (Adc, (Dio or (Pwm and thereby makes the I/O signals of the ECU available to the SWCs.

IpduM

(I-PDU Multiplexer) This module is responsible for multiple use of fixed PDUs with different data contents.

J1939Dcm

(SAE J1939 Diagnostic Communication Manager) The module implements Diagnostic Messages of the SAE J1939-73 protocol, e.g. for reading out the fault memory.

J1939Nm

(SAE J1939 Network Management) J1939 supports adding ECUs to networks on-the-fly. The J1939Nm module is responsible for negotiating a unique ECU address ("AddressClaim") and unlike other NM modules for handling the wake-up or going to sleep of the bus.

J1939Rm

(SAE J1939 Request Manager) The J1939Rm module implements requesting of data via Request Handling that is defined in the SAE J1939 protocol.

J1939Tp

(SAE J1939 Transport Layer) The J1939TP module contains the transport protocols BAM (Broadcast Announce Message) and CMDT (Connection Mode Data Transfer) of the SAE J1939 standard.

LdCom

(Large Data COM) The LdCom module is optimized for routing large signals. Thereby it avoids an unnecessary copying of data. The LdCom is typically used together with (SomeIpXf as a serialization transformer.

Lin

(LIN Driver) The LIN Driver provides services for initiating frame transmission (header, response, sleep-mode and wake up), and for receiving responses, checking the current state and validating wake up events.



LinIf

(LIN Interface) The LIN Interface offers abstracted (PDU-based) access to the LIN hardware. It also handles schedule table processing and contains the LIN transport protocol ((LinTp).

LinNm

(LIN Network Management) The LinNm module contains a hardware-independent protocol, which coordinates the transition between normal operation and the bus sleep mode of the LIN network.

LinSM

(LIN State Manager) The LIN State Manager switches between schedule tables and PDU groups in the (Com module and services the LIN interface with regard to sleep and wake up.

LinTp

(LIN Transport Layer) The LIN transport protocol is responsible for segmenting data in the Tx direction, collecting data in the Rx direction and monitoring the data stream. According to the AUTOSAR specification, LinTp is part of (LinIf.

LinTrcv

(LIN Transceiver Driver) The module for an external LIN transceiver is responsible for monitoring and driving the wake up and sleep functions.

LinXcp

(LIN XCP-Module) The LinXcp module contains LIN-specific contents of the XCP module ((Xcp).

Mcu

(Micro Controller Unit Driver) The MCU driver provides the services for: - A microcontroller reset triggered by software - Selecting microcontroller states (STOP, SLEEP, HALT, etc.) - Configuring wake up behavior - Managing the internal PLL clock unit - Initializing RAM areas with predefined values.

MemIf

(Memory Abstraction Interface) This module provides uniform access to the services of (Ea and (Fee. This lets you use multiple instances of these modules.

Mfl

(Mathematical Floating Point) Library with arithmetic functions for floating point values

Mfx

(Mathematical Fixed Point) Library with arithmetic functions for fixed point values

Nm

(Generic Network Management Interface) The Nm module offers a general and network-independent interface for accessing the bus-dependent network management modules ((CanNm, (LinNm, (UdpNm and (FrNm). In addition, the module handles synchronous, inter-network shutdown of the communication system in coordination with the other ECUs.

NvM

(Non Volatile RAM Manager) The NvM module manages, reads and writes data to a nonvolatile memory ((Ea or (Fee). At system start and at shutdown, it synchronizes the data in the RAM



areas of the application. The module provides services such as saving of redundant blocks for a higher level of data protection. Since AR 4.0.3, the (RTE also provides a simpler and more flexible interface to Nv data (NvDataInterfaces).

Ocu

(Output Compare Unit Driver) The OCU driver standardizes initialization and access to the Output Compare Unit.

Os

(Operating System) This module is the operating system of an AUTOSAR ECU. It is actually an extended OSEK operating system. Extensions are divided into so-called Scalability Classes (SC1-SC4). They cover the following functionalities: - SC1: schedule tables - SC2: timing protection + schedule tables - SC3: memory protection + schedule tables - SC4: memory protection + timing protection + schedule tables In safety-related ECUs, the OS is one of the safety-related modules. The operating system also supports multi-core microcontrollers.

PduR

(PDU Router) The PDU Router handles distribution of the communication packets (PDUs) between the bus systems and the various BSW modules. In addition it offers gateway mechanisms for routing PDUs (FIFO) and TP-PDUs (on-the-fly) between the bus systems.

Port

(Port Driver) This module is responsible for initialization of the entire port structure of the microcontroller.

PTP

(Precision Time Protocol) The PTP module is used to distribute a precise system time to all devices participating in an AVB network. It is implemented according to IEEE 802.1AS. The implementation consists of microprocessor-dependent and independent parts.

Pwm

(Pulse Width Modulation Driver) The Pwm driver provides services for initialization and control of the PWM (pulse-width modulation) channels of the microcontroller.

RamTst

(Ram Test) This module tests internal microcontroller RAM cells. A complete test is triggered during startup and shutdown of the ECU or by a diagnostic command. During normal operation, a periodic test is performed (block-by-block or cell-by-cell).

Rte

(Runtime Environment) The RTE implements the Virtual Functional Bus and the execution of the SWCs. It also ensures consistent data exchange between the SWCs themselves and between the SWCs and the basic software. The execution of the basic software is realized by the integrated (SchM. The RTE also supports communication beyond partition boundaries (multi-core/trusted/untrusted). In addition, it offers simplified access to NVRAM data and calibration data. In safety-related ECUs, the RTE is one of the safety-related modules.



Rtm

(Runtime Measurement) You use the Rtm module to determine the runtime and CPU load of BSW modules and of application code. The module is typically used during development.

SCC

(Smart Charge Communication) This module is responsible for smart charge communication according to ISO 15118 or DIN 70121 and contains the V2GTP transport protocol that is used for this protocol. It supports DC and AC charging and the associated profiles Plug-and-Charge (PnC) and External Identification Means (EIM).

SchM

(BSW Scheduler) The SchM module is integrated in the (RTE, calls the periodic "main" function of the individual BSW modules, and provides functions for critical sections. For the distribution of the BSW (master satellite concept) via partitions and core boundaries, the SchM provides nearly identical communication interfaces like the (RTE.

Sd

(Service Discovery) Service Discovery was first specified in AR 4.1.1. An ECU communicates the availability of its services to communication partners via the publish-subscribe protocol implemented in this module. In addition, ECUs can register to receive automatic notifications, e.g. on a signal update.

SecOC

(Secure Onboard Communication) The SecOC is used to send or receive authenticated messages. Unauthorized, repeated or manipulated messages are detected. The SecOC is part of the AUTOSAR security solution.

She

(Secure Hardware Extension) The She driver implements the cryptographic hardware functions of a Hardware Security Module (HSM). This functionality is used by the (Csm module, for instance.

SoAd

(Socket Adaptor) The socket adaptor converts the communication via PDUs as it is defined in AUTOSAR into a socket-based communication. In AR 4.0, the SoAd also contains the diagnostic functionality defined in ISO 13400-2 ((DoIP). Since AR 4.1, this plug-in is removed and specified as a stand-alone module ((DoIP).

SomeIpXf

(SOME/IP Transformer) The module SOME/IP Transformer is an RPC and serialization protocol. It is used to call methods on other ECUs, which for example were made known in the system beforehand via Service Discovery ((Sd).

Spi

(SPI Handler/ Driver) The SPI Driver handles data exchange over the SPI interface. It is primarily used in conjunction with external hardware such as EEPROM, Watchdog, etc.

StbM

(Synchronized Time-Base Manager) The Synchronized Time-Base Manager enables time synchronization. Since AR 4.2 a time base prescribed by the Time Master can be synchronized



with other ECUs via bus systems.

TcpIp

(TCP/IP Stack) This module contains all protocols for UDP and TCP-based communication. It supports the versions IPv4 and IPv6 as well as parallel operation of IPv4 and IPv6 in one ECU. It contains the following protocols: - IPv4, ICMPv4 and ARP - IPv6, ICMPv6 and NDP - UDP, TCP, DHCPv4 (client) and DHCPv6 (client)

TLS

(Transport Layer Security) This module contains a Transport Layer Security client. The TCP-based communication is encrypted with TLS. The encryption algorithm used can be selected.

Tm

(Time Services) The Tm module is used for such tasks as measuring execution times and functions and implementing active waiting. It offers a resolution from 1µs to 4.9 days.

Ttcan

(TTCAN Driver) The TTCAN Driver offers the same functionality for a TTCAN controller (ISO 11898-4) as the CAN Driver ((Can) does for a CAN controller.

TtcanIf

(TTCAN Interface) The TTCAN Interface offers the same functionality for a TTCAN controller (ISO 11898-4) as the CAN Interface ((CanIf) does for a CAN controller.

UdpNm

(UDP Network Management) You use network management over UDP to implement synchronous transition to sleep mode for Ethernet ECUs.

Wdg

(Watchdog Driver) This module provides services for controlling and triggering the watchdog hardware. The trigger routine is called by the Watchdog Manager ((WdgM). For safety-related ECUs, the Wdg module must be developed according to ISO 26262.

WdgIf

(Watchdog Interface) This module enables uniform access to services of the Watchdog Driver ((Wdg), such as mode switching and triggering. For safety-related ECUs, the WdgIf module must be developed according to ISO 26262.

WdgM

(Watchdog Manager) The Watchdog Manager monitors the reliability and functional safety of the applications in an ECU. This includes monitoring for correct execution of SWCs and BSW modules and triggering of the watchdogs at the required time intervals. The WdgM module reacts to potential faulty behavior with multiple escalation stages. An important fact for safety-related functions according to ISO 26262 is the monitoring of correct flow sequences of critical tasks (logical supervision). For safety-related ECUs, the WdgM must be developed according to ISO 26262.

Xcp

(Universal Measurement and Calibration Protocol) XCP is a protocol for communication between a master (PC tool) and a slave (ECU). It is standardized by ASAM and is used



primarily for measuring, calibrating, flashing and testing ECUs. XCP supports the bus systems CAN ((CanXcp), FlexRay ((FrXcp), Ethernet ((EthXcp) and LIN ((LinXcp).

XML Security

(XML Security) You use this module to generate and validate XML signatures to EXI-encoded data based on the W3C XML security standard. It is used in the Vehicle2Grid environment.



5 Index

A

Alarm 149

Assistant

 Component Connection Assistant 56

 Data Mapping Assistant 58

 Project Assistant 52, 56, 82, 128, 130

 Task Mapping Assistant 61

B

Basic Editor 49, 91-92, 139, 148, 158, 162,
 173

BSW Modules

 CANIF 38, 177

C

Configuration Editors 36, 92

 Base Services 36

 Communication 21, 37, 77, 88

 Diagnostics 39, 77

 Memory 40, 75, 114, 125, 141, 147, 175

 Mode Management 42

 Network Management 47

 Runtime System 48, 56, 144

D

DaVinci Configurator

 Input Files 26, 126, 153

 On-demand Validation 50

 Project Settings 19, 26, 84, 88, 125, 127,
 151, 157

E

ECU Instance 154

Event 39

I

Installation Guide 20

P

Project folder 129

R

Runnables 62, 75, 93, 144

S

Start Menu 25

T

Task 49, 61, 114



Get more Information!

Visit our Website for:

- > News
- > Products
- > Demo Software
- > Support
- > Training Classes
- > Addresses

www.vector.com